

Course Notes: Deep Learning and Approximation theory

Nadav Dym

October 5, 2026

Preface

These are lecture notes for the course ‘Deep Learning and Approximation Theory’, taught in the Technion in the spring of 2026. The course is intended for graduate and advanced undergraduate students. Its goal is to introduce students to modern research which focuses on understanding the approximation power of neural networks, with a special emphasis on group equivariant networks.

The course is divided into four chapters. Chapters 1-2 focus on fully connected neural networks. The first chapter proves the well known results that fully connected neural networks can approximate any continuous function. The second chapter deals with the properties of ReLU neural networks, including the benefit of deep networks in increasing expressivity, and quantitative approximation results for ReLU networks.

Chapters 3-4 focus on Equivariant Learning, or Geometric Deep Learning, with a focus on approximation results. Chapter 3 discusses the general notion of equivariance and invariance, and focuses on universality results for invariant networks for multisets. Chapter 4 discusses graph neural networks and their separation and approximation properties.

Some apologies and disclaimers are in order: firstly, not all course material appears in the notes: throughout the course I showed some slides motivating the problems discussing these questions. Some of the statements in the text are referred to homework, which will not be available to the reader. Secondly, the choice of topics and references in this manuscript may not always be the most central and common in the field. Thirdly, though I am confident in the overall correctness of the results discussed in these notes, they have not been seriously proof read, and they are bound to contain more errors and inconsistencies than a textbook. I will be happy to receive feedback via email on any errors and I will try to fix them and update the notes.

Despite the above disclaimers, I am publishing these notes online. For me, teaching this course in the past five years was an opportunity to organize and unify several interesting results in my research area into a hopefully consistent and coherent story. I hope that this effort will be of some use to other students and researchers interested in these topics.

Contents

0	Some math preliminaries	5
0.0.1	Normed function spaces	7
0.0.2	The Stone Weierstrass Theorem	10
1	Neural Networks and Universality	13
1.1	Introduction	13
1.1.1	Supervised Learning	13
1.1.2	Neural Network Functions	14
1.1.3	Neural Network Architectures	14
1.1.4	The ‘Neural’ Terminology	15
1.2	Universality of Neural Networks	16
1.2.1	Proof of Universality of Shallow Neural Networks with smooth activations	18
1.2.2	Universality: from smooth to continuous	21
2	ReLU Neural Networks	25
2.0.1	Basic approximation properties of ReLU networks	25
2.0.2	Expressing functions as ReLU networks: first examples	27
2.1	ReLU Networks, CPwL Functions and Depth Separation	27
2.1.1	ReLU networks are Continuous Piecewise Linear functions	27
2.1.2	Depth separation and counting linear regions	31
2.1.3	Shallow univariate neural networks	31
2.1.4	Deep Univariate Neural Networks	33
2.2	Approximating polynomials and smooth functions	35
2.2.1	Approximating x^2	35
2.2.2	Approximating multiplication	37
2.2.3	Approximation of univariate smooth functions	39
2.2.4	Additional results	42
3	Equivariant Learning and Approximation Theory	43
3.1	Basic notions	43
3.1.1	Permutation equivariance for point sets	46
3.1.2	Invariant pooling operations	47
3.1.3	Invariant functions and functions on quotient spaces	49
3.1.4	Separating Invariants and Universality	50
3.1.5	Universal Equivariant models	54
3.1.6	Multisets, sets, and permutation invariant functions	56
4	Graph Neural Networks and Graph Isomorphism tests	59
4.0.1	Symmetries of graphs	59
4.0.2	GIN*	69
4.0.3	Maximally expressive MPNN using analytic activations	71
4.0.4	k -WL Tests and Graph Neural Networks	75
4.0.5	Geometric graphs and point clouds	77

4.0.6	Generic recovery for sparse graphs	82
4.0.7	Equivariant Geometric MPNN	82

Chapter 0

Some math preliminaries

Lesson 2

The first goal of this lesson is to show some fundamental facts about approximation: basically that polynomials are dense in the space of continuous functions, while polynomials of finite degrees are not. The secondary goal is to clarify some of the math background necessary for this course, and help close the gaps to those who need to. For additional reading see [Simmons, 1963].

Definition 0.1. Let X be a vector space over $\mathbb{R}$. A norm is a function $\|\cdot\| : X \rightarrow \mathbb{R}$ satisfying

1. Positivity: For every $x \in X$, $\|x\| \geq 0$, with equality if and only if $x = 0$.
2. Triangle inequality: $\|x + y\| \leq \|x\| + \|y\|$ for every $x, y \in X$.
3. Homogeneity: $\|\lambda x\| = |\lambda| \|x\|$ for all $x \in X$ and $\lambda \in \mathbb{R}$.

Example 0.2. In $\mathbb{R}^n$, we have the norms

1. the standard 2 norm $\|x\|_2 = \sqrt{\sum_{i=1}^n (x_i)^2}$.
2. the 1 norm $\|x\|_1 = \sum_{i=1}^n |x_i|$
3. In general, for $p \geq 1$ we can define the norm $\|x\|_p = (\sum_{i=1}^n |x_i|^p)^{1/p}$
4. The ∞ norm $\|x\|_\infty = \max_{1 \leq i \leq n} |x_i|$.

We also say that $(X, \|\cdot\|)$ is a normed space. Later today we will use the reverse triangle inequality:

$$|||x| - |y|| \leq \|x - y\|$$

which follows from the triangle inequality (check).

Definition 0.3. Let X be a non-empty set. A metric d is a function $d : X \times X \rightarrow \mathbb{R}$ which satisfies the following conditions

1. Positivity: for every $x, y \in X$, $d(x, y) \geq 0$ with equality if and only if $x = y$.
2. Symmetry: for every $x, y \in X$, $d(x, y) = d(y, x)$.
3. Triangle inequality: for every $x, y, z \in X$

$$d(x, z) \leq d(x, y) + d(y, z).$$

We also say that (X, d) is a metric space.

Example 0.4. 1. For any non-empty set X , we can define the ‘discrete metric’ $d(x, x) = 0$ and $d(x, y) = 1$ if $x \neq y$.

2. On $X = \mathbb{R}$ we have the metric $d(x, y) = |x - y|$.

3. On $X = \mathbb{R}^n$ we have the metric $d_2(x, y) = \|x - y\|_2 = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$, and more generally $d_p(x, y) = \|x - y\|_p$

4. If (X, d) is a metric space, and $Y \subseteq X$, then (Y, d) is a metric space.

As examples 2-3 suggest, norms induce metrics:

Proposition 0.5. *If $(X, \|\cdot\|)$ is a normed space then $d(x, y) = \|x - y\|$ is a metric on X .*

Problem 0.6. The discrete metric on $\mathbb{R}$ is not induced from a norm. Explain why.

Definition 0.7 (Convergence). Let (X, d) be a metric space. We say that a sequence $x_n, n = 1, 2, \dots$ of elements in X converges to x if

$$d(x_n, x) \rightarrow 0$$

If a limit of a sequence exists, it is unique. That is, if x_n converges to x , and x_n also converges to y , then $x = y$ (verify).

Definition 0.8 (Cauchy sequences). Let (X, d) be a metric space. We say that a sequence $x_n, n = 1, 2, \dots$ of elements in X is Cauchy if for every $\epsilon > 0$ there exists N such that for all $n, m \geq N$ we have $d(x_n, x_m) < \epsilon$.

A converging sequence is always a Cauchy sequence: If $x_n \rightarrow x$ then for all $\epsilon > 0$ there exists N such that for all $n > N$ we have $d(x_n, x) < \epsilon/2$, and so for $n, m > N$ we use the triangle inequality to obtain

$$d(x_n, x_m) \leq d(x_n, x) + d(x_m, x) < \epsilon/2 + \epsilon/2 = \epsilon$$

Definition 0.9. A metric space is *complete* if every Cauchy sequence has a limit.

For example $\mathbb{R}^d$ and closed subsets of $\mathbb{R}^d$ are complete, while the set $[0, 1)$ or the set of rational numbers are not complete.

Definition 0.10. Let (X, d) be a metric space. For $x_0 \in X$ and $r > 0$, the open ball $B_r(x_0)$ is the set

$$B_r(x_0) = \{x \in X \mid d(x, x_0) < r\}$$

Definition 0.11 (Open sets). Let (X, d) be a metric space. We say that $U \subseteq X$ is an open set if for all $x \in U$ there exists some $r > 0$ such that $B_r(x) \subseteq U$.

Example 0.12. 1. An open ball is an open set.

2. In $\mathbb{R}^d$, the hyper-cube $(0, 1)^d$ is an open set.

Definition 0.13 (Closed sets). Let (X, d) be a metric space. We say that a set $C \subseteq X$ is closed if for every sequence $x_n \in C$ which converges to a point $x \in X$, we have that $x \in C$.

Problem 0.14. Show that a set $C \subseteq X$ is closed if and only if its complement is open.

Definition 0.15 (Bounded sets). Let (X, d) be a metric space. We say that a set $C \subseteq X$ is bounded, if there exists some $r > 0$ and $x_0 \in X$ such that $C \subseteq B_r(x_0)$.

Definition 0.16 (Compactness). Let (X, d) be a metric space. We say that a set $K \subseteq X$ is compact if every sequence $x_n \subseteq K$ has a subsequence which converges to a limit in K .

We note that a compact set is always bounded and closed. In $\mathbb{R}^n$, the converse is also true, but this is not true for all metric spaces. You will give an example of this in homework.

Example 0.17. The set $(0, 1]$ is not a compact subset of $\mathbb{R}$ because the sequence $1/n$, and all its subsequences, converge to a point which is not in the set. The set $[0, 1]$ is compact. In general in $\mathbb{R}^n$ (with the standard metric) a set is compact if and only if it is closed and bounded. This is not true for all metric spaces, as we will see in homework.

Definition 0.18 (Continuity). Let (X, d_X) and (Y, d_Y) be metric spaces. We say that a function $f : X \rightarrow Y$ is continuous if for every sequence $x_n \rightarrow x$ in X , we have that $f(x_n) \rightarrow f(x)$ in Y .

Proposition 0.19. Let (X, d_X) and (Y, d_Y) be metric spaces. If $f : X \rightarrow Y$ is continuous and $K \subseteq X$ is compact, then $f(K)$ is compact.

Proof. We need to show that $f(K)$ is compact. Let y_n be a sequence in $f(K)$. There exist $x_n \in K$ such that $f(x_n) = y_n$. By compactness of K , the sequence x_n has a subsequence x_{n_k} which converges to some $x \in K$. By continuity

$$y_{n_k} = f(x_{n_k}) \rightarrow f(x) \in f(K)$$

and so we showed that y_n has a converging subsequence, and thus $f(K)$ is compact. $\square$

Problem 0.20. Give an example of a continuous function $f : \mathbb{R} \rightarrow \mathbb{R}$ and a compact set $K \subseteq \mathbb{R}$ such that $f^{-1}(K)$ is not compact.

Corollary 0.21. Let (X, d) be a metric space. $K \subseteq X$ compact, and $f : X \rightarrow \mathbb{R}$ a continuous function. Then f obtains a maximum and minimum on K .

Proof. By the previous proposition $f(K)$ is a compact subset of $\mathbb{R}$ and so it is bounded: there exists some $M > 0$ such that $|f(x)| < M$. It follows that $S = \sup_{x \in K} f(x)$ is finite. There exists a sequence $x_n \subseteq K$ such that $f(x_n) \rightarrow S$. By compactness x_n has a subsequence which converges to $x \in K$ and by continuity $f(x) = S$ so x is a maximum. The same argument can be used to show the existence of a minimum. $\square$

0.0.1 Normed function spaces

Let $K \subseteq \mathbb{R}^d$ be a compact subset. Denote

$$C(K) = \{f : K \rightarrow \mathbb{R} \mid f \text{ is continuous}\}.$$

The set $C(K)$ is a vector space, where the zero is the zero function, addition is standard function addition $(f+g)(x) = f(x) + g(x)$ and scalar-vector multiplication is the standard multiplication by scalars $(cf)(x) = c \cdot f(x)$.

We can define a norm on $C(K)$ by

$$\|f\| = \max_{x \in K} |f(x)|$$

by the previous corollary the maximum is indeed well defined. We can verify that this is indeed a norm. Positivity and homogeneity are rather obvious. As for the triangle inequality: For $f, g \in C(K)$ and for all $x \in K$

$$|f(x) + g(x)| \leq |f(x)| + |g(x)| \leq \|f\| + \|g\|$$

and since this inequality is true for all x it is also true for the x which maximize $|f(x) + g(x)|$.

Problem 0.22. Give an example of f, g for which (i) the inequality is strict and (ii) the inequality is an equality.

The norm on $C(K)$ induces the metric $\|f - g\| = \max_{x \in K} |f(x) - g(x)|$ on functions. This metric space will be central in this course. We will consider questions such as: given a subset of function $P \subseteq C(K)$, can any function $f \in C(K)$ be approximated by functions in P ? When this happens, we will say that P is dense in $C(K)$. This notion is formally defined as follows

Definition 0.23 (Dense subsets). Let C be a normed vector space, we say that a subspace P is dense in C if for every $f \in C$ and $\epsilon > 0$ there is some $p \in P$ such that $\|f - p\| < \epsilon$.

Remark 0.24. Note that P is dense in a normed space C , if and only if, for all $f \in C$, there exists a sequence $(p_n)_{n=1}^\infty$ such that p_n converges to f .

In our context, C will be $C(K)$, the subset P is a set of functions we are interested in (polynomials, neural networks), and the norm is the uniform function norm, that is

$$\|f - p\| = \max_{x \in K} |f(x) - p(x)|$$

The convergence of a sequence p_n to f in the norm of $C(K)$, is exactly the notion of uniform convergence which you learned in previous courses.

As a first example of such a denseness result, we quote Weierstrass's theorem

Theorem 0.25 (Weierstrass). *For every compact $K \subseteq \mathbb{R}$ the space of polynomials P is dense in $C(K)$.*

We will later discuss this theorem's generalization, the Stone-Weierstrass theorem. In particular, this will show that multivariate continuous functions $f : \mathbb{R}^d \rightarrow \mathbb{R}$ can be approximated uniformly by $p : \mathbb{R}^d \rightarrow \mathbb{R}$. Later on we will also show that polynomials can be replaced by neural networks to get the same approximation properties.

For now, we want to show that we *cannot* approximate a continuous function by polynomials of degree $\leq D$. Denote

$$\mathbb{R}_D[x] = \{ \text{polynomials of degree } \leq D \}$$

This is a vector space of dimension $D + 1$, with the norm $\|f\|_{C[a,b]} = \max_{x \in [a,b]} |f(x)|$. We will show that all finite dimensional normed spaces are 'topologically equivalent' and through this show that they cannot approximate all functions. In contrast, note that the space of *all polynomials* $\mathbb{R}[x]$ is infinite dimensional.

Definition 0.26. Let $(X, \|\cdot\|_X)$ and $(Y, \|\cdot\|_Y)$ be normed spaces over $\mathbb{R}$. We say that a map $T : X \rightarrow Y$ is an isomorphism of normed spaces if T is linear, onto, and there exists $0 < m < M$ such that

$$m\|x\|_X \leq \|Tx\|_Y \leq M\|x\|_X.$$

If $T : X \rightarrow Y$ is an isomorphism of normed spaces, then T has an inverse $T^{-1} : Y \rightarrow X$ which is also an isomorphism of normed spaces.

Problem 0.27. Show that the identity map $Ix = x$ is a isomorphism between $(\mathbb{R}^n, \|\cdot\|_\infty)$ and $(\mathbb{R}^n, \|\cdot\|_1)$. What are the constants? Can you do the same when ∞ is replaced with 2?

Solution. We have that

$$\|x\|_1 = \sum_{i=1}^n |x_i| \geq \max_i |x_i| = \|x\|_\infty,$$

and on the other hand

$$\|x\|_1 = \sum_{i=1}^n |x_i| \leq \sum_{i=1}^n |x|_\infty = n|x|_\infty,$$

and so we obtained

$$m\|x\|_\infty \leq \|x\|_1 \leq M\|x\|_\infty.$$

with $m = 1$ and $M = n$. Note that these constants are optimal: we cannot take M to be smaller, or m to be larger, as shown by considering

$$x = [1, 0, 0, \dots, 0] \text{ and } x = [1, 1, \dots, 1]$$

□

We say that $(X, \|\cdot\|_X)$ and $(Y, \|\cdot\|_Y)$ are isomorphic if there exists an isomorphism between them.

As it turns out, there is always an isomorphism between normed spaces of the same finite dimension.

Theorem 0.28. *Let $(V, \|\cdot\|)$ be a vector space over $\mathbb{R}$ of dimension n . Then there is a isomorphism between V and $(\mathbb{R}^n, \|\cdot\|_2)$.*

Proof. Let $v_1, \dots, v_n$ be a basis for V , and $e_1, \dots, e_n$ denote the standard basis for $\mathbb{R}^n$. We define a linear map $T : \mathbb{R}^n \rightarrow V$ by

$$T(x) = \sum_{i=1}^n x_i v_i.$$

The map T is linear. It is onto since $v_1, \dots, v_n$ is a basis. It is also one-to-one, for $T(x) = 0$ implies that all x_i are zero. It remains to show that the norm $\|Tx\|$ can be bounded from above and below by $\|x\|$ up to some constant. Define

$$M = \sqrt{\sum_{i=1}^n \|v_i\|^2}$$

we have, using the properties of the norm and Cauchy-Schwarz, that

$$\|Tx\| = \left\| \sum_{i=1}^n x_i v_i \right\| \leq \sum_{i=1}^n \|x_i v_i\| = \sum_{i=1}^n |x_i| \|v_i\| \leq \sqrt{\sum_{i=1}^n |x_i|^2} \sqrt{\sum_{i=1}^n \|v_i\|^2} = M \|x\|_2$$

this gives us the first inequality we need. We can also use it to get the other direction: the inequality we just proved shows that the map

$$N : \mathbb{R}^n \rightarrow \mathbb{R}, N(x) = \|Tx\|$$

is continuous. For if $x_n \rightarrow x$ then, using the reverse triangle inequality:

$$|N(x_n) - N(x)| = \left| \|Tx_n\| - \|Tx\| \right| \leq \|T(x_n - x)\| \leq M \|x_n - x\|_2 \rightarrow 0$$

The continuous function N obtains a minimum m on the compact set $S_1 = \{x \in \mathbb{R}^n \mid \|x\|_2 = 1\}$. This minimum is clearly non-negative. Moreover, we saw that T is injective and so doesn't assume zero values on S_1 , which then implies that since $\|\cdot\|$ is a norm, $N(x) = \|Tx\|$ is never zero on S_1 , and so there is a positive minimum $m > 0$. It follows that for all $x \in S_1$

$$\|Tx\| \geq m = m \|x\|_2$$

Finally, we claim that this inequality holds for all $x \in \mathbb{R}^n$ and not only for $x \in S_1$. If $x = 0$ then both sides of the inequality are zero. Otherwise we have

$$\|Tx\| = \|x\|_2 \cdot \left\| T\left(\frac{x}{\|x\|_2}\right) \right\| \geq \|x\|_2 \cdot m$$

□

The theorem above implies, at a high level, that topologically all finite dimensional normed spaces are equivalent. In particular, we obtain

Corollary 0.29. *Every real finite dimensional normed spaces is complete.*

sketch of proof. Let $(V, \cdot)$ be a real normed space of dimension n and let $T : V \rightarrow \mathbb{R}^n$ be an isomorphism. Let v_n be a Cauchy sequence in V . Since T is an isomorphism, Tv_n is a Cauchy sequence in $\mathbb{R}^n$ (verify). Since $\mathbb{R}^n$ is complete, Tv_n has a limit x . Since Tv_n converges to x , and T is an isomorphism, v_n converges to $T^{-1}x$. □

Corollary 0.30. *Let $(V, \|\cdot\|)$ be a real normed vector space, and $P \subseteq V$ a finite dimensional subspace, and assume that $p_n \subseteq P$ converges to some limit $f \in V$. Then $f \in P$.*

Proof. Since $p_n \rightarrow f$ the sequence p_n is a Cauchy sequence in P and so it has a limit in P . Thus f is in P . □

As mentioned above, this corollary shows in particular that polynomials of degree $\leq D$ cannot approximate *any* continuous functions which are not polynomials of degree $\leq D$.

0.0.2 The Stone Weierstrass Theorem

In the previous section we saw that polynomials are dense in $C[a, b]$. Now we will discuss the Stone-Weierstrass theorem which gives an analogous result for $C(K)$ where K is any compact metric space (it also hold for compact Hausdorff spaces but we do not discuss non-metric topologies in this course). We start by defining multivariate polynomials:

A monomial function $m : \mathbb{R}^n \rightarrow \mathbb{R}$ is a function of the form

$$x = (x_1, \dots, x_n) \mapsto x_1^{\alpha_1} \cdot x_2^{\alpha_2} \dots \cdot x_n^{\alpha_n}$$

where $\alpha = (\alpha_1, \dots, \alpha_n) \in \mathbb{N}_0^n$ (here $\mathbb{N}_0 = \mathbb{N} \cup \{0\}$). We use the shortened notation x^α to described this function. The degree of a monomial is the sum of all indices of α which is denoted by $|\alpha| = \|\alpha\|_1$. A polynomial function $p : \mathbb{R}^n \rightarrow \mathbb{R}$ is a finite linear combination of monomials. The degree of a polynomials is the maximal degree of a monomial in the linear combination which has a non-zero coefficient.

The Stone-Weierstrass theorem shows that multi-variate polynomials are dense in $C(K)$ for compact $K \subseteq \mathbb{R}^n$. In fact, it applies to a wider family of functions called sub-algebras:

Definition 0.31. Let K be a metric space. We say that a subset $\mathcal{A} \subseteq C(K)$ is a subalgebra of $C(K)$ if for all $f, g \in \mathcal{A}$ and $c \in \mathbb{R}$

1. $f + g \in \mathcal{A}$
2. $cf \in \mathcal{A}$
3. $f \cdot g \in \mathcal{A}$

Note that the first two conditions are equivalent to saying that $\mathcal{A}$ is a linear subspace of $C(K)$. Thus a subalgebra is a subspace with the additional condition of closure to multiplication.

Example 0.32. 1. Polynomials $p : \mathbb{R} \rightarrow \mathbb{R}$ are a sub-algebra of $C(\mathbb{R})$.

2. Polynomials of degree $\leq D$ are not a sub-algebra of $C(\mathbb{R})$.
3. Multi-variate polynomials $p : \mathbb{R}^d \rightarrow \mathbb{R}$ are a sub-algebra of $C(\mathbb{R}^d)$.
4. The space of trigonometric polynomials is the function space

$$T = \text{span}\{1, \cos(mx), \sin(nx), m, n \in \mathbb{N}\},$$

where the span of a collection of functions is the set of all possible linear combinations of the functions. Namely, elements in T are functions of the form

$$a_0 + \sum_{m=1}^M a_m \cos(mx) + \sum_{n=1}^N b_n \sin(nx)$$

for appropriate $N, M \in \mathbb{N}$ and $a_0, \dots, a_M, b_1, \dots, b_N \in \mathbb{R}$.

The function space T is a subalgebra of $C(\mathbb{R})$. This can be seen using the trigonometric identities

$$\begin{aligned} 2 \sin(x) \sin(y) &= \cos(x - y) - \cos(x + y) \\ 2 \sin(x) \cos(y) &= \sin(x + y) + \sin(x - y) \\ 2 \cos(x) \cos(y) &= \cos(x - y) + \cos(x + y) \end{aligned}$$

5. For given $a \in \mathbb{R}^n, b \in \mathbb{R}$, we define functions $E_{a,b} : \mathbb{R}^n \rightarrow \mathbb{R}$ of the form

$$E_{a,b}(x) = e^{\langle a, x \rangle + b}.$$

Denote

$$\mathcal{E} = \text{span}\{E_{a,b} \mid a \in \mathbb{R}^d, b \in \mathbb{R}\}. \quad (1)$$

Then $\mathcal{E}$ is a subalgebra of $C(\mathbb{R}^d)$.

We will need two more definitions:

Definition 0.33. A sub-algebra $\mathcal{A} \subseteq C(K)$ separates points in K , if for any points $x, y \in K$ with $x \neq y$ there exists some $f \in \mathcal{A}$ such that $f(x) \neq f(y)$.

Theorem 0.34 (Stone-Weierstrass). *Let K be a compact metric space, and let $\mathcal{A} \subset C(K)$ be a sub-algebra which contains a non-zero constant function and separate points, then $\mathcal{A}$ is dense in $C(K)$.*

We will not prove this theorem, but we will practice using it.

Problem 0.35. Using the Stone-Weierstrass Theorem, answer the following questions:

1. Show that the space of univariate polynomials $\mathbb{R}[x]$ is dense in $C(K)$ for every compact $K \subseteq \mathbb{R}$ (that is, that Weierstrass follows from Stone Weierstrass)
2. Are the multivariate polynomials dense in $C(K)$ for any compact subset $K \subseteq \mathbb{R}^d$?
3. Are the trigonometric polynomials dense in $C(K)$ for any compact subset $K \subseteq \mathbb{R}$?
4. Is $\mathcal{E}$ from Equation 1 dense in $C(K)$ for any compact subset $K \subseteq \mathbb{R}^d$?

Chapter 1

Neural Networks and Universality

Lesson Three

1.1 Introduction

1.1.1 Supervised Learning

Remark 1.1. We will not discuss the subsection on supervised learning in class, as this was covered in the slides shown in the first lesson.

The motivation for most questions discussed in this course come from a *supervised learning* problem: Let $K \subseteq \mathbb{R}^d$ be a compact set, and $f : K \rightarrow \mathbb{R}^\ell$ a continuous function. We have ‘no access’ to f , except via its samples $f(x_i), i = 1, \dots, N$ at some points $x_i, i = 1, \dots, N$ in K . Our aim is to find a good approximation for f , by choosing an appropriate hypothesis class of parametric functions

$$\mathcal{H} = \{h(x; \theta) : K \times \mathbb{R}^p \rightarrow \mathbb{R}^\ell\}$$

and searching for the best approximation of f by h in terms of the given data, by solving an optimization problem such as

$$\min_{\theta \in \mathbb{R}^p} L(\theta) = \sum_{i=1}^N [h_\theta(x_i) - f(x_i)]^2.$$

Typically $L(\theta)$ is differentiable (at least at ‘most’ points), and the methods used for searching for the minimizer of $L(\theta)$ are variations of the ‘gradient descent’ method. In its simplest form, in gradient descent we are given an initial guess θ_0 for the minimum of L , and we iteratively update our guess for θ by going in the direction in which L is reduced most:

$$\theta_{t+1} = \theta_t - \nabla L(\theta_t).$$

Once this procedure is terminated and some final θ_* is reached, we choose h_{θ_*} as our approximation of f . There are several theoretical (and practical) issues that can arise:

1. **Optimization** Can gradient descent (or a different algorithm) find a solution θ_* which is a good approximation of the true minimum?
2. **Generalization** Will a good approximation of f on the data x_i give a good approximation for other points in K ?
3. **Approximation** Does our hypothesis class H contain a good approximation for f ?

In this course we focus only on the question of approximation. We would like to show that the hypothesis classes we construct can approximate any continuous function. This property is called ‘universality’. In the previous chapter we saw some examples of parametric function classes which can approximate all continuous

functions, at the limit where the number of parameters goes to infinity. In practice, we will focus mostly on hypothesis classes called neural networks, which are popular in modern applications. Our first goal will be to show that they too, like polynomials, can approximate all continuous functions. But before we do this, we will start by defining these hypothesis classes:

1.1.2 Neural Network Functions

We refer to functions which are compositions of affine functions and *activation functions* as *neural network functions*. Recall that an affine function $h : \mathbb{R}^{w_0} \rightarrow \mathbb{R}^{w_1}$ is a function of the form

$$h(x) = Ax + b, \text{ where } A \in \mathbb{R}^{w_1 \times w_0}, x \in \mathbb{R}^{w_0} \text{ and } b \in \mathbb{R}^{w_1}$$

An activation function is basically any function $\sigma : \mathbb{R} \rightarrow \mathbb{R}$. We extend σ to a function from $\mathbb{R}^w \rightarrow \mathbb{R}^w$ via

$$\sigma(x_1, \dots, x_w) = (\sigma(x_1), \sigma(x_2), \dots, \sigma(x_w))$$

Fixing some activation function σ , and natural numbers $w_0, \dots, w_L, w_{L+1}$, a *Neural Network Function (NNF)* with widths $w_0, \dots, w_L, w_{L+1}$ is defined to be a function $h : \mathbb{R}^{w_0} \rightarrow \mathbb{R}^{w_{L+1}}$ of the form

$$h_{L+1} \circ \sigma \circ h_L \dots \circ \sigma \circ h_1 \text{ where } h_i : \mathbb{R}^{w_{i-1}} \rightarrow \mathbb{R}^{w_i} \text{ is affine } \forall i = 1, \dots, L+1. \quad (1.1)$$

Choices of activation function Possibly the most popular choice of activation function today is the *Rectified Linear Unit (ReLU)*, defined as

$$\text{ReLU}(x) = \max(x, 0).$$

A very popular choices of activation function in the past was the smooth sigmoid function

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}.$$

A smooth version of the ReLU can be obtained by considering

$$\text{SiLU}(x) = x \cdot \text{sigmoid}(x) = \frac{x}{1 + e^{-x}}.$$

Another common option is the hyperbolic tangent

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

In the literature discontinuous functions like

$$\text{sign}(x) = \begin{cases} 1 & \text{if } x > 0 \\ -1 & \text{if } x \leq 0 \end{cases}$$

are also discussed though they are difficult to work with using gradient descent.

These five activations are visualize in Figure 1.1.

1.1.3 Neural Network Architectures

Neural Network Architectures We consider two types of *Neural Network Architectures*:

1. **Fully connected neural networks** Let $\mathcal{X} = \mathbb{R}^{w_0}$ be our domain. Let $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be some function, which we will call an *activation function*. Let

$$\vec{w} = (w_0, w_1, \dots, w_L, w_{L+1}) \in \mathbb{N}^{L+2}$$

for some $L \in \mathbb{N}$. For fixed $\vec{w}, \sigma$ this defines a *hypothesis class*

$$\mathcal{FC}(\vec{w}; \sigma) = \{h(x) = h_{L+1} \circ \sigma \circ h_L \dots \circ \sigma \circ h_1(x) \text{ where } h_i : \mathbb{R}^{w_{i-1}} \rightarrow \mathbb{R}^{w_i} \text{ is affine } \forall i = 1, \dots, L+1\}$$

The number L is called the depth of the network, and each number w_ℓ is called the width of the ℓ -th layer. We usually think of the vector $\vec{w}$ (and the activation function σ) as *hyper-parameters*: once they are fixed, we can use gradient descent to find the parameters which define the affine functions h_i .

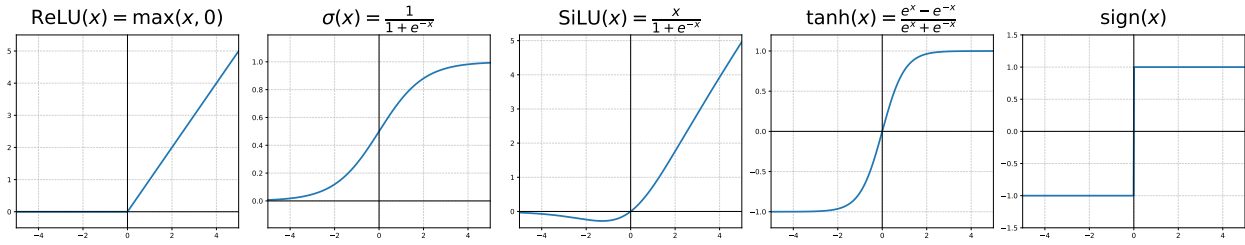
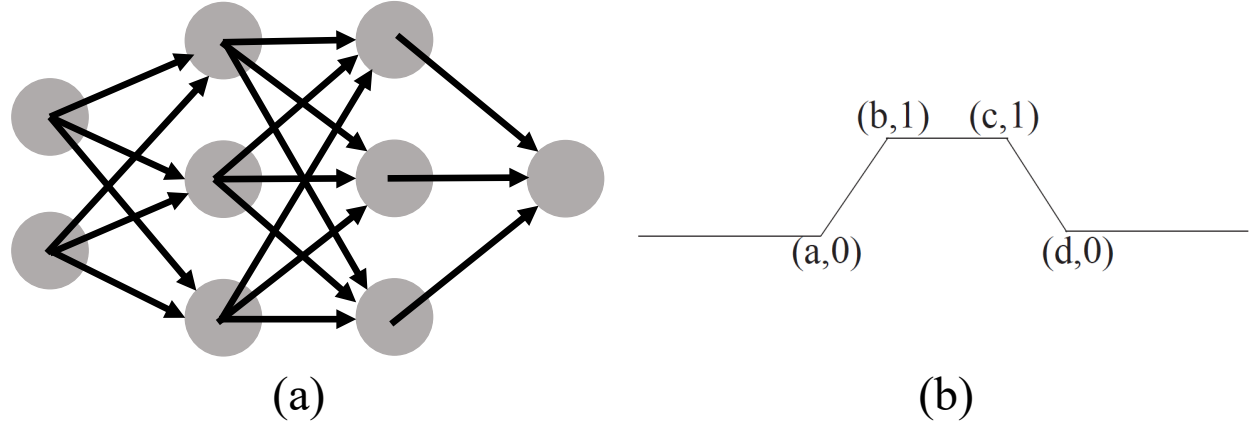


Figure 1.1: Five activations described in the main text

Figure 1.2: (a) The directed graph corresponding to the architecture $\mathcal{FC}(3, 2; 2, 1, \sigma)$. (b) The ‘trapezoid function’ $h_{a,b,c,d}$.

2. Fully connected neural networks with fixed width When studying the approximation power of fully connected neural networks we will usually simplify things by assuming that all hidden layer have the same width. The width vector $\vec{w}$ is then described by four hyper-parameters

$$\vec{w}(W, L, w_{in}, w_{out}) = (w_0 = w_{in}, w_1 = W, w_2 = W, \dots, w_L = W, w_{L+1} = w_{out})$$

and accordingly these hyper-parameters define the hypothesis class

$$\mathcal{FC}(W, L; w_{in}, w_{out}, \sigma) = \mathcal{FC}(\vec{w}(W, L, w_{in}, w_{out}); \sigma)$$

Note that w_{in}, w_{out} are determined by the problem we solved. The parameters W, L are an example of ‘hyperparameters’ chosen by the user. Larger W, L will lead to improved expressive power, at the price of increased computational power, and increased danger of overfitting. Once $w_{in}, w_{out}, W, L, \sigma$ are determined, functions in $\mathcal{FC}(W, L; w_{in}, w_{out}, \sigma)$ can be parameterized by a sequence of matrices and bias vectors $(A_1, \dots, A_{L+1}, b_1, \dots, b_{L+1})$. The value of these parameters which will be used is typically determined by some sort of gradient descent procedure, as described previously.

1.1.4 The ‘Neural’ Terminology

The source of terms such as neural networks, and activation functions comes from the origin of neural networks as an attempt to create learning models which imitate the way the human brain work. For this reason the neural networks we discuss in this course are sometimes called Artificial Neural Networks, to differentiate them from the neural networks which compose our brain.

The general idea is that humans process information through a network of neurons which communicate between themselves through electrical signals. Based on the electrical signals a neuron obtains from its neighbors, the neuron decides whether to ‘fire’ or not. This decision in turn influences its neighbors etc.

Consider a neural network architecture of the form $\mathcal{FC}(W, L; w_{in}, w_{out}, \sigma)$. We can associate with this function a directed graph (V, E) : the vertices in this graph are indexed by

$$v_i^{(\ell)}, \ell = 0, \dots, L \text{ and } i = 1, \dots, W$$

Each vertex is called a *neuron*, and the collection of all vertices with superindex ℓ are called the ℓ -th layer. The 0-th layer is called the input layer, the last layer is called the output layer, and the intermediate layers are called ‘hidden layers’. Figure 1.2(a) depicts the directed graph corresponding to the architecture $\mathcal{FC}(3, 2; 2, 1, \sigma)$

For a given vertex=neuron v_j^ℓ , the incoming edges in the graph connect the neuron to all neurons in the previous layer

$$v_i^{(\ell-1)}, i = 1 \dots w_{\ell-1}$$

while the outgoing edges connect v_j^ℓ to all neurons in the next layer

$$v_i^{(\ell+1)}, i = 1 \dots w_{\ell+1}$$

when $\ell = 0$ there are no incoming edges and when $\ell = L$ is maximal there are no outgoing edges. In Figure 1.2(a) we visualize the directed graph associated with the architecture $\mathcal{FC}(W = 3, L = 2; w_{in} = 2, w_{out} = 1, \sigma)$.

We note that the neural networks we are discussing are often called *fully connected neural networks* because in the directed graph, all nodes in a given layer ℓ are connected to all nodes in the $\ell - 1$ and $\ell + 1$ layer. This is in contrast with, for example, convolutional neural networks, where the connections between layers are sparse.

The function in (1.1) can be rewritten as a composition of functions, where we denote the input $x \in \mathbb{R}^{w_0}$ by $x^{(0)}$ and then recursively define

$$x^{(\ell)} = \sigma \left(h_\ell(x^{(\ell-1)}) \right) = \sigma \left(A^{(\ell)} x^{(\ell-1)} + b^{(\ell)} \right). \quad (1.2)$$

The final vector we obtain by this process $x^{(L)}$ is the output of the neural network function defined in (1.1). Looking at the j -th coordinate of $x^{(\ell)}$ in (1.2) we obtain

$$x_j^{(\ell)} = \sigma \left(\sum_{k=1}^{w_{\ell-1}} A_{jk}^{(\ell)} x_k^{(\ell-1)} + b_k^{(\ell)} \right)$$

That is, the neuron $v_j^{(\ell)}$ aggregates the signals it obtains from its neighbors according to weights $A_{jk}^{(\ell)}$ (which are defined per edges) and $b_k^{(\ell)}$ (which is defined per vertex), and then decides whether to fire. In this context the sigmoid and sign activation functions seem more natural. When using the sign function, the neuron decides to fire (or activate) if the signal it obtained from its neighbors is above a certain threshold. with sigmoid a smoother approach is taken, where the amount of activation varies continuously with the input. Of course, in practice we are interested in which functions perform best in terms of learning, irrespective to how well they resemble our understanding of ‘natural neural networks’. Indeed, ReLU is often the activation of choice in practical applications, although biologically its ‘infinite firing power’ is unrealistic.

1.2 Universality of Neural Networks

Universality theorem We now turn to discuss the approximation power of neural networks. We’d like to have an analogue of the Weierstrass theorem for polynomials- that is- we’d like to say that for every compact set $K \subseteq \mathbb{R}^d$, some neural network function space will be dense (with respect to the ‘uniform convergence’ norm defined in the first lesson) in

$$C(K) = \{f : K \rightarrow \mathbb{R} \text{ is continuous} \}$$

Problem 1.2. Two of the function classes below are *not* dense in $C(K, \mathbb{R})$. Which are they?

1. The function class

$$\bigcup_{(W,L) \in \mathbb{N}^2} \mathcal{FC}(W, L; w_{in} = d, w_{out} = 1, \sigma)$$

with activation function $\sigma(x) = x$.

2. The function class

$$\bigcup_{(W,L) \in \mathbb{N}^2} \mathcal{FC}(W, L; w_{in} = d, w_{out} = 1, \sigma)$$

with activation function $\sigma(x) = \text{ReLU}(x)$.

3. The function class

$$\bigcup_{W \in \mathbb{N}} \mathcal{FC}(W, L = 1; w_{in} = d, w_{out} = 1, \sigma)$$

with activation function $\sigma(x) = \text{ReLU}(x)$. These networks are called shallow neural networks.

4. The function class

$$\bigcup_{W \in \mathbb{N}} \mathcal{FC}(W, L = 4; w_{in} = d, w_{out} = 1, \sigma)$$

with activation function $\sigma(x) = x^2$.

Solution: The first is not dense as it contains only linear functions. The last is not dense as it contains only polynomials of degree $\leq 2^L = 16$. In the previous lesson we saw that polynomials of bounded degree, with the $\|\cdot\|_\infty$ norm, are a finite dimensional normed space, and hence complete, so they cannot approximate functions outside this space. The second and third function classes are dense as we will now discuss.

As it turns out, for ReLU activation functions, a single hidden layer is sufficient to obtain universality(=denseness), as the width is taken to infinity. Which other activation functions have this property? Clearly this cannot occur for polynomial activation functions of some degree D because then $\mathcal{FC}(W, L = 1; w_{in}, w_{out}, \sigma)$ only contains polynomials $p : \mathbb{R}^{w_{in}} \rightarrow \mathbb{R}^{w_{out}}$ of degree $\leq D$. The space of such polynomials is finite dimensional, and by Corollary 0.30 functions from this space cannot approximate any functions from outside that space.

Remarkably, this is the only restriction on the activation function. The well known universality theorem for neural networks (perhaps the most well known result in neural network theory...) states

Theorem 1.3 ([Pinkus, 1999]). *Let $K \subseteq \mathbb{R}^d$ be a compact set, and $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be a continuous function which is not a polynomial. Then*

$$\bigcup_{W \in \mathbb{N}} \mathcal{FC}(W, L = 1; w_{in} = d, w_{out} = 1, \sigma)$$

is dense in $C(K)$.

Due to this theorem, neural networks are often called ‘universal approximators’. That is: for any given unknown continuous f , there is hope to approximate it using a neural network hypothesis class, as such hypothesis classes can approximate *any* continuous function.

A standard reference for the theorem and its historical development, is in the review article by Allan Pinkus [Pinkus, 1999], (who by the way is a retired professor from our very own Technion Math department). In this course we will not present the full proof, but we will use parts of his proof to prove Theorem 1.3 for *smooth* non-polynomial activations. Later on, we will present a different proof for ReLU activations, where we will allow the depth of the network to grow and not only the width.

1.2.1 Proof of Universality of Shallow Neural Networks with smooth activations

Our goal in this section is to prove Theorem 1.3. We begin with some preliminaries.

First, let us reformulate our goal. In general, we would like to prove denseness of functions of the form $h_2 \circ \sigma \circ h_1$ where $h_1 : \mathbb{R}^d \rightarrow \mathbb{R}^W$ and $h_2 : \mathbb{R}^W \rightarrow \mathbb{R}$ are affine and W can be arbitrarily large. In the proof we will always only consider h_2 which are linear, that is $h_2(0) = 0$. We can then write

$$y = \sigma \circ h_1(x) = \sigma(Ax + b)$$

$$h_2(y) = \sum_{w=1}^W c_w y_w$$

Denoting the rows of A by $a^{(1)}, \dots, a^{(W)}$, we have that

$$y_w = \sigma(\langle a^{(w)}, x \rangle + b_w)$$

$$h_2(y) = \sum_{w=1}^W c_w y_w = \sum_{w=1}^W c_w \sigma(\langle a^{(w)}, x \rangle + b_w)$$

We conclude

Lemma 1.4. *Let $K \subseteq \mathbb{R}^d$ be a compact set, and $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be some function, if*

$$\text{span}\{\sigma(\langle a, x \rangle + b) \mid a \in \mathbb{R}^d, b \in \mathbb{R}\} \quad (1.3)$$

is dense in $C(K)$, then

$$\bigcup_{W \in \mathbb{N}} \mathcal{FC}(W, L = 1; w_{in} = d, w_{out} = 1, \sigma)$$

is dense in $C(K)$.

It is convenient to note that for a *specific* choice of activation $\sigma(t) = \exp(t)$, we obtain that the expression in (1.3) is actually equal to $\mathcal{E}$ from (1). Accordingly, we know that

$$\bigcup_{W \in \mathbb{N}} \mathcal{FC}(W, L = 1; w_{in} = d, w_{out} = 1, \sigma = \exp)$$

is dense in $C(K)$ for all $K \subseteq \mathbb{R}^d$. We will want to prove the theorem for more general activations, but the denseness for $\sigma = \exp$ can help us get there, by reducing the denseness question for general activations from $d > 1$ to $d = 1$. You will prove this in homework...

Lemma 1.5. *$\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be some function, if*

$$\text{span}\{\sigma(ax + b) \mid a \in \mathbb{R}, b \in \mathbb{R}\}$$

is dense in $C(K)$ for every $K \subseteq \mathbb{R}$, then for every natural $d \geq 1$,

$$\text{span}\{\sigma(\langle a, x \rangle + b) \mid a \in \mathbb{R}^d, b \in \mathbb{R}\}$$

is dense in $C(K)$ for every $K \subseteq \mathbb{R}^d$

Lesson May 11

Our goal in this lesson is to prove

Lemma 1.6 (Universality for $d = 1$, smooth activations). *Let $K \subseteq \mathbb{R}$ be a compact set, and $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be a C^∞ function which is not a polynomial. Then*

$$P(\sigma) = \text{span}\{\sigma(at + b) \mid a, b \in \mathbb{R}\}$$

is dense in $C(K)$.

This is the essential ingredient in the proof of the universal approximation power of neural networks with smooth activations. In the previous lesson we saw that this lemma proves Theorem 1.3 for the case $d = 1$, and that the general case $d \geq 1$ will then follow from Lemma 1.5.

We begin with an equivalent way to define denseness, via closures. the following description uses abstract definitions but you can think of P below as $P(\sigma)$ from Lemma 1.6, and C below as $C(K)$ from Lemma 1.6.

Closure Next, recall that if C is a normed space and $P \subseteq C$ is a subset, then the *closure* of P is denoted by $\bar{P}$, and defined as the set

$$\bar{P} = \{f \in C \mid \exists (p_n)_{n \in \mathbb{N}} \subseteq P \text{ such that } \|p_n - f\| \rightarrow 0\}.$$

Lemma 1.7. *If C is a normed space and $P \subseteq C$ is a subset, then P is dense in C if and only if $C = \bar{P}$.*

Proof. If P is dense in C . We need to show $C = \bar{P}$. By definition $\bar{P}$ only contains elements in C and therefore $\bar{P} \subseteq C$. For the other containment, fix $f \in C$, we need to show $f \in \bar{P}$. Indeed, for every $n \in \mathbb{N}$ there exists a $p_n \in P$ such that $\|f - p_n\| < 1/n$. It follows that p_n converge to f and so $f \in \bar{P}$.

Conversely, if $\bar{P} = C$, choose some $f \in C$ and $\epsilon > 0$. There exists a sequence $(p_n)_{n \in \mathbb{N}} \subseteq P$ such that $\|p_n - f\| \rightarrow 0$, and thus in particular for large enough N we have that $\|p_N - f\| < \epsilon$. $\square$

The following will be given in homework:

Proposition 1.8. *Let C be a normed space and $P \subseteq C$ a subset, then*

1. $\bar{P}$ contains P , is closed, and is the smallest set having these two properties.
2. If P is a linear subspace of C , then $\bar{P}$ is also a linear subspace of C .

The most interesting (and challenging) idea of the proof is the following lemma. It will be important for later on to note that this specific lemma does not require the assumption that σ is not a polynomial:

Lemma 1.9. *Let $K \subseteq \mathbb{R}$ be a compact set, and $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be a C^∞ function. Then for any $a, b \in \mathbb{R}$ and $k \in \mathbb{N}$, the function $t^k \sigma^{(k)}(at + b)$ is in $\overline{P(\sigma)}$.*

Proof. Fix any $b \in \mathbb{R}$. We need to show that, for every $a \in \mathbb{R}$ and every $k = 0, 1, \dots$ the function

$$f_a^{(k)}(t) = \frac{d^k}{da^k} \sigma(at + b) = t^k \sigma^{(k)}(at + b)$$

is in $\overline{P(\sigma)}$. We prove the claim by induction on k .

For $k = 0$ this is obvious because $\sigma(at + b)$ is in $P(\sigma)$ for every a .

Let us now assume correctness for k and prove for $k + 1$. Using the induction hypothesis and the fact that $\overline{P(\sigma)}$ is a linear subspace, we have for every $h \neq 0$ that $\frac{1}{h} \left(f_{a+h}^{(k)}(t) - f_a^{(k)}(t) \right)$ is in $\overline{P(\sigma)}$. We need to show that they converge uniformly on K to $f_a^{(k+1)}(t)$ and therefore since $\overline{P(\sigma)}$ is closed this function is also

in $\overline{P(\sigma)}$. Indeed, using Taylor's expansion on $f_a(t)$ centered at a , where $t \in K$ is fixed, that there exists some $\hat{a} \in [a-1, a+1]$ (which depends on t) such that for every non-zero h with $|h| < 1$:

$$\begin{aligned} \left| \frac{1}{h} \left(f_{a+h}^{(k)}(t) - f_a^{(k)}(t) \right) - f_a^{(k+1)}(t) \right| &= \left| \frac{1}{h} \left(f_{a+h}^{(k)}(t) - f_a^{(k)}(t) - h f_a^{(k+1)}(t) \right) \right| \\ &= \left| \frac{1}{h} \frac{1}{2} h^2 f_{\hat{a}}^{(k+2)}(t) \right| \\ &= \frac{|h|}{2} |t|^{k+2} |\sigma^{(k+2)}(\hat{a}t + b)| \\ &\leq \frac{|h|}{2} \max_{(\hat{a}, t) \in [a-1, a+1] \times K} |t|^{k+2} \sigma^{(k+2)}(\hat{a}t + b), \end{aligned}$$

where the existence of the maximum follows from the fact that all derivatives of σ exists and are continuous, and the set $[a-1, a+1] \times K$ is compact. It follows that $\frac{1}{h} \left(f_{a+h}^{(k)}(t) - f_a^{(k)}(t) \right)$ converges uniformly on K to $f_a^{(k+1)}(t)$ as required, because

$$\begin{aligned} \left\| \frac{1}{h} \left(f_{a+h}^{(k)} - f_a^{(k)} \right) - f_a^{(k+1)} \right\| &= \max_{t \in K} \left| \frac{1}{h} \left(f_{a+h}^{(k)}(t) - f_a^{(k)}(t) \right) - f_a^{(k+1)}(t) \right| \\ &\leq \frac{|h|}{2} \max_{(\hat{a}, t) \in [a-1, a+1] \times K} |t|^{k+2} \sigma^{(k+2)}(\hat{a}t + b) \xrightarrow{h \rightarrow 0} 0 \end{aligned}$$

Thus, $f_a^{(k+1)} \in \overline{P(\sigma)}$ for all $a \in \mathbb{R}$ and all k . □

Lesson May 12

We can now easily prove our main lemma for smooth activations;

Proof of Lemma 1.6. By Lemma 1.7, we need to show that $\overline{P(\sigma)} = C(K)$. Using Lemma 1.9 in the case $a = 0$, we obtain that for all k the function $t^k \sigma^{(k)}(b)$ is in $\overline{P(\sigma)}$, for all b . Next, note that for every fixed k , there exists some $b = b(k)$ for which $\sigma^{(k)}(b) \neq 0$ (otherwise, σ would be a polynomial of degree $< k$). By choosing this b and dividing $t^k \sigma^{(k)}(b)$ by $\sigma^{(k)}(b)$, we see that all monomials t^k are in $\overline{P}$. Since $\overline{P}$ is a subspace of $C(K)$ it follows that the space $\mathbb{R}[x]$ of all polynomials is contained in $\overline{P(\sigma)}$. From Weierstrass theorem, the polynomials are dense in $C(K)$, and it follows that $C(K) = \overline{\mathbb{R}[x]} \subseteq \overline{P(\sigma)} \subseteq C(K)$ and so $\overline{P(\sigma)} = C(K)$ and we are done. $\square$

1.2.2 Universality: from smooth to continuous

We proved universality for smooth non-polynomial activations. Our next step is to extend this result to continuous non-polynomial activations. We accomplish this using a standard methodology to approximate continuous functions by smooth functions, which we will now introduce.

Firstly, we denote by $C_0^\infty(\mathbb{R})$ the set of functions $\phi : \mathbb{R} \rightarrow \mathbb{R}$ which have derivatives of all order and have bounded support. Namely there exists some $M > 0$, such that $\phi(t) = 0$ for all t outside of $[-M, M]$. A standard example of such a function is

$$\varphi(t) = \begin{cases} \exp(-\frac{1}{1-t^2}) & \text{if } t \in (-1, 1) \\ 0 & \text{if } |t| \geq 1 \end{cases} \quad (1.4)$$

This function is non-negative, and is supported in $[-1, 1]$ (that is, it is zero outside this interval). We can obtain more useful examples by playing around with this example:

For every $\eta \in (0, 1]$ denote

$$\varphi_\eta(t) = \varphi(t/\eta)$$

so that φ_1 is equal to φ from (1.4), and in general φ_η is supported in $[-\eta, \eta]$. Next define

$$\psi_\eta(t) = \frac{1}{\int_{-\infty}^{\infty} \varphi_\eta(s) ds} \varphi_\eta(t)$$

so that ψ_η is a non-negative function whose integral is one, and is supported in $[-\eta, \eta]$. We use these functions to approximate σ using convolutions:

For every $\sigma \in C(\mathbb{R})$ and $\phi \in C_0^\infty(\mathbb{R})$, we define the convolution of these functions to be the function $\sigma * \phi$ defined by

$$(\sigma * \phi)(t) = \int_{-\infty}^{\infty} \sigma(t-y) \phi(y) dy.$$

Note that the integral above is actually only an integral on the support $[-M, M]$ of ϕ , and hence it is well defined.

In the special case where $\phi = \psi_\eta$, what the convolution is doing is basically obtaining a new function $\sigma * \psi_\eta(t)$ which at each t is a weighted average of σ at a small interval $(t-\eta, t+\eta)$. This convolution is illustrated in Figure 1.3. Intuitively, one can be convinced by the picture that (i) $\sigma * \psi_\eta$ converges to σ as $\eta \rightarrow 0$ and (ii) while σ is in general only continuous, $\sigma * \psi_\eta$ is smooth. We will first prove (i) and then (ii):

Lemma 1.10. *Let $L \in \mathbb{N}$. The functions $\sigma * \psi_\eta$ are in $C^\infty(\mathbb{R})$, and converge to σ uniformly in $[-L, L]$ as $\eta \rightarrow 0$.*

Proof. Let $\epsilon > 0$. We need to show that there exists $\delta > 0$, such that for all $\eta < \delta$, we have that

$$\max_{t \in [-L, L]} |\sigma(t) - \sigma * \psi_\eta(t)| < \epsilon. \quad (1.5)$$

Since σ is continuous, it is uniformly continuous on $[-L-1, L+1]$, and therefore we can choose some $\delta > 0$ such that, if $x, y \in [-L-1, L+1]$ and $|x-y| < \delta$, then $|\sigma(x) - \sigma(y)| < \epsilon$.

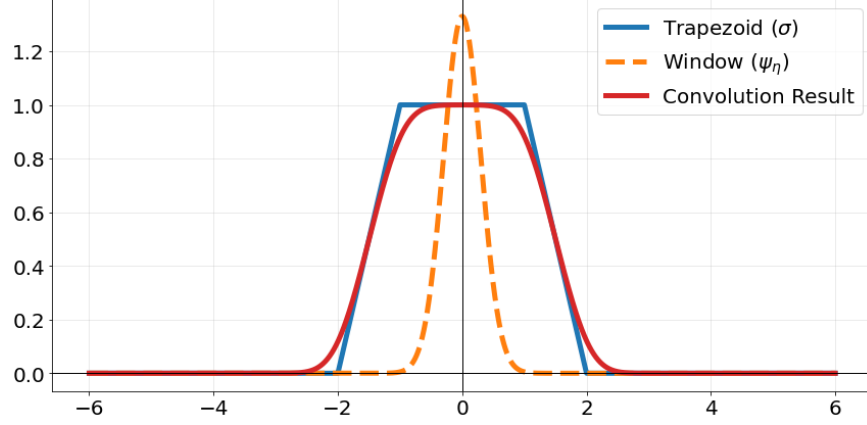


Figure 1.3: Illustration of the convolution of a continuous function σ and a smooth 'window' function ψ_η .

$$\begin{aligned}
 |\sigma(t) - \sigma * \psi_\eta(t)| &= \left| \int_{-\eta}^{\eta} \sigma(t) \psi_\eta(y) dy - \int_{-\eta}^{\eta} \sigma(t-y) \psi_\eta(y) dy \right| \\
 &\leq \int_{-\eta}^{\eta} |\sigma(t) - \sigma(t-y)| \psi_\eta(y) dy \\
 &\leq \epsilon \int_{-\eta}^{\eta} \psi_\eta(y) dy = \epsilon
 \end{aligned}$$

Since this bound holds for all t we obtain (1.5). □

Here are some useful properties of convolution

Proposition 1.11. *Let $\sigma \in C(\mathbb{R})$ and $\phi \in C_0^\infty(\mathbb{R})$, then*

1. *The convolution is 'commutative', namely*

$$(\sigma * \phi)(t) = \int_{-\infty}^{\infty} \phi(t-y) \sigma(y) dy.$$

2. *The function $\sigma * \phi$ is in $C^\infty(\mathbb{R})$, and for any $k \in \mathbb{N}$ the derivative is given by $(\sigma * \phi)^{(k)}(t) = \sigma * \phi^{(k)}(t)$*

Partial proof. The first part follows from the change of variable $x = t - y$,

$$(\sigma * \phi)(t) = \int_{-\infty}^{\infty} \sigma(t-y) \phi(y) dy = \int_{-\infty}^{\infty} \sigma(x) \phi(t-x) dx$$

For the second part, to show the argument for $k = 1$, we would need to show that the following expression goes to zero $h \rightarrow 0$ (for every fixed t):

$$\begin{aligned}
 \frac{(\sigma * \phi)(t+h) - (\sigma * \phi)(t)}{h} - \sigma * \phi'(t) &= \frac{1}{h} \int_{-\infty}^{\infty} \sigma(x) (\phi(t+h-x) - \phi(t-x)) dx - \int_{-\infty}^{\infty} \sigma(x) \phi'(t-x) dx \\
 &= \int_{-\infty}^{\infty} \sigma(x) \left(\frac{\phi(t+h-x) - \phi(t-x)}{h} - \phi'(t-x) \right) dx
 \end{aligned}$$

The expression inside the parenthesis clearly goes to zero for fixed t , as $h \rightarrow 0$. To complete the proof we need to show that the integral indeed goes to zero, and to repeat this recursively to get the result for all derivatives. □

We are now ready to prove universality for continuous non-polynomial activations:

Lemma 1.12 (Universality for $d = 1$, continuous activations). *Let $K \subseteq \mathbb{R}$ be a compact set, and $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be a continuous function which is not a polynomial. Then*

$$P(\sigma) = \text{span}\{\sigma(at + b) \mid a, b \in \mathbb{R}\}$$

is dense in $C(K)$.

Proof. We first make the following claim

Lemma 1.13 (proof in homework). *Let $\psi \in C_0^\infty(\mathbb{R})$ and $\sigma \in C(\mathbb{R})$, then $\overline{P(\sigma * \psi)} \subseteq \overline{P(\sigma)}$.*

The proof of this lemma is homework, the general idea is that $\sigma * \psi(at + b) = \int \sigma(at + b - y)\psi(y)dy$ can be uniformly approximated by functions of the form $f_N(t) = \sum_{n=1}^N h_n \sigma(at + b - y_n)\psi(y_n)$ which we can think of as Riemann sums for every fixed t . Each f_N is a function in $P(\sigma)$.

Now, let us choose a sequence of $C_0^\infty(\mathbb{R})$ functions ψ_η as in Lemma 1.10, such that $\sigma * \psi_\eta$ converges to σ uniformly on the domain K . Note that we know these functions are smooth, and thus if we know that even one of these functions was not a polynomial we would be done. Unfortunately we don't know this, but we will use a similar argument.

We claim that for every $k \in \{0\} \cup \mathbb{N}$ the polynomial t^k is in $\overline{P(\sigma)}$. To show this, for fixed k , we will use the fact that the functions $\sigma * \psi_\eta$ cannot all be polynomials of degree $\leq k$. Indeed, if they were, then σ would also be a polynomial of degree $\leq k$ according to Corollary 0.30, in contradiction to the assumption that σ is not a polynomial. Now, choose some η such that $\sigma * \psi_\eta$ is not a polynomial of degree $\leq k$, and denote $\sigma_k = \sigma * \psi_\eta$. We know that there exists some $b \in \mathbb{R}$ such that $\sigma_k^{(k)}(b) \neq 0$. Applying Lemma 1.9 for the smooth activation σ_k with this choice of b and $a = 0$ we obtain that $t^k \sigma_k^{(k)}(b) \in \overline{P(\sigma_k)}$, and since $\sigma_k^{(k)}(b) \neq 0$ we can divide by it to obtain that $t^k \in \overline{P(\sigma_k)} \subseteq \overline{P(\sigma)}$

□

This concludes the proof of the lemma, and thus also of the universality of neural networks with continuous non-polynomial activations (Theorem 1.3).

Chapter 2

ReLU Neural Networks

New Lesson (May 19)

In this chapter we focus on neural networks with the most common activation: ReLU. Recall that in the previous section, we showed universality of neural networks with continuous non-polynomial activations. This result includes ReLU which is a continuous, non-polynomial activation. In our discussion in this chapter we will dive deeper into ReLU networks. Our main goals will be (i) Understand what functions ReLU networks can express (without any error) and (ii) get quantitative approximation rates, which guarantees the number of parameters necessary to attain ϵ accuracy. In other words, universality guarantees that for a give continuous $f : [0, 1]^d \rightarrow \mathbb{R}$ there exists some $W = W(f, \epsilon, d) \in \mathbb{N}$ and $h_1 : \mathbb{R}^d \rightarrow \mathbb{R}^W, h_2 : \mathbb{R}^W \rightarrow \mathbb{R}$, such that

$$\max_{x \in [0, 1]^d} |f(x) - h_2 \circ \text{ReLU} \circ h_1(x)| < \epsilon$$

our second goal will be to understand how $W(f, \epsilon, d)$ depends on f, ϵ, d . This will be described during this chapter.

We will begin with some reminders and basic properties. Recall that for a fixed activation function $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ and integer vector

$$\vec{w} = (w_0, w_1, \dots, w_L, w_{L+1}) \in \mathbb{N}^{L+2},$$

we defined a *hypothesis class*

$$\mathcal{FC}(\vec{w}; \sigma) = \{h(x) = h_{L+1} \circ \sigma \circ h_L \dots \circ \sigma \circ h_1(x) \text{ where } h_i : \mathbb{R}^{w_{i-1}} \rightarrow \mathbb{R}^{w_i} \text{ is affine } \forall i = 1, \dots, L+1\}.$$

We also said that we will often focus on the case where all hidden layers have the same width. The width vector $\vec{w}$ is then described by four hyper-parameters

$$\vec{w}(W, L, w_{in}, w_{out}) = (w_0 = w_{in}, w_1 = W, w_2 = W, \dots, w_L = W, w_{L+1} = w_{out})$$

and accordingly these hyper-parameters define the hypothesis class

$$\mathcal{FC}(W, L, w_{in}, w_{out}, \sigma) = \mathcal{FC}(\vec{w}(W, L, w_{in}, w_{out}); \sigma)$$

From now on we will focus mostly on the case where $\sigma = \text{ReLU}$. When referring to the architectures above with ReLU activation we will drop the activation from the notation, so that our architectures will just be denoted by $\mathcal{FC}(\vec{w})$ and $\mathcal{FC}(W, L, w_{in}, w_{out})$. We will sometimes use ρ as a notation for ReLU.

2.0.1 Basic approximation properties of ReLU networks

We now discuss some elementary useful properties of fully connected neural networks. For example, it would be natural to hope that smaller networks will be contained in bigger networks. This is indeed the case (up to a small caveat...)

Composition, concatenation, and linear combinations

1. **Going wider can't hurt** Given $\vec{w} = (w_0, \dots, w_{L+1}) \in \mathbb{N}^{L+2}$, every NNF which is in $\mathcal{FC}(\vec{w})$ is also in $\mathcal{FC}([w_0, w_1, \dots, w_j + n, w_{j+1}, \dots, w_{L+1}])$, for every $1 \leq j \leq L$ and every $n \in \mathbb{N}$. Let

$$h(x) = h_{L+1} \circ \rho \circ h_L \dots \circ \rho \circ h_1(x) \text{ where } h_i : \mathbb{R}^{w_{i-1}} \rightarrow \mathbb{R}^{w_i} \text{ is affine } \forall i = 1, \dots, L+1$$

be a function in $\mathcal{FC}(\vec{w})$. To see our claim is true, it is sufficient to show that there exist $\tilde{h}_{j+1} : \mathbb{R}^{w_j+n} \rightarrow \mathbb{R}^{w_{j+1}}$ and $\tilde{h}_j : \mathbb{R}^{w_{j-1}} \rightarrow \mathbb{R}^{w_j+n}$ such that

$$\tilde{h}_{j+1} \circ \rho \circ \tilde{h}_j = h_{j+1} \circ \rho \circ h_j$$

which can be obtained by setting

$$\tilde{h}_j(x) = \begin{pmatrix} 0_n \\ h_j(x) \end{pmatrix}, \text{ and } \tilde{h}_{j+1} \begin{pmatrix} y \\ z \end{pmatrix} = h_{j+1}(z).$$

We can deduce that if the first and last coordinates of $\vec{w}' \in \mathbb{N}^{L+2}$ and $\vec{w} \in \mathbb{N}^{L+2}$ are the same, and all other coordinates of $\vec{w}'$ are larger, then $\mathcal{FC}(\vec{w})$ is contained in $\mathcal{FC}(\vec{w}')$. In particular, for a given $\vec{w} \in \mathbb{N}^{L+2}$, we always have that $\mathcal{FC}(\vec{w}) \subseteq \mathcal{FC}(W, L, w_0, w_{L+1})$ where $W = \max_{1 \leq i \leq L} w_i$.

2. **Composition** If $h \in \mathcal{FC}(W, L, w_{in}, w_{out})$ and $h' \in \mathcal{FC}(W', L', w'_{in}, w'_{out})$ and $w'_{in} = w_{out}$, then

$$h' \circ h \in \mathcal{FC}(W'', L'', w_{in}, w'_{out})$$

where

$$L'' = L + L' \text{ and } W'' = \max\{W, W'\}$$

This is because h and h' are of the form

$$\begin{aligned} h(x) &= h_{L+1} \circ \rho \circ h_L \dots \circ \rho \circ h_1(x) \\ h'(x) &= h'_{L'+1} \circ \rho \circ h'_{L'} \dots \circ \rho \circ h'_1(x) \end{aligned}$$

and so

$$h' \circ h = h'_{L'+1} \circ \rho \circ h'_{L'} \dots \circ \rho \circ h'_1 \circ h_{L+1} \circ \rho \circ h_L \dots \circ \rho \circ h_1$$

we get a fully connected network with ‘width vector’

$$\vec{w} = (w_{in}, W, \dots, W, W', \dots, W', w'_{out})$$

and since ‘going wider can't hurt’ we can replace W, W' with $W'' = \max(W, W')$.

3. **Concatenation** If h_1 and h_2 are neural networks in $\mathcal{FC}(W_1, L, w_{in}, w_{out} = w_1)$ and $\mathcal{FC}(W_2, L, w_{in}, w_{out} = w_2)$ respectively, then

$$h(x) = \begin{pmatrix} h_1(x) \\ h_2(x) \end{pmatrix}, h : \mathbb{R}^{w_{in}} \rightarrow \mathbb{R}^{w_1+w_2}$$

is in $\mathcal{FC}(W_1 + W_2, L, w_{in}, w_1 + w_2)$.

4. **Going deeper doesn't hurt** Given a NNF $h \in \mathcal{FC}(W, L; w_{in}, w_{out})$ with $L \geq 1$, so

$$h(x) = h_{L+1} \circ \rho \circ h_L \dots \circ \rho \circ h_1(x)$$

we have for all $\tilde{L} \geq L$ that $h \in \mathcal{FC}(W, \tilde{L}; w_{in}, w_{out})$ since we can write

$$h(x) = h_{L+1} \circ \rho \circ I \dots \circ \rho \circ I \circ \rho \circ I \circ \rho \circ h_L \dots \circ \rho \circ h_1(x)$$

and we are using the fact that since $\rho(x)$ is always non-negative

$$\rho \circ \rho(x) = \begin{cases} \rho(x) & \text{if } \rho(x) > 0 \\ 0 & \text{if } \rho(x) < 0 \end{cases} = \rho(x).$$

5. **Affine functions** Can an affine function $h : \mathbb{R}^a \rightarrow \mathbb{R}^b$ (we can think of this as a neural network of depth $L = 0$) be expressed as a neural network with $L \geq 1$? This actually is not so straightforward.

Let us focus for now on the identity function $x \mapsto x$ which is a linear function from $\mathbb{R}^n$ to $\mathbb{R}^n$. One would hope that this function is in $\mathcal{FC}(W = n, L = 1, w_{in} = n, w_{out} = n)$. However, this is not true as you will see in homework. There are two tricks to overcome this: the first is to increase the width: note that

$$\text{ReLU}(x) - \text{ReLU}(-x) = x$$

and so

$$h_2 \circ \text{ReLU} \circ h_1(x) = x, \forall x \in \mathbb{R}^n, \text{ where } h_1(x) = (x, -x), h_2(y, z) = y - z.$$

so the identity function is in $\mathcal{FC}(W = 2n, L = 1, w_{in} = n, w_{out} = n)$. By the composition rule, we can compose the identity function with itself L times for any $L \in \mathbb{N}$ and obtain that it is in $\mathcal{FC}(W = 2n, L, w_{in} = n, w_{out} = n)$.

An alternative trick does not require increasing width, but only applies when restricting to compact sets. That is, for any compact $K \subseteq \mathbb{R}^n$ and every $L \in \mathbb{N}$ we can find a function $h \in \mathcal{FC}(W = n, L, w_{in} = n, w_{out} = n)$ such that

$$h(x) = x, \forall x \in K.$$

This is done as follows: choose $b \in \mathbb{R}^n$ whose coordinates are all large enough so that

$$x_i + b_i \geq 0, \forall i = 1, \dots, n, \forall x \in K.$$

and thus $\rho(x + b) = x + b$ for all $x \in K$. Then apply the identity transformation as much as necessary and conclude with the inverse translation $x \mapsto x - b$. In homework you will see how to extend this trick to any affine function.

2.0.2 Expressing functions as ReLU networks: first examples

Some functions can be not only approximated by ReLU networks, but actually can be exactly *expressed* as neural networks. We will now see some simple but useful examples:

- Problem 2.1.** 1. Write the function $x \mapsto |x|$ as a neural network. What are the width and depth?
2. Write the function $(x_1, x_2) \mapsto \max(x_1, x_2)$ as a ReLU neural network. What are the width and depth?

Solution: Note that there can be more than one type of solution for this type of problem. One solution for the absolute value function is

$$|x| = \rho(x) + \rho(-x).$$

This can be written as a neural network in $\mathcal{FC}(W = 2, L = 1, n_{in} = 1, n_{out} = 1)$ We have

$$\max(x_1, x_2) = \max(x_2 - x_1, 0) + x_1 = \text{ReLU}(x_2 - x_1) + x_1 = \text{ReLU}(x_2 - x_1) + \text{ReLU}(x_1) - \text{ReLU}(-x_1)$$

so $\max \in \mathcal{FC}(W = 3, L = 1, n_{in} = 2, n_{out} = 1)$. In homework you will show this can be extended to finding the maximum of a d dimensional vector:

Lemma 2.2. The function $h : \mathbb{R}^d \rightarrow \mathbb{R}$ defined by $h(x) = \max(x)$ is expressible as a neural network in $\mathcal{FC}(W, L, n_{in} = d, n_{out} = 1)$ with width $W = 3d$ and depth $L = \lceil \log_2(d) \rceil$.

2.1 ReLU Networks, CPwL Functions and Depth Separation

2.1.1 ReLU networks are Continuous Piecewise Linear functions

A good reference for this topic is again [DeVore et al., 2021] and references therein. ReLU networks always produce continuous functions, as they are compositions of affine functions and ReLU which are both continuous. In fact, ReLU networks always produce Continuous Piecewise Linear (CPwL) functions, since ReLU and affine functions are CPwL functions. Before we see this, we need to formally defined what CPwL functions are. For this we will need some more definitions...

Definition 2.3. A polytope $P \subseteq \mathbb{R}^d$ is a set of the form

$$P = \{x \in \mathbb{R}^d \mid \langle a_i, x \rangle + b_i \geq 0 \mid i = 1, \dots, m \text{ where for all } i \ a_i \in \mathbb{R}^d, b_i \in \mathbb{R}\} \quad (2.1)$$

Polytopes are always closed and convex sets (see homework). Examples include

1. The whole space $P = \mathbb{R}^d$ (with $m = 0$ constraints, or with the constraint $\langle 0, x \rangle + 1 \geq 0$).
2. The empty set is a polytope defined by the constraint $\langle 0, x \rangle - 1 \geq 0$.
3. The positive quadrant

$$Q = \{x \in \mathbb{R}^d \mid x_i \geq 0, i = 1, \dots, d\}.$$

This expression is of the same form as in (2.1) where $a_i = e_i, b_i = 0$ and $m = d$.

4. The unit hyper cube

$$C = \{x \in \mathbb{R}^d \mid -1 \leq x_i \leq 1, i = 1, \dots, d\}$$

which can be rewritten as in (2.1)

$$C = \{x \in \mathbb{R}^d \mid x_i + 1 \geq 0 \text{ and } 1 - x_i \geq 0, \quad i = 1, \dots, d\}$$

5. The plane

$$P = \{(x_1, x_2, x_3) \mid x_3 = 0\}$$

which can be rewritten as in (2.1)

$$P = \{x \in \mathbb{R}^3 \mid x_3 \geq 0 \text{ and } -x_3 \geq 0\}.$$

The last trick we saw in the examples above, show that closed convex polytopes can be defined by affine equalities as well as affine inequalities.

Definition 2.4. A polytope covering $\mathcal{P}$ of $\mathbb{R}^d$ is a finite collection of polytopes $\mathcal{P} = \{P_1, \dots, P_R\}$ such that $\bigcup_{r=1}^R P_r = \mathbb{R}^d$

For example, The collection of polytopes

$$P_1 = \{(x, y) \mid x \geq 0\}, P_2 = \{(x, y) \mid x \leq 0\}, P_3 = \{(x, y) \mid y \geq 0\}, P_4 = \{(x, y) \mid y \leq 0\}$$

is a polytope covering of $\mathbb{R}^2$.

We can finally define a CPwL function

Definition 2.5. We say that $f : \mathbb{R}^d \rightarrow \mathbb{R}^\ell$ is a CPwL function subordinate to the polytope covering $\mathcal{P} = \{P_1, \dots, P_R\}$ if f is continuous, and the restriction of f to each P_i is an affine function.

We say that f is a CPwL function if it is CPwL function subordinate to some polytope partition.

Remark 2.6. We follow the definition in [DeVore et al., 2021] and require explicitly that f is continuous, though this follows from the other assumptions. On the other hand, we do not require as in [DeVore et al., 2021] that the polytope covering is a partition (i.e., that the interiors of the polytopes do not intersect). This makes the proof of Proposition 2.8 easier and does not hurt generality: if f is CPwL subordinate to some covering, then this covering can be refined to a partition which f will be subordinate to. We will see this soon.

Let us see some examples for CPwL functions. Perhaps the two most important examples for us are affine functions, which are CPwL with respect to the trivial partition $\mathcal{P} = \{\mathbb{R}^d\}$, and the ReLU function. The ReLU function is a linear function on each of the 2^d quadrants of $\mathbb{R}^d$. Formally, for every vector $s \in \{-1, 1\}^d$ we can define the polytope

$$Q_s = \{x \in \mathbb{R}^d \mid s_i x_i \geq 0, \forall i = 1, \dots, d\}$$

The collection of all such polytopes form a polytope partition, and

$$ReLU(x) = \frac{1}{2} \begin{pmatrix} 1+s_1 & 0 & 0 & \dots & 0 \\ 0 & 1+s_2 & 0 & \dots & 0 \\ 0 & 0 & 1+s_3 & \dots & 0 \\ 0 & 0 & 0 & \ddots & 0 \\ 0 & 0 & 0 & 0 & 1+s_d \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_d \end{pmatrix}, \forall x \in Q_s. \quad (2.2)$$

Thus, ReLU is a CPwL function.

Problem 2.7. Show that $\max : \mathbb{R}^d \rightarrow \mathbb{R}$ is CPwL.

Lesson May 25

As we saw that affine functions and ReLU functions are both CPwL, we can now deduce that every NNF is a CPwL function by showing that the composition of CPwL functions are also CPwL.

Proposition 2.8. *If $f : \mathbb{R}^d \rightarrow \mathbb{R}^n$ and $g : \mathbb{R}^n \rightarrow \mathbb{R}^m$ are CPwL functions then $g \circ f$ is a CPwL functions.*

Proof. By assumption f is subordinate to some polytope covering $\mathcal{P} = \{P_1, \dots, P_R\}$ and g is subordinate to some polytope covering $\mathcal{Q} = \{Q_1, \dots, Q_S\}$. The restriction of f to each P_r is an affine function f_r , and the restriction of g to each Q_s is an affine function g_s . We can now define a new polytope covering $\{T_{r,s}\}_{1 \leq r \leq R, 1 \leq s \leq S}$ by

$$T_{r,s} = \{x \in P_r \text{ and } f_r(x) \in Q_s\}$$

We note that these sets do indeed cover all of $\mathbb{R}^d$, that $g \circ f$ is continuous and its restriction to $T_{r,s}$ is the affine function $g_s \circ f_r$, and finally that each $T_{r,s}$ is a polytope, for if

$$P_r = \{x \in \mathbb{R}^d \mid \langle a_i, x \rangle + b_i \geq 0, i = 1, \dots, m\}, f_r(x) = Ax + b$$

and

$$Q_s = \{y \in \mathbb{R}^n \mid \langle c_j, y \rangle + d_j \geq 0, j = 1, \dots, \ell\}$$

then

$$\begin{aligned} T_{r,s} &= \{x \in P_r \text{ and } f_r(x) \in Q_s\} \\ &= \{x \in \mathbb{R}^d \mid \langle a_i, x \rangle + b_i \geq 0 \text{ and } \langle c_j, Ax + b \rangle + d_j \geq 0, \quad i = 1, \dots, m, j = 1, \dots, \ell\} \\ &= \{x \in \mathbb{R}^d \mid \langle a_i, x \rangle + b_i \geq 0 \text{ and } \langle c_j, Ax \rangle + [d_j + \langle c_j, b \rangle] \geq 0, \quad i = 1, \dots, m, j = 1, \dots, \ell\} \\ &= \{x \in \mathbb{R}^d \mid \langle a_i, x \rangle + b_i \geq 0 \text{ and } \langle A^T c_j, x \rangle + [d_j + \langle c_j, b \rangle] \geq 0, \quad i = 1, \dots, m, j = 1, \dots, \ell\} \end{aligned}$$

□

Counting parameters in $\mathcal{FC}(W, L; w_{in}, w_{out})$ The larger W and L are, the more functions we can approximate. However, taking W, L larger also means increasing the computational burden and can quickly become infeasible if we take W, L to be very large. To measure how large an architecture $\mathcal{FC}(W, L; w_{in}, w_{out})$ is, we would like to count how many parameters $n(W, L; w_{in}, w_{out})$ are needed to define it.

First, let us recall that affine functions $h : \mathbb{R}^a \rightarrow \mathbb{R}^b$ are of the form $h(x) = Ax + b$, where $A \in \mathbb{R}^{b \times a}$, $b \in \mathbb{R}^b$. Thus the number of parameters needed to define affine functions from $\mathbb{R}^a$ to $\mathbb{R}^b$ is $a \cdot b + b = b(a + 1)$.

Next, recall that a function $h \in \mathcal{FC}(W, L; w_{in}, w_{out})$ is of the form

$$h = h_{L+1} \circ \rho \circ h_L \dots \circ \rho \circ h_1(x)$$

where

$$h_1 : \mathbb{R}^{w_{in}} \rightarrow \mathbb{R}^W, \quad h_{L+1} : \mathbb{R}^W \rightarrow \mathbb{R}^{w_{out}}, h_j : \mathbb{R}^W \rightarrow \mathbb{R}^W, j = 2, \dots, L.$$

Therefore, the number of parameters needed to define a neural network in $\mathcal{FC}(W, L; w_{in}, w_{out})$ is

$$n(W, L; w_{in}, w_{out}) = W(w_{in} + 1) + (L - 1)(W^2 + W) + w_{out}(W + 1)$$

Generally w_{in}, w_{out} are given by the problem we consider, and we think of them as fixed, and of W, L as free parameters which we take to infinity. We note that when $L = 1$ the number of parameters depends linearly on the width

$$n(W, L = 1; w_{in}, w_{out}) = W(w_{in} + 1) + w_{out}(W + 1),$$

while when $L \geq 2$ we see that $n(W, L, w_{in}, w_{out})$ grows like LW^2 - linearly in the depth and quadratically in the width.

Scalar to scalar neural networks We will next discuss the special case where $w_{in} = 1 = w_{out}$. In this case we denote $n(W, L) = n(W, L, w_{in} = 1, w_{out} = 1)$. We have

$$\begin{aligned} n(W, L) &= 2W + (L - 1)(W^2 + W) + W + 1 \\ &= 3W + 1 + (L - 1)(W^2 + W) \end{aligned}$$

When $L = 1$ we have that $n(W, L) = 3W + 1$. When $L \geq 2$ we see that $n(W, L)$ is proportional to LW^2 , as for all integer W and $L \geq 2$

$$\begin{aligned} 3W + 1 + (L - 1)(W^2 + W) &\leq 3W^2 + W^2 + 2(L - 1)W^2 = W^2(2L + 2) \leq 3LW^2 \\ 3W + 1 + (L - 1)(W^2 + W) &\geq (L - 1)W^2 \geq \frac{1}{2}LW^2 \end{aligned}$$

2.1.2 Depth separation and counting linear regions

In this subsection we will address the following questions: can we show that neural networks have some expressive advantage over standard function bases (e.g., standard CPwL function spaces)? Can we show that deep neural networks ($L \gg 1$) have an expressive advantage over shallow networks ($L = 1$)? These questions are motivated by the successfulness of deep learning in the past years, in comparison with alternative function spaces or shallow networks.

In this subsection we will consider univariate neural networks with $n_{in} = 1 = n_{out}$. We will also restrict our attention to the behavior of these functions on $[0, 1]$. Thus we are interested in the function space

$$\Upsilon(W, L) = \{g : [0, 1] \rightarrow \mathbb{R} \mid g \in \mathcal{FC}(W, L, 1, 1)\}$$

Most of the results described here come from [Daubechies et al., 2021] unless mentioned otherwise.

As we saw, any function $f \in \mathcal{FC}(W, L, 1, 1)$ is a CPwL function $f : \mathbb{R} \rightarrow \mathbb{R}$ subordinate to a polytope partition of $\mathbb{R}$. In homework you will show the following:

Problem 2.9. 1. Let $P \subseteq \mathbb{R}$ be a polytope. Show that P is either an empty set, a singleton, or a closed interval of the form $[a, b]$ or $[a, \infty)$ or $(-\infty, b]$ or $(-\infty, \infty)$.

2. Let $f : \mathbb{R} \rightarrow \mathbb{R}$ be a CPwL function as defined in class. Show that there exists for some $n \in \mathbb{N} \cup \{0\}$, numbers

$$k_0 = 0 < k_1 < k_2 < \dots < k_n < k_{n+1} = 1$$

such that, for all $i = 0, 1, \dots, n$, the restriction of f to $[k_i, k_{i+1}]$ is an affine function.

For a CPwL $f : [0, 1] \rightarrow \mathbb{R}$, we call a point in $(0, 1)$ at which f is not differentiable a **breakpoint** of f . We denote the space of functions with at most n breakpoints by Σ_n . We note that functions in Σ_n can be determined by $2n + 2$ parameters: n "free" knots $0 < k_1 < \dots < k_n < 1$, n assigned values y_i to each of the knots, and values y_0 and y_{n+1} assigned to two "boundary" knots $k_0 = 0$ and $k_{n+1} = 1$. These values then uniquely define a CPwL function on $[0, 1]$ which attains these values and is affine in the intervals defined by the knots. We refer to this function spaces as the space of free knot splines with n knots. Note that a function in Σ_n can have a breakpoint (a non-differentiable point) only at a knot, but a knot does not necessarily have to be a breakpoint. For example, the function $f = 0$ is differentiable everywhere, and is in Σ_n by choosing any n knots and choosing the corresponding values y_n to be zero. The functions Σ_n with this parameterization are known as free-knot splines. They are a standard way to represent CPwL univariate functions in the approximation theory literature.

In the following we'd like to compare the function space Σ_n and $\Upsilon(W, L)$. Both of these spaces contain CPwL functions from $[0, 1]$ to $\mathbb{R}$. But while, for fixed n , it is clear what functions reside in Σ_n , the contents of $\Upsilon(W, L)$ are more mysterious. We will consider the following questions: can every CPwL univariate function be expressed as a NNF? If so, is it more efficient to parameterize CPwL functions as free knot splines or as neural networks, and if so, how do the choice of W, L affect the efficiency of the representation?

2.1.3 Shallow univariate neural networks

We begin by discussing shallow networks, that is networks with $L = 1$. We prove the following

Proposition 2.10. *For every $W \in \mathbb{N}$ we have*

$$\Sigma_{W-1} \subseteq \Upsilon(W, L = 1) \subseteq \Sigma_W$$

Recall that $n(W, L = 1) = 3W + 1$ while functions in Σ_n are determined by $2n$ parameters. Thus this theorem says that the number of parameters needed to express a CPwL functions via shallow neural networks or via free knot splines is roughly the same.

Proof. Let us first show that $\Upsilon(W, L = 1) \subseteq \Sigma_W$. Recall that a function f in $\Upsilon(W, L = 1)$ is of the form

$$f(x) = h_2 \circ \text{ReLU} \circ h_1(x)$$

where $h_1 : \mathbb{R} \rightarrow \mathbb{R}^W$ and $h_2 : \mathbb{R}^W \rightarrow \mathbb{R}$ are affine, and can be written as

$$h_1(x) = x\vec{a} - \vec{b} \text{ and } h_2(\vec{y}) = \langle \vec{c}, \vec{y} \rangle + d$$

so

$$f(x) = d + \sum_{j=1}^W c_j \text{ReLU}(a_j x - b_j) \quad (2.3)$$

If f is non-differentiable at a point x , necessarily for some j the function $\text{ReLU}(a_j x - b_j)$ is non-differentiable at x . If $a_j = 0$ this functions is constant, hence differentiable everywhere. If $a_j \neq 0$ then this function is differentiable except for the point $x = b_j/a_j$. There are at most W different points b_j/a_j in $(0, 1)$. Thus $f \in \Sigma_W$.

In the other direction: Let $g \in \Sigma_{W-1}$. There exist knots $k_0 = 0 < k_1 < k_2 < \dots < k_{W-1} < k_W = 1$ such that g is affine in the intervals denoted by each pair of consecutive knots. We need to show that $g \in \Upsilon(W, L = 1)$. To do so, we consider neural networks $N \in \Upsilon(W, L = 1)$ of the form

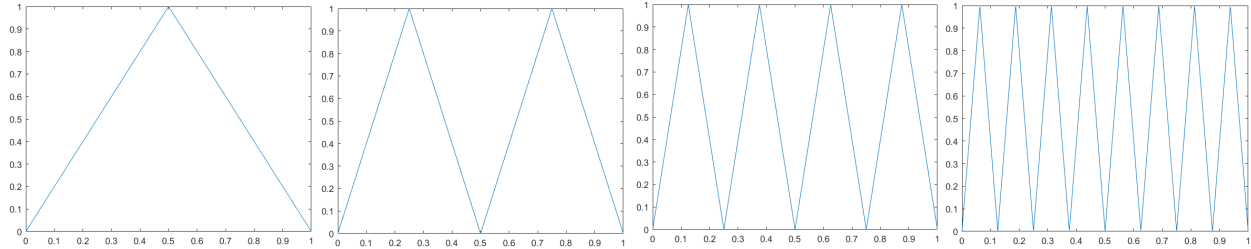
$$N(x) = d + \sum_{i=0}^{W-1} c_i \text{ReLU}(x - k_i), \quad \forall x \in [0, 1]$$

We need to show that we can choose the parameters $d, c_0, \dots, c_{W-1}$ so that $N(x) = g(x)$ for all $x \in [0, 1]$. Note that, by construction, N is affine on the same intervals $[k_{i-1}, k_i]$ that g is affine on, and therefore it is sufficient to choose the parameters so that $g(k_i) = N(k_i)$ for all $i = 0, \dots, W$. This gives us the following equations

$$\begin{aligned} g(k_0 = 0) &= d \\ g(k_1) &= d + c_0(k_1 - k_0) \\ g(k_2) &= d + c_0(k_2 - k_0) + c_1(k_2 - k_1) \\ &\vdots \\ g(k_W) &= d + c_0(k_W - k_0) + c_1(k_W - k_1) + \dots + c_{W-1}(k_W - k_{W-1}) \end{aligned}$$

This equation can be easily solved recursively from top to bottom, and the solution $d, c_0, \dots, c_{W-1}$ induces a function N satisfying $N(x) = g(x)$ as required. $\square$

Problem 2.11. Give an example of a univariate CPwL function with W breakpoints that is not in $\Upsilon(W, 1)$.

Figure 2.1: The function $H^{\circ s}$ for $s = 1, 2, 3, 4$.

Lesson May 26

2.1.4 Deep Univariate Neural Networks

We've seen in Proposition 2.10 that shallow neural networks of width W are essentially the same as free knot CPwL splines. What happens when $L > 1$? The following theorem, which is a simplified version of Theorem 3.1 in [Daubechies et al., 2021], shows that also when $L > 1$, we can express all functions with n breakpoints with $n \sim n(W, L)$:

Theorem 2.12. *[[Daubechies et al., 2021]] There exists an absolute constant $C > 0$, such that for every natural $W \geq 8$ and $n \geq \frac{1}{6}(W - 2)^2$, there exists an $L \in \mathbb{N}$ satisfying*

$$n(W, L) \leq Cn \text{ and } \Sigma_n \subseteq \Upsilon(W, L)$$

NNF with exponentially many knots Interestingly, for deep networks, it is possible to express *some* functions with $n \gg n(W, L)$ breakpoints as we will now see. The following question will be in the homework:

Problem 2.13. Consider the function

$$H(x) = \begin{cases} 2x & \text{if } 0 \leq x \leq 1/2 \\ 2 - 2x & \text{if } 1/2 \leq x \leq 1 \end{cases}.$$

1. Show that $H \in \Upsilon(2, 1)$ and that the function $H^{\circ s}$ (the composition of H with itself s times) is in $\Upsilon(2, L = s)$.
2. Define for every $s \in \mathbb{N}$ the points

$$k_0 = 0, k_1 = \frac{1}{2^s}, k_2 = \frac{2}{2^s} \dots, k_{2^s} = 1.$$

Show that $H^{\circ s}(x)$ is the unique CPwL function which is affine on each interval $[k_{j-1}, k_j], j = 1, \dots, 2^s$ and satisfies

$$H^{\circ s}(k_j) = \begin{cases} 0 & \text{if } j \text{ is even} \\ 1 & \text{if } j \text{ is odd} \end{cases}$$

3. How many breakpoints does $H^{\circ s}$ have?

We call the function H the ‘hat function’. The composition of H with itself s times is called the sawtooth function(s), or sometimes Telgarsky’s function. These are illustrated in figure 2.1. The function $H^{\circ s}$ are in $\Upsilon(2, s)$, so the number of parameters needed to express it grows linearly in s . However it has $2^s - 1$ breakpoints. In particular, $H^{\circ s}$ is in Σ_W only for $W \geq 2^s - 1$, and by Proposition 2.10 it can therefore be expressed by a shallow neural network $L = 1$ only if $W \geq 2^s - 1$. Thus this is an example of a function expressed by deep neural networks of size linear in s , but expressing it using shallow neural networks or free knot splines requires a number of parameters exponential in s .

Here is a helpful way to see how many breakpoints H has easily:

Problem 2.14. say $f : [0, 1] \rightarrow \mathbb{R}$ is CPwL and is affine in the intervals $[k_{j-1}, k_j]$ defined by the points $0 < k_1 < k_2 < \dots < k_n < 1$. Show that $f \circ H$ is affine in the intervals defined by the points

$$0 < \frac{k_1}{2} < \dots < \frac{k_n}{2} < \frac{1}{2} < 1 - \frac{k_n}{2} < \dots < 1 - \frac{k_1}{2} < 1.$$

Using this argument we can compute the knots of H^{os} . The calculation of H^{os} is then reduced to finding the value of H^{os} at these knots.

Summary To summarize our discussion of univariate ReLU networks: we saw that shallow neural networks are more or less the same as free knot linear splines (Proposition 2.10). Deep networks can also express all free knot splines with $n(W, L)$ parameter (Theorem 2.12), however, they can also express some functions, such as ‘sawtooth functions’, with a number of breakpoints which is exponential in L (but not all such functions, see proof in [Dym et al., 2020]).

The attractiveness of the latter results as that they point to an expressive advantage of deep neural networks over a ‘standard function base’-free knot linear splines. It also points to an advantage over shallow neural networks- a network with depth one, or with fixed depth, would need its width to grow exponentially with s to express the function H^{os} , rather than the linear growth we have observed. This type of result is sometimes called a ‘depth separation’ result. There are many similar arguments in various papers, see e.g., [Telgarsky, 2016, Eldan and Shamir, 2016].

Possible criticisms of these arguments are that highly oscillating functions such as the sawtooth function are difficult to learn using gradient descent (see e.g., [Malach and Shalev-Shwartz, 2019, Hanin and Rolnick, 2019]). Another possible objection is that the functions H^{os} don’t seem to be a very natural approximation target. However, we will next show how they pave the way to very efficient approximation of the function x^2 , and from there we can reach approximation rates for other smooth functions.

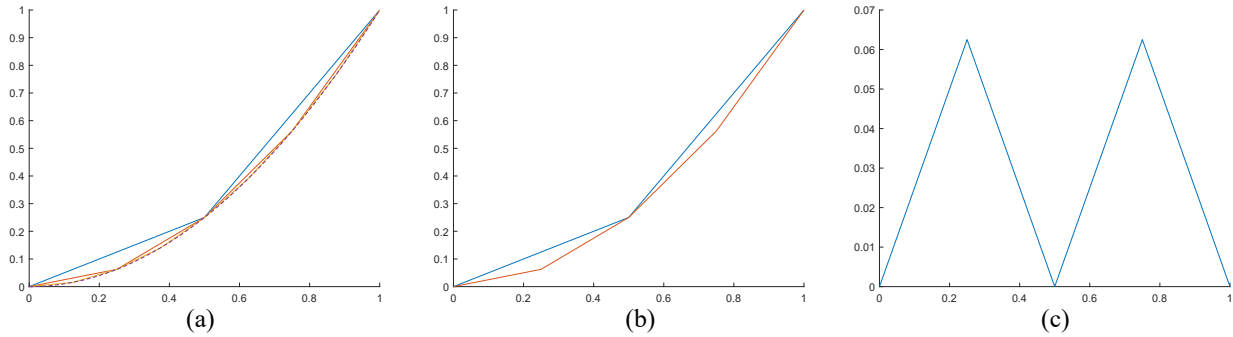


Figure 2.2: (a) Swift convergence of the approximations $f_m(x)$ of x^2 (b) the functions f_1 and f_2 and (c) the function $f_1 - f_2$ is a scaling of the sawtooth function $H \circ H$.

Lesson June 1st

2.2 Approximating polynomials and smooth functions

We saw that functions like the sawtooth function can be expressed with deep neural networks significantly more efficiently than with shallow networks. We will now show how this can be leveraged to obtain similar results for approximation of the function x^2 . We will then build upon this further to approximate monomials, polynomials, and smooth functions in general. This discussion is taken from [Yarotsky, 2017].

The sawtooth function will actually turn out to be a useful tool in our discussion. Recall that the sawtooth function $H^{\circ s}$ can be implemented with width 2 and depth s despite having $2^s - 1$ breakpoints. It is linear on the intervals $[k_i, k_{i+1}]$ where

$$k_0 = 0, k_1 = \frac{1}{2^s}, k_2 = \frac{2}{2^s} \dots, k_{2^s} = 1$$

and $H^{\circ s}(k_\ell)$ is 0 if ℓ is even and one if ℓ is odd.

2.2.1 Approximating x^2

Let us consider the problem of approximating x^2 with a ReLU network. As a first step, let us consider approximations with shallow networks, or equivalently, with functions in Σ_N .

Proposition 2.15. *Let g_N be the unique CPwL function which satisfies $g_N(k/N) = (k/N)^2$ for all $k = 0, \dots, N$, and is affine on the intervals $[\frac{k-1}{N}, \frac{k}{N}]$, $k = 1, \dots, N$. Then*

$$\max_{x \in [0, 1]} |x^2 - g_N(x)| \leq \left(\frac{1}{2N} \right)^2$$

Proof. We will use the following lemma

Lemma 2.16. *Let $g(x) = g(x|a, b)$ be the linear interpolation of x^2 at a, b , that is*

$$g(x) = \frac{1}{b-a} [(b-x)a^2 + (x-a)b^2]$$

then

$$\max_{x \in [a, b]} |g(x) - x^2| = g\left(\frac{a+b}{2}\right) - \left(\frac{a+b}{2}\right)^2 = \left(\frac{b-a}{2}\right)^2$$

Proof of the Lemma. The continuous function $q(x) = g(x) - x^2 = g(x) - x^2$ is maximized and minimized at a or b or at a point $x \in (a, b)$ with $q'(x) = 0$. We have that $q(a) = 0 = q(b)$. Solving the equation

$$0 = q'(x) = \frac{b^2 - a^2}{b-a} - 2x$$

we obtain that the derivative is zero at the midpoint $\frac{a+b}{2}$. At this point we have

$$q\left(\frac{a+b}{2}\right) = \frac{a^2+b^2}{2} - \left(\frac{a+b}{2}\right)^2 = \frac{1}{4}(2a^2+2b^2-a^2-b^2-2ab) = \frac{1}{4}(b-a)^2$$

So the minimum of q is obtained at a and b , and the minimal value of q is zero, while the maximum is obtained at the center and the maximal values is $\frac{1}{4}(b-a)^2$. Thus

$$\forall x \in [a, b], \quad |g(x) - x^2| = |q(x)| \leq \frac{1}{4}(b-a)^2$$

This concludes the proof of the lemma. $\square$

Returning to our definition of g_N we see that for every $x \in [0, 1]$ there exists some k such that x is in $[\frac{k}{N}, \frac{k+1}{N}]$, and then $g_N(x)$ is equal to $g(x|a, b)$ with $a = k/N$ and $b = (k+1)/N$. Therefore

$$\begin{aligned} |x^2 - g_N(x)| &\leq \max_{x \in [\frac{k}{N}, \frac{k+1}{N}]} |x^2 - g(x|k/N, (k+1)/N)| \\ &= \left(\frac{1}{2N}\right)^2 \end{aligned}$$

This concludes the proof of the proposition $\square$

We see that with N knots we can get an approximation of $\left(\frac{1}{2N}\right)^2$. Stated differently, to achieve ϵ accuracy we need to take N such that $\left(\frac{1}{2N}\right)^2 < \epsilon$ so $N > (1/(4\epsilon))^{1/2}$.

To approximate g_N , which is a univariate CPwL function with $\sim N$ breakpoints, we can use a shallow neural network with $\sim N$ width and parameters, as we saw previously.

We will now see that we can do much better with deep networks: denote $f_m = g_{2^m}$: that is, f_m is the CPwL function with knots in $0, \frac{1}{2^m}, \frac{2}{2^m}, \dots, 1$ and whose value at these points is

$$f_m\left(\frac{k}{2^m}\right) = \left(\frac{k}{2^m}\right)^2, \quad k = 0, 1, \dots, 2^m.$$

By our previous discussion we have that

$$\max_{x \in [0, 1]} |f_m(x) - x^2| \leq \frac{1}{2^{2m+2}}.$$

This is a great approximation rate but of course the function f_m has 2^m knots and so is expensive to compute using free knot splines or shallow networks. However, this can be done much more efficiently with deep networks, using the sawtooth function we studied earlier, and the following recursive formula for computing f_m using f_{m-1} and $H^{\circ m}$:

Lemma 2.17. *Let f_m denote the Piecewise linear interpolation of x^2 at the points $k2^{-m}, k = 0, \dots, 2^m$ as defined above, and let H denote the sawtooth function, then*

$$f_m(x) = f_{m-1}(x) - \frac{H^{\circ m}(x)}{2^{2m}}, \quad \forall x \in [0, 1] \quad (2.4)$$

An illustration of this result is shown in Figure 2.2.

Proof. By rearranging (2.4), we need to prove

$$f_{m-1}(x) - f_m(x) = \frac{H^{\circ m}(x)}{2^{2m}}, \quad \forall x \in [0, 1]$$

Let us consider the difference $f_{m-1}(x) - f_m(x)$. This function can only have breakpoint at points $\frac{k}{2^m}$ for $k = 0, \dots, 2^m$, which are exactly the breakpoints of $H^{\circ m}$. It is thus sufficient to show that $f_{m-1} - f_m$ and $\frac{H^{\circ m}}{2^{2m}}$ agree on all knots of the form $k/2^m$. The knots where k is even are also knots of f_{m-1} and therefore at

these knots $f_{m-1}(\frac{k}{2^m}) = (\frac{k}{2^m})^2 = f_m(\frac{k}{2^m})$, and the difference between the functions is zero at these points. When k is odd, we can repeat the computation we did in Lemma 2.16: Let us denote $a = \frac{k-1}{2^m}$ and $b = \frac{k+1}{2^m}$. Since $k-1$ and $k+1$ are even, we have that $f_{m-1}(a) = a^2$, $f_{m-1}(b) = b^2$, while $\frac{k}{2^m} = \frac{1}{2}(a+b)$ and so

$$f_{m-1}\left(\frac{k}{2^m}\right) - f_m\left(\frac{k}{2^m}\right) = \frac{1}{2}(a^2 + b^2) - \left(\frac{1}{2}(a+b)\right)^2 = \frac{1}{4}(2a^2 + 2b^2 - a^2 - 2ab - b^2) = \frac{1}{4}(b-a)^2 = \frac{1}{2^{2m}}$$

and so altogether we have

$$f_{m-1}\left(\frac{k}{2^m}\right) - f_m\left(\frac{k}{2^m}\right) = \begin{cases} 0 & \text{if } k \text{ is even} \\ \frac{1}{2^{2m}} & \text{if } k \text{ is odd} \end{cases}$$

This is exactly the value of the scaled sawtooth function $\frac{H^{\circ m}}{2^{2m}}$, and thus we deduce that (2.4) holds. $\square$

We can now deduce a result on very efficient approximation of x^2 by deep relu networks:

Theorem 2.18. *There exists some natural numbers W_0, L_0 , such that the function $f(x) = x^2$ can be approximated to ϵ accuracy in $[0, 1]$ with a NNF in $\Upsilon(W_0, L_0 \lfloor \log(1/\epsilon) \rfloor)$.*

Remark Recall that a network with width W and depth L has $\sim W^2 L$ parameters, so the total number of parameters is linear in $\log(1/\epsilon)$.

Proof. We saw that $f_m(x)$ approximate x^2 with an error rate of $\frac{1}{2^{2m+2}}$. To obtain ϵ accuracy we need to have

$$2^{-2m-2} < \epsilon \text{ if and only if } 2(m+1) > \log_2(1/\epsilon) \text{ if and only if } m+1 > \frac{1}{2} \log_2(1/\epsilon).$$

So we can choose $m = \lfloor 1/2 \log_2(1/\epsilon) \rfloor$.

We express f_m as a ReLU network via the recursive formula (2.4), via the formula

$$x \mapsto \begin{bmatrix} H(x) \\ f_0(x) = x \end{bmatrix} \xrightarrow{F_2} \begin{bmatrix} H^{\circ 2}(x) \\ f_1(x) \end{bmatrix} \xrightarrow{F_3} \dots \xrightarrow{F_{m-1}} \begin{bmatrix} H^{\circ m-1}(x) \\ f_{m-1}(x) \end{bmatrix} \mapsto f_m(x), \text{ where } F_s \begin{bmatrix} y \\ z \end{bmatrix} = \begin{bmatrix} H(y) \\ z - y/2^{2s} \end{bmatrix}$$

which shows that f_m can be constructed with a neural network with fixed width W_0 and depth $mL_0 = \lfloor 1/2 \log_2(1/\epsilon) \rfloor L_0$. This concludes the proof of the theorem. $\square$

Lesson June 2nd

2.2.2 Approximating multiplication

An important consequence of the previous result is that the product function $(x, y) \mapsto xy$ can also be approximated by neural networks with exponential accuracy: in general, this can be done by noting

$$xy = \frac{1}{2}((x+y)^2 - x^2 - y^2)$$

and then using our approximation of the squaring function. Recall that our approximation of x^2 was obtained on $[0, 1]$. We will want to obtain approximation for xy on $[-M, M]^2$. To do this note that we can rewrite the previous equation as

$$xy = 2M^2 \left(\left(\frac{|x+y|}{2M} \right)^2 - \left(\frac{|x|}{2M} \right)^2 - \left(\frac{|y|}{2M} \right)^2 \right). \quad (2.5)$$

The advantage of this is that we are now applying the squaring function to inputs in the interval $[0, 1]$.

Proposition 2.19. *There exists $(W_1, L_1) \in \mathbb{N}^2$, such that for all $M > 0$ and $\epsilon > 0$, there is an NNF $NN_\times : \mathbb{R}^2 \rightarrow \mathbb{R}$ satisfying*

1. *for all x, y with $|x|, |y| < M$ we have $|NN_\times(x, y) - xy| < \epsilon$.*

2. $NN_\times$ is in $\mathcal{FC}(W_1, L_1(1 + \log(1/\epsilon) + \log(M)), 2, 1)$.

Proof. Choose some $\epsilon > 0$ and $M > 0$. Recall that there exists $(W_0, L_0) \in \mathbb{N}^2$ such that for all $\tilde{\epsilon} > 0$ there exists $s \in \mathcal{FC}(W_0, L_0 \log(1/\tilde{\epsilon}), 1, 1)$ such that $|s(x) - x^2| < \tilde{\epsilon}$ for all $x \in [0, 1]$. Now set

$$\tilde{\epsilon} = \frac{\epsilon}{6M^2}.$$

and let s be the function in $\mathcal{FC}(W_0, L_0 \log(1/\tilde{\epsilon}), 1, 1)$ such that $|s(x) - x^2| < \tilde{\epsilon}$. We replace the squares in (2.5) with s to obtain the function

$$NN_\times(x, y) := 2M^2 \left(s\left(\frac{|x+y|}{2M}\right) - s\left(\frac{|x|}{2M}\right) - s\left(\frac{|y|}{2M}\right) \right)$$

We can verify directly that this function satisfied (1) due to (2.5), namely

$$\begin{aligned} |NN_\times(x, y) - x \cdot y| &= 2M^2 \left| \left(s\left(\frac{|x+y|}{2M}\right) - s\left(\frac{|x|}{2M}\right) - s\left(\frac{|y|}{2M}\right) \right) - \left(\left(\frac{|x+y|}{2M}\right)^2 - \left(\frac{|x|}{2M}\right)^2 - \left(\frac{|y|}{2M}\right)^2 \right) \right| \\ &\leq 2M^2 \left(\left| s\left(\frac{|x+y|}{2M}\right) - \left(\frac{|x+y|}{2M}\right)^2 \right| + \left| s\left(\frac{|x|}{2M}\right) - \left(\frac{|x|}{2M}\right)^2 \right| + \left| s\left(\frac{|y|}{2M}\right) - \left(\frac{|y|}{2M}\right)^2 \right| \right) \\ &\leq 2M^2 \cdot 3\tilde{\epsilon} = \epsilon \end{aligned}$$

and satisfies (2) as $NN_\times$ can be written as

$$\begin{bmatrix} x \\ y \end{bmatrix} \xrightarrow{F_1} \begin{bmatrix} \frac{|x+y|}{2M} \\ \frac{|x|}{2M} \\ \frac{|y|}{2M} \end{bmatrix} \xrightarrow{F_2} \begin{bmatrix} s\left(\frac{|x+y|}{2M}\right) \\ s\left(\frac{|x|}{2M}\right) \\ s\left(\frac{|y|}{2M}\right) \end{bmatrix} \xrightarrow{F_3} 2M^2 \left(s\left(\frac{|x+y|}{2M}\right) - s\left(\frac{|x|}{2M}\right) - s\left(\frac{|y|}{2M}\right) \right)$$

Since the absolute value function

$$|z| = \text{ReLU}(z) + \text{ReLU}(-z)$$

can be expressed as a ReLU network (of depth 1 and width 2), the function F_1 , has constant width W_{const} and depth L_{const} . The function F_3 is linear. The function F_2 is three copies of s , and so can be expressed with width $3W_0$ and depth

$$L_0 \log_2(1/\tilde{\epsilon}) = L_0 \log_2\left(\frac{6M^2}{\epsilon}\right) = L_0 (\log_2(6) + \log_2(1/\epsilon) + 2 \log_2(M))$$

Thus overall, the network will have width

$$W_1 = \max\{W_{const}, 3W_0\}$$

and depth

$$L_{const} + L_0 (\log_2(6) + \log_2(1/\epsilon) + 2 \log_2(M))$$

For convenience, we can replace this expression of the depth with a larger depth which has a simpler formula, namely

$$L_{const} + L_0 (\log_2(6) + \log_2(1/\epsilon) + 2 \log_2(M)) \leq L_{const} + 2L_0 (\log_2(1/\epsilon) + \log_2(M)) \leq L_1 (1 + \log_2(1/\epsilon) + \log_2(M))$$

where

$$L_1 = \max\{L_{const}, 2L_0\}$$

□

We can think of this result as follows: up to now we knew that if f, g were NNFs then we can take linear combinations of them and get another NNF. We can also concatenate or take compositions and get an NNF. This result means that the same is almost true for the product $f \cdot g$. While we cannot exactly express $f \cdot g$ as an NNF, we can approximate it extremely well.

We now prove a similar lemma for the multi-dimensional product function

Proposition 2.20. *For every $s \in \mathbb{N}$ there exist $W_2 = W_2(s), L_2 = L_2(s)$ such that for every $1 > \epsilon > 0$, there exists a function $m \in \mathcal{FC}(W_2, L_2(1 + \log(1/\epsilon)), d_{in} = s, 1)$ with*

$$|x_1 \cdot x_2 \cdot \dots \cdot x_s - m(x)| < \epsilon, \quad \forall x = (x_1, \dots, x_s) \in [-1, 1]^s$$

Proof (we skipped this in class). For given $\epsilon > 0$, let $NN_\times$ be the function promised by Proposition 2.5 with respect to the parameters $\tilde{\epsilon} = \epsilon/(s-1)$ and $M = 2$. Define recursively the functions

$$m_1(x) = x_1, \quad m_2(x) = NN_\times(m_1(x), x_2), \quad p_3(x) = NN_\times(m_2(x), x_3) \quad \dots, \quad m_s(x) = NN_\times(m_{s-1}(x), x_s).$$

We can think of each m_k as an approximation of the product of the first k entries of x and $m := m_s$ is the final approximation we need. Note that we can apply Proposition 2.19 recursively to see that:

Since for all $x \in [-1, 1]^s$ we have that $|m_2(x) - x_2 \cdot m_1(x)| < \tilde{\epsilon}$ and both x_2 and $m_1(x) = x_1$ are in $[-1, 1]$, we deduce that $|m_2(x)| < 1 + \tilde{\epsilon} < 1 + 1/(s-1) < 2$.

Since $|m_2(x)| < 2$ we know that for all $x \in [-1, 1]^s$ we have $|m_3(x) - x_3(x)m_2(x)| < \tilde{\epsilon}$, and therefore $|m_3(x)| < 1 + 2/(s-1) < 2$

continuing recursively with this process we see that for all $j = 1, \dots, s-1$,

$$|m_{j+1}(x) - x_{j+1}m_j(x)| < \tilde{\epsilon}.$$

Using this we can see that the error of the approximation $m = m_s$ is bounded for all $x \in [-1, 1]^s$ by

$$\begin{aligned} |x_1 \cdot \dots \cdot x_s - m_s(x)| &\leq |m_s(x) - x_s m_{s-1}(x)| + |x_s m_{s-1}(x) - x_s x_{s-1} m_{s-2}(x)| + \dots \\ &\quad + |x_s x_{s-1} \dots x_3 m_2(x) - x_s x_{s-1} \dots x_2 m_1(x)| \\ &\leq \sum_{i=1}^{s-1} |m_{i+1}(x) - x_{i+1} m_i(x)| \\ &\leq (s-1) \tilde{\epsilon} = \epsilon \end{aligned}$$

A simple way to construct m_s as a neural networks is through the architecture

$$x \mapsto \begin{pmatrix} m_1(x) = x_1 \\ x_2 \\ \vdots \\ x_s \end{pmatrix} \mapsto \begin{pmatrix} m_2(x) = NN_\times(m_1(x), x_2) \\ x_3 \\ \vdots \\ x_s \end{pmatrix} \mapsto \begin{pmatrix} m_3(x) = NN_\times(m_2(x), x_3) \\ x_4 \\ \vdots \\ x_s \end{pmatrix} \dots \mapsto m_s(x)$$

The width of this construction does not depend on ϵ and the depth is proportional to $s \log(1/\tilde{\epsilon}) = s \log(s-1) + s \log(1/\epsilon)$. $\square$

2.2.3 Approximation of univariate smooth functions

We now consider approximation of functions from the function space

$$\mathcal{F}_n = \{f : [0, 1] \rightarrow \mathbb{R} \mid |f^{(k)}(x)| \leq 1, k = 0, \dots, n \text{ and } x \in [0, 1]\}.$$

We will prove the following

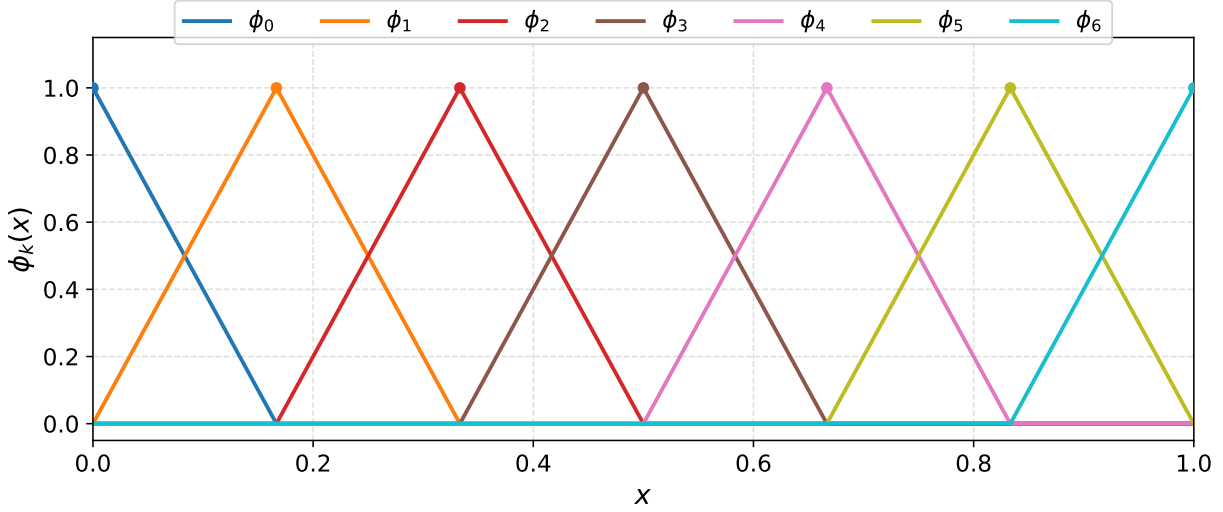
Theorem 2.21. *Let $n \in \mathbb{N}$. There exist $W = W(n), L = L(n)$ such that for every $\epsilon \in (0, 1)$ and every $f \in \mathcal{F}_n$, there exists $\tilde{f} \in \mathcal{FC}(W, L(1 + (1/\epsilon)^{1/n} \log(1/\epsilon)), d_{in} = 1, d_{out} = 1)$ with*

$$|f(x) - \tilde{f}(x)| < \epsilon, \quad \forall x \in [0, 1]$$

Remark 2.22. Note that this result can be used, in a sense, for any n times continuously differentiable f : since the derivatives are continuous they are bounded on a bounded interval, and so there will be some M for which

$$|f^{(k)}(x)| \leq M, \quad \forall k = 0, \dots, n \text{ and } x \in [0, 1].$$

We then have that $\frac{1}{M}f$ is in $\mathcal{F}_n$. If we use the theorem to obtain a neural network $\tilde{f}$ which approximates $\frac{1}{M}f$ to within an error of $\frac{\epsilon}{M}$, then $M\tilde{f}$ will approximate f to within an error of ϵ , and this would require $\sim (M/\epsilon)^{1/n}$ parameters (up to constant and logarithmic factors).

Figure 2.3: Illustration of the functions ϕ_k from the main text.

The idea of this proof is as following: we first consider approximation of f by a function space of ‘piecewise polynomials’, and then use what we showed so far to efficiently approximate these piecewise polynomials.

Let us first define our piecewise polynomials. We first consider CPwL functions $\phi_j, j = 0, \dots, N$ defined by the requirement that they are linear on the intervals $[k/N, (k+1)/N]$ for all $k = 0, \dots, N-1$, and satisfy

$$\phi_j(k/N) = \begin{cases} 1 & \text{if } k = j \\ 0 & \text{if } k \neq j \end{cases}.$$

These functions are illustrated in Figure 2.3, they have the following properties

1. Each ϕ_j is a univariate CPwL function defined by 3 knots, and hence can be expressed as a shallow neural network of width 4.
2. Non-negative $\phi_j(x) \geq 0$ for all $x \in \mathbb{R}, j = 0, \dots, N$.
3. Bounded support: for any $j = 0, \dots, N$ we have that $\phi_j(x) = 0$ if $|x - j/N| > 1/N$.
4. Sum to one: for any $j = 0, \dots, N$

$$\sum_{j=0}^N \phi_j(x) = 1, \forall x \in [0, 1] \quad (2.6)$$

Problem 2.23. Prove (2.6).

The functions $\phi_j, j = 0, \dots, N$ form a ‘partition of unity’ which allows us to approximate f by piecewise polynomials, using the fact that locally f is well approximated by Taylor polynomials:

Proposition 2.24. For any $n \in \mathbb{N}$, there exists some $C = C(n)$ such that for every $\epsilon > 0$ and every $f \in \mathcal{F}_n$ there exists $N \leq 1 + C(n)\epsilon^{-1/n}$ polynomials $p_0, p_1, \dots, p_N$ of degree $\leq n$ such that

$$|f(x) - \sum_{j=0}^N \phi_j(x)p_j(x)| < \frac{\epsilon}{2}, \quad \forall x \in [0, 1].$$

Moreover, each p_j is of the form

$$p_j(x) = \sum_{k=0}^{n-1} a_{j,k} \left(x - \frac{j}{N}\right)^k$$

where $|a_{j,k}| \leq 1$ for all j, k .

Proof. Assume we are given n , given $f \in \mathcal{F}_n$ and given $\epsilon > 0$. Assume we pick some N . For given $j = 0, \dots, N$, we define p_j to be the Taylor expansion of order $n - 1$ of f around j/N . So p_j is of the form

$$p_j(x) = \sum_{k=0}^n \frac{f^{(k)}(j/N)}{k!} \left(x - \frac{j}{N}\right)^k$$

and since the derivatives of f are all bounded by one we see that the coefficients of this polynomials ($a_{j,k}$ in the proposition) are all smaller than one.

Then we know that for all $x \in [0, 1]$ with $|x - j/N| \leq 1/N$, there exists some $c \in [0, 1]$ such that

$$|f(x) - p_j(x)| \leq \left| \frac{1}{n!} (x - j/N)^n f^{(n)}(c) \right| \leq \frac{1}{n!} (1/N)^n,$$

where we used the fact that the n -th derivative is bounded by one.

We want the expression on the right hand side to be smaller than $\epsilon/2$, which equivalently means

$$N \geq \left(\frac{2}{\epsilon n!} \right)^{1/n} = C(n)(1/\epsilon)^{1/n} \text{ for } C(n) = \left(\frac{2}{n!} \right)^{1/n}.$$

In particular, there exist such an N satisfying $N \leq 1 + C(n)(1/\epsilon)^{1/n}$. For this value of N we get for all $x \in [0, 1]$ that

$$\begin{aligned} |f(x) - \sum_{j=0}^N \phi_j(x) p_j(x)| &= \left| \sum_{j=0}^N \phi_j(x) (f(x) - p_j(x)) \right| \\ &\leq \sum_{j=0}^N \phi_j(x) |f(x) - p_j(x)| \\ &= \sum_{j: |x - j/N| < 1/N} \phi_j(x) |f(x) - p_j(x)| \\ &\leq \frac{\epsilon}{2} \sum_{j: |x - j/N| < 1/N} \phi_j(x) \\ &\leq \frac{\epsilon}{2} \sum_{j=1}^N \phi_j(x) = \epsilon/2 \end{aligned}$$

□

We now turn to prove Theorem 2.21, where we see that we can do almost as well with neural networks. The idea behind this claim is as follows: the construction we just saw is based on products of functions we can easily build with neural networks: they are a product of hat functions with polynomials, which themselves are linear combinations of products of linear functions. Since we know how to swiftly approximate products, we can successfully ‘imitate’ the approximation we just saw with neural networks.

Proof of Theorem 2.21. For given $\epsilon > 0$, let $N < C2^{1/n}(1/\epsilon)^{1/n}$ be such that the piecewise polynomials $P(x) := \sum_{j=0}^N \phi_j(x) p_j(x)$ defined in Proposition 2.24 are an $\epsilon/2$ approximation of f . We can rewrite P as

$$P(x) = \sum_{j=0}^N \sum_{k=0}^{n-1} a_{j,k} \underbrace{\phi_j(x) (x - j/N)^k}_{P_{j,k}(x)}$$

where by the proposition we have that $|a_{j,k}| \leq 1$. We will show that we can approximate each $P_{j,k}$ with a neural network $h_{j,k}$ up to accuracy of $\tilde{\epsilon} = \frac{\epsilon}{2n(N+1)}$. For fixed j, k , we know from Proposition 2.20 that there

is some $m_{k+1} \in \mathcal{FC}(W_2, L_2(1 + \log(1/\tilde{\epsilon}) + \log(k+1)), d_{in} = k+1, 1)$ (where W_2, L_2 depend on n , but not on ϵ) which can approximate the function $x_1 x_2 \dots x_{k+1}$ to accuracy of $\tilde{\epsilon}$. We thus have that

$$|P_{j,k}(x) - m_{k+1} \underbrace{\left(\phi_j(x), x - \frac{j}{N}, x - \frac{j}{N}, \dots, x - \frac{j}{N} \right)}_{h_{j,k}}| \leq \tilde{\epsilon}$$

and $h_{j,k}$ can be written as

$$x \mapsto \begin{pmatrix} \phi_j(x) \\ (x - j/N) \\ \vdots \\ (x - j/N) \end{pmatrix} \mapsto m_{k+1} \left(\phi_j(x), x - \frac{j}{N}, x - \frac{j}{N}, \dots, x - \frac{j}{N} \right) = h_{j,k}(x)$$

which is a neural network with width independent of $\tilde{\epsilon}$ and depth logarithmic in $1/\tilde{\epsilon}$ and hence in $1/\epsilon$. We can approximate f by $h = \sum_{j=0}^N \sum_{k=0}^{n-1} a_{j,k} h_{j,k}(x)$ with error

$$\begin{aligned} |f(x) - h(x)| &\leq |f(x) - P(x)| + |P(x) - h(x)| \leq \epsilon/2 + \sum_{j=0}^N \sum_{k=0}^{n-1} |a_{j,k}| |P_{j,k}(x) - h_{j,k}(x)| \\ &\leq \epsilon/2 + \sum_{j=0}^N \sum_{k=0}^{n-1} \tilde{\epsilon} = \epsilon. \end{aligned}$$

The network h is a sum of $(N+1)n \sim (1/\epsilon)^{1/n}$ networks $h_{j,k}$ with width W and depth proportional to $L \log(1/\epsilon)$, where the constants W, L depend on n but not on ϵ . In the homework you will show that this addition can be done with fixed width and depth which grows linearly in the number of functions (in our case, linearly in $(1/\epsilon)^{1/n}$). This concludes the proof. $\square$

2.2.4 Additional results

Approximation of multivariate smooth functions The paper [Yarotsky, 2017] focuses on the more general case of multivariate smooth functions. The function class $\mathcal{F}_{d,n}$ it discusses is functions $f : [0, 1]^d \rightarrow \mathbb{R}$ which have continuous partial derivatives of order $\leq n$ all bounded by one. The construction used in this case the story is similar to the 1D case: the function f is approximated by expression $\sum_{j=1}^N \phi_j(x) p_j(x)$ where p_j are multivariate polynomials of degree $\leq n-1$ and ϕ_j is a partition of unity supported on sets with small diameter δ . The number of parameters in this construction is $\sim (1/\epsilon)^{d/n}$, that is, it increases exponentially with the dimension and decreases exponentially with the smoothness, a known phenomena in approximation theory. Thus the ‘curse of dimensionality’ can be countered by smoothness. Neural network approximation in the multidimensional case again performs similarly to the piecewise polynomials up to a logarithmic factor $\sim (1/\epsilon)^{d/n}(1 + \log(1/\epsilon))$.

‘Super approximation’ of univariate functions For the function space $\mathcal{F}_{1,1} = \mathcal{F}_1$ the theorem we saw guarantees ϵ approximation with $\sim (1/\epsilon)$ parameters. This result can also be obtained using products of polynomials with partitions of unity: indeed, the proof used this. Thus, this does not yet give us an explanation why neural networks should perform *better* than locally supported polynomials or other constructions.

However [Yarotsky, 2017] showed that this can be improved to $\sim (1/(\epsilon \log(1/\epsilon)))$ and this was later improved even further to $\sim 1/\epsilon^2$ (see [DeVore et al., 2021]. Theorem 8.11). More generally, in [Lu et al., 2021] it is shown that approximation of functions on a d -dimensional cube $[0, 1]^d$ with s derivatives can be obtained with $\sim (1/\epsilon)^{\frac{d}{2s}}$ (up to logarithmic factors). We stress that unlike the previous results where neural networks achieved slightly worse rates than piecewise polynomials, these rates are actually better than what locally supported polynomials and other standard function bases can do...

Chapter 3

Equivariant Learning and Approximation Theory

New Lecture (June 8th)

In class we saw slides summarizing the first part of the course, and introducing the second part: Equivariant learning and approximation theory. These slides be found in Moodle.

New Lecture (June 9th)

3.1 Basic notions

Definition 3.1 (Group). A pair $(G, \cdot)$, where G is a set and $\cdot : G \times G \rightarrow G$ is called a group if

1. Associativity For all $a, b, c \in G$,

$$(a \cdot b) \cdot c = a \cdot (b \cdot c)$$

.

2. Identity There exists an element $e \in G$ such that

$$a \cdot e = a = e \cdot a, \forall a \in G.$$

3. Inverse element For every $a \in G$ there exists some element $b \in G$ such that

$$a \cdot b = e = b \cdot a.$$

We denote b by a^{-1} .

Examples: The pair $(\mathbb{R}^{n \times n}, +)$ is a group whose identity element is the zero matrix. The pair $(\mathbb{R}^{n \times n}, \cdot)$ is not a group (where $\cdot$ denotes matrix multiplication). The pair $(GL(n), \cdot)$, where

$$GL(n) = \{A \in \mathbb{R}^{n \times n} \mid \det(A) \neq 0\}$$

is a group whose identity element is the identity matrix I_n .

A subgroup of a group $(G, \cdot)$ is a subset H of G which is non-empty and is closed under products and inverses. Namely, if h, h' are in H , then also $h \cdot h'$ and h^{-1} are in H . In this case, $(H, \cdot)$ is a group, where $\cdot$ is the group action of G .

Examples of sub-groups of $GL(n)$ (with the action of matrix multiplication) include:

$$\begin{aligned} O(n) &= \{R \in \mathbb{R}^{n \times n} \mid RR^T = I_n\} \\ SO(n) &= \{R \in \mathbb{R}^{n \times n} \mid RR^T = I_n, \det(R) = 1\} \\ SL(n) &= \{A \in \mathbb{R}^{n \times n} \mid \det(A) = 1\} \end{aligned}$$

For our discussion next, the most important example will be the group of permutations. Permutations are bijections from $[n] := \{1, \dots, n\}$ to $[n]$. The set of all permutations is denoted by S_n . They are a group with respect to function composition, where the identity element is the identity function.

Definition 3.2 (Group Action). Let $(G, \cdot)$ be a group with an identity element e , and let S be a set. We say that $\rho : G \times S \rightarrow S$ is a group action if

1. For all $s \in S$,

$$\rho(e, s) = s.$$

2. for all $g, h \in G$ and $s \in S$ we have

$$\rho(g, \rho(h, s)) = \rho(g \cdot h, s)$$

Remark 3.3. We will use the notation

$$g.s = \rho(g, s).$$

With this notation the requirements of a group action can be written more conveniently as

$$e.s = s \text{ and } g.(h.s) = (g \cdot h).s$$

Example: A natural attempt to define a group action of S_n on $\mathbb{R}^n$ would be

$$[\sigma.x]_i = x_{\sigma(i)}, \sigma \in S_n, x \in \mathbb{R}^n, i = 1, \dots, n.$$

However as it turns out this is not a group action in our definition because

$$[\tau.(\sigma.x)]_i = [\sigma.x]_{\tau(i)} = x_{\sigma\tau(i)}$$

which generally may not be equal to

$$[(\tau \circ \sigma).x]_i = x_{\tau \circ \sigma(i)}$$

The correct way to define a group action of S_n on $\mathbb{R}^n$ is via the group action

$$[\sigma.x]_i = x_{\sigma^{-1}(i)}, \forall \sigma \in S_n, x \in \mathbb{R}^n, i = 1, \dots, n$$

Note that

$$[\sigma.(\tau.x)]_i = [\tau.x]_{\sigma^{-1}(i)} = x_{\tau^{-1} \circ \sigma^{-1}(i)} = x_{(\sigma \circ \tau)^{-1}(i)} = [(\sigma \circ \tau).x]_i$$

Example 3.4 (The action of permutations on matrices). Similarly, we can define an action of S_n on $\mathbb{R}^{d \times n}$ by

$$[\sigma(X)]_{ij} = X_{i\sigma^{-1}(j)}$$

Two ways to think of the permutation action We will devote a significant amount of time to this action. We will denote the n columns of X by $x_1, \dots, x_n$, and the d rows of X by $X_{1*}, \dots, X_{d*}$. The first way to think of the action we just defined is via swapping the columns of X

$$\sigma(X) = \sigma(x_1, \dots, x_n) = (x_{\sigma^{-1}(1)}, \dots, x_{\sigma^{-1}(n)}).$$

The second way we can think of it is applying the same permutation to each of the rows of X , namely

$$\sigma(X) = \sigma \left(\begin{pmatrix} X_{1*} \\ \vdots \\ X_{d*} \end{pmatrix} \right) = \begin{pmatrix} \sigma(X_{1*}) \\ \vdots \\ \sigma(X_{d*}) \end{pmatrix}.$$

Definition 3.5 (Invariant). Let S, T be sets, and let G be a group acting on S . We say that $f : S \rightarrow T$ is a G -invariant function if

$$f(g.s) = f(s), \forall g \in G, s \in S$$

Permutation invariant functions We will now focus on permutation invariant functions. Namely, functions $f : \mathbb{R}^{d \times n} \rightarrow \mathbb{R}^m$ satisfying

$$f(\tau.X) = f(X), \forall \tau \in S_n, X \in \mathbb{R}^{d \times n}.$$

Our motivation comes from permutation invariant functions which we would like to 'learn'. For example, consider recommendation problems: assume that you are working for a streaming service like netflix. You have a collection of n movies which a certain user watched, each characterized by a certain feature vector x_i . You want to find some recommendations for the user to watch. This can be encoded as a vector $y \in \mathbb{R}^m$ where each y_j encodes the probability that the user likes the j -th movie in netflix. Thus, we want to create a function of the form

$$f(x_1, \dots, x_n) = y.$$

It is reasonable to require that this function is permutation invariant- we would like two users who watched the same movies, possibly in different order, to get the same recommendations.

In permutation invariant learning, we want to build a collection of parameterized functions $\{h_\theta : \mathbb{R}^{d \times n} \rightarrow \mathbb{R}^m \mid \theta \in \mathbb{R}^p\}$ which are *permutation invariant* by construction. This generally reduces overfitting and leads to better generalization. Thus, we could just take standard ReLU neural networks and apply them to X , identifying the $d \times n$ matrix with a $n \cdot d$ vector, but this would not be permutation invariant by construction: for some (probably most) choices of parameters of the neural networks, permutations of X would lead to different results. Instead, we would like to consider parametric functions which are permutation invariant by construction.

To construct invariant parametric models, it is useful to introduce the notion of *equivariance*.

Definition 3.6 (Equivariant). Let $(G, \cdot)$ be a group, let S_1, S_2 be sets, and $\rho_i : G \times S_i \rightarrow S_i, i = 1, 2$ be group actions. We say that $f : S_1 \rightarrow S_2$ is equivariant if

$$f(\rho_1(g, s)) = \rho_2(g, f(s)), \forall g \in G, s \in S$$

It is important to note that invariance is a special case of equivariance: namely, if we take ρ_2 to be the trivial group action $\rho_2(g, x) = x$, then equivariance just means invariance:

$$f(\rho_1(g, s)) = \rho_2(g, f(s)) = f(s), \forall g \in G, s \in S$$

The following proposition is very useful for the construction of equivariant neural networks, since it means that we can construct equivariant networks by compositions of equivariant functions.

Proposition 3.7. Let $(G, \cdot)$ be a group acting on S_1, S_2, S_3 through group actions ρ_1, ρ_2, ρ_3 respectively. If $f_1 : S_1 \rightarrow S_2$ and $f_2 : S_2 \rightarrow S_3$ are equivariant then $f_2 \circ f_1$ is equivariant.

Proof. For every $g \in G, s \in S_1$ we have

$$f_2 \circ f_1(\rho_1(g, s)) = f_2(\rho_2(g, f_1(s))) = \rho_3(g, f_2 \circ f_1(s))$$

□

Of course, this proposition is true not only for 3 functions, but for any number of functions. Accordingly, to construct an equivariant function $f : S \rightarrow T$, where S and T are each endowed with an action of the same group G , we can construct a chain of intermediate spaces

$$S = S_0, S_1, \dots, S_L, S_{L+1} = T$$

each with their own action of G , and then construct functions spaces of the form

$$h = h_{L+1} \circ h_L \circ \dots \circ h_1$$

where for all $i = 1, \dots, L + 1$ the function $h_i : S_{i-1} \rightarrow S_i$ is equivariant. This will ensure that the final function $f : S \rightarrow T$ is equivariant.

3.1.1 Permutation equivariance for point sets

We will now focus for a while on construction of permutation invariant networks for $\mathbb{R}^{n \times d}$ and their theoretical properties.

We will consider a simple family of permutation invariant functions which generalizes the proposal in the famous PointNet [Qi et al., 2017] and Deepsets [Zaheer et al., 2017] papers: Namely, we consider functions of the form

$$H_\theta(X) = h_{\theta_2} \circ \text{Pool} \circ Q_{\theta_1}(X) \quad (3.1)$$

where $\theta = (\theta_1, \theta_2)$ and

1. $Q_{\theta_1} : \mathbb{R}^{d \times n} \rightarrow \mathbb{R}^{k \times n}$ is permutation equivariant. Namely, for every θ_1 we have

$$Q_{\theta_1}(\tau X) = \tau(Q_{\theta_1}(X))$$

Here, we consider the standard action of a permutation τ on $\mathbb{R}^{d \times n}$ and $\mathbb{R}^{k \times n}$.

2. $\text{Pool} : \mathbb{R}^{k \times n} \rightarrow \mathbb{R}^a$ is permutation invariant function, namely

$$\text{Pool}(\tau.Y) = \text{Pool}(Y)$$

We can also think of this as an equivariant function where the action of τ on $\mathbb{R}^a$ is the trivial action.

3. $h_{\theta_2} : \mathbb{R}^a \rightarrow \mathbb{R}^m$ is a standard neural network. We can also think of each h_{θ_2} as an equivariant function where the action of τ on $\mathbb{R}^a$ and $\mathbb{R}^m$ is the trivial action, because in this case equivariance does not mean anything: for any function we will have

$$h_{\theta_2}(\tau.y) = h_{\theta_2}(y) = \tau.h_{\theta_2}(y).$$

According to what we say, $H_\theta(X)$ will be a permutation invariant function for all θ . Explicitly,

$$H_\theta(\tau.X) = h_{\theta_2} \circ \text{Pool} \circ Q_{\theta_1}(\tau.X) = h_{\theta_2} \circ \text{Pool}(\tau.Q_{\theta_1}(X)) = h_{\theta_2} \circ \text{Pool} \circ Q_{\theta_1}(X) = H_\theta(X), \quad \forall X \in \mathbb{R}^{d \times n}, \tau \in S_n$$

We now discuss how to construct (1) permutation equivariant function and (2) invariant 'pooling' functions.

Pointwise Permutation equivariant models The simplest way to construct permutation equivariant models is via elementwise application of a ReLU network. Namely, Let $q_{\theta_1} : \mathbb{R}^d \rightarrow \mathbb{R}^k$ denote a ReLU neural network. And define

$$Q_{\theta_1}(x_1, \dots, x_n) = (q_{\theta_1}(x_1), \dots, q_{\theta_1}(x_n))$$

This is clearly equivariant because for every permutation τ and $X \in \mathbb{R}^{d \times n}$,

$$Q_{\theta_1}(\tau.X) = (q_{\theta_1}(x_{\tau^{-1}(1)}), \dots, q_{\theta_1}(x_{\tau^{-1}(n)})) = \tau.(q_{\theta_1}(x_1), \dots, q_{\theta_1}(x_n)) = \tau.(Q_{\theta_1}(X))$$

Equivariant Self Attention A disadvantage of the pointwise approach applied before is that it cannot take into account relationships between pairs of points. The popular way to solve this is via self attention, which is a permutation equivariant operation used in the set transformer model [Lee et al., 2019], which is a permutation equivariant version of the popular transformer model.

For given natural numbers n, d, d', d_{out} , Self attention is a mapping $Z_\theta(X) : \mathbb{R}^{d \times n} \rightarrow \mathbb{R}^{d_{out} \times n}$, where θ denotes the concatenation of three matrices $W_K \in \mathbb{R}^{d' \times d}, W_Q \in \mathbb{R}^{d' \times d}, W_V \in \mathbb{R}^{d_{out} \times d}$ denoting the parameters of the function. For a given input $X = (x_1, \dots, x_n) \in \mathbb{R}^{d \times n}$, this function is defined as follows

$$\begin{aligned} k_j &= W_K x_j, \quad q_j = W_Q x_j, \quad v_j = W_V x_j, \quad j = 1, \dots, n \\ S_{i,j} &= \frac{\exp(q_i \cdot k_j)}{\sum_{\ell=1}^n \exp(q_i \cdot k_\ell)} \\ z_i &= \sum_{j=1}^n S_{i,j} v_j, \quad i = 1, \dots, n \\ Z_\theta(X) &:= (z_1, \dots, z_n) \end{aligned}$$

The interpretation of this is as follows: each point x_j is associated with three vectors: key k_j , query q_j , and value v_j . The key and query are used to assign a weight $S_{i,j}$ signifying the 'relevance' of x_j to x_i , and then the final new value of z_i is taken to be a weighted sum of all the vectors v_j , weighted according to $S_{i,j}$.

Proposition 3.8. *For any choice of parameters θ , the self attention function Z_θ is permutation equivariant.*

Proof. We need to show that for any given $X \in \mathbb{R}^{d \times n}$, parameters $\theta = (W_K, W_Q, W_V)$ and permutation $\tau \in S_n$, we have that $\tau Z_\theta(X) = Z_\theta(\tau X)$.

Let $k_j, q_j, v_j, S_{i,j}$ and z_i denote the outputs obtained from applying self attention to X , and let $\tilde{k}_j, \tilde{q}_j, \tilde{v}_j, \tilde{S}_{i,j}$ and $\tilde{z}_i$ denote the outputs obtained from applying self attention to $\tau X = (x_{\tau^{-1}(1)}, \dots, x_{\tau^{-1}(n)})$. Then, we need to show that

$$\tilde{z}_i = z_{\tau^{-1}(i)}, \quad \forall i = 1, \dots, n.$$

Indeed, note that

$$\tilde{k}_j = W_K x_{\tau^{-1}(j)} = k_{\tau^{-1}(j)}. \text{ Similarly } \tilde{q}_j = q_{\tau^{-1}(j)}, \quad \tilde{v}_j = v_{\tau^{-1}(j)}, \quad j = 1, \dots, n$$

It follows that

$$\tilde{S}_{i,j} = \frac{\exp(\tilde{q}_i \cdot \tilde{k}_j)}{\sum_{\ell=1}^n \exp(\tilde{q}_i \cdot \tilde{k}_\ell)} = \frac{\exp(q_{\tau^{-1}(i)} \cdot k_{\tau^{-1}(j)})}{\sum_{\ell=1}^n \exp(q_{\tau^{-1}(i)} \cdot k_{\tau^{-1}(\ell)})} = \frac{\exp(q_{\tau^{-1}(i)} \cdot k_{\tau^{-1}(j)})}{\sum_{\ell=1}^n \exp(q_{\tau^{-1}(i)} \cdot k_\ell)} = S_{\tau^{-1}(i), \tau^{-1}(j)}$$

and therefore, for all $i = 1, \dots, n$,

$$\tilde{z}_i = \sum_{j=1}^n \tilde{S}_{i,j} \tilde{v}_j = \sum_{j=1}^n S_{\tau^{-1}(i), \tau^{-1}(j)} v_{\tau^{-1}(j)} = \sum_{j=1}^n S_{\tau^{-1}(i), j} v_j = z_{\tau^{-1}(i)}$$

□

Remark 3.9 (Size agnostic). In application it is often desirable to be able to process point sets of different cardinalities: for example, in the Netflix example we discussed before, the number of movies each user watched (denoted by n) usually differs between users. In this context, it is useful that the number of parameters in the models we defined do not depend on n , and that the functions we defined can essentially be applied for all values of n , provided that d is held fixed.

Remark 3.10 (Positional encoding). Permutation equivariance is a desirable property in point cloud processing, but not so in other applications of self-attention, such as transformers for language and vision applications: for example, in language applications, text is converted to a sequence $x_1, \dots, x_n$ where the x_j are 'tokens': vectors representing words or other text fragments. The raw self-attention mechanism would treat this sequence as a 'bag of words', and would not be able to distinguish between 'A wolf ate a man' and 'A man ate a wolf'. To avoid this problem, the position of each token x_j in the document is appended to x_j , obtaining a new representation $(x_j, f(j))$ for an appropriate function f . This process is called 'positional encoding'.

Moving forwards we will not discuss equivariant self attention, and only focus on the simpler elementwise layers.

3.1.2 Invariant pooling operations

As discussed earlier, to produce a function space of invariant functions $h : \mathbb{R}^{a \times n} \rightarrow \mathbb{R}^c$, we can construct an equivariant function space $h : \mathbb{R}^{a \times n} \rightarrow \mathbb{R}^{b \times n}$, and then compose these equivariant functions with a simple invariant function $Pool : \mathbb{R}^{b \times n} \rightarrow \mathbb{R}^c$. Examples include:

1. **Max pooling** Here

$$\text{rowmax}(X) = \text{rowmax} \left(\begin{pmatrix} X_{1*} \\ \vdots \\ X_{d*} \end{pmatrix} \right) = \begin{pmatrix} \max(X_{1*}) \\ \vdots \\ \max(X_{d*}) \end{pmatrix}$$

This is invariant because the maximum of a vector is invariant to ordering, and therefore for every permutation σ ,

$$\text{rowmax}(\sigma X) = \text{rowmax} \left(\begin{pmatrix} X_{1*} \\ \dots \\ X_{d*} \end{pmatrix} \right) = \begin{pmatrix} \max(\sigma(X_{1*})) \\ \dots \\ \max(\sigma(X_{d*})) \end{pmatrix} = \begin{pmatrix} \max(X_{1*}) \\ \dots \\ \max(X_{d*}) \end{pmatrix} = \text{rowmax}(X)$$

Here $b = c$

2. **sum-pooling** Summing each one of the rows independently. Since summation of a vector is permutation invariant, sum-pooling is permutation invariant. Here again $b = c$

In summary, we have obtained two types of parameter invariant models, using sum and max pooling. We can build a hypothesis class of invariant functions from $\mathbb{R}^{a \times n} \rightarrow \mathbb{R}^b$ by choosing two fully connected functions space $\mathcal{F}_1 = \mathcal{FC}(W_1, L_1; w_{in} = a, w_{out} = d', \sigma_1)$ and $\mathcal{F}_2 = \mathcal{FC}(W_2, L_2; w_{in} = d', w_{out} = b, \sigma_2)$

which then gives us function spaces

$$\mathcal{H}_{\text{deepsets}}^{\text{max}}(\mathcal{F}_1, \mathcal{F}_2) = \{H : \mathbb{R}^{a \times n} \rightarrow \mathbb{R}^b \mid H(x_1, \dots, x_n) = h_{\theta_2} \circ \text{rowmax} \circ (q_{\theta_1}(x_1), \dots, q_{\theta_1}(x_n))\}$$

where $q_{\theta_1} \in \mathcal{F}_1$ and $h_{\theta_2} \in \mathcal{F}_2$

$$\mathcal{H}_{\text{deepsets}}^{\text{sum}}(\mathcal{F}_1, \mathcal{F}_2) = \{H : \mathbb{R}^{a \times n} \rightarrow \mathbb{R}^b \mid H(x_1, \dots, x_n) = h_{\theta_2} \circ \text{rowsum} \circ (q_{\theta_1}(x_1), \dots, q_{\theta_1}(x_n))\}$$

where $q_{\theta_1} \in \mathcal{F}_1$ and $h_{\theta_2} \in \mathcal{F}_2$

For a fixed choice of $(W_1, W_2, L_1, L_2, w_{in}, w_{out}, d')$ each one of these models can be parameterized by matrices of fixed size, and then we can use gradient descent based learning.

The goal of our theoretical analysis will be to understand whether these models are universal: Each one of these models represents permutation invariant functions from $\mathbb{R}^{a \times n}$ to $\mathbb{R}^b$. But can they approximate all such functions, as we let the size of the functions (namely the parameters W_1, W_2, L_1, L_2 go to infinity? For simplicity, we will let $\mathcal{H}_{\text{deepsets}}^{\text{max}}$ to denote all functions which are in some space $\mathcal{H}_{\text{deepsets}}^{\text{max}}(\mathcal{F}_1, \mathcal{F}_2)$ for some choice of $\mathcal{F}_1, \mathcal{F}_2$, and use the corresponding notation for $\mathcal{H}_{\text{deepsets}}^{\text{sum}}$ as well.

As it turns out, the answer for the question of universality of $\mathcal{H}_{\text{deepsets}}^{\text{max}}$ is negative, and the answer for $\mathcal{H}_{\text{deepsets}}^{\text{sum}}$ is positive. The negative result is discussed in [Xu et al., 2018], and we will begin from it.

$\mathcal{H}_{\text{deepsets}}^{\text{max}}$ is not a universal approximator of permutation invariant functions Assume that $n \geq 3, d \geq 1$. Let us construct two point sets in $\mathbb{R}^{d \times n}$ which have the same elements, repeated a different number of times. We will choose

$$X = (1_d, 0_d, 0_d, \dots, 0_d) \text{ and } Y = (1_d, 1_d, 0_d, \dots, 0_d).$$

Now, note that any function h in $\mathcal{H}_{\text{deepsets}}^{\text{max}}$ will assign X and Y the same value, that is $h(X) = h(Y)$. As a results, such functions will not be a good approximation of invariant functions which can separate X from Y . For example, we could take the function $f(Z) = \sum_{1 \leq i \leq n, 1 \leq j \leq d} Z_{ij}$. Then

$$f(X) = d \neq 2d = f(Y)$$

and as a result for every h in $\mathcal{H}_{\text{deepsets}}^{\text{max}}$,

$$\max_{Z \in [0,1]^{nd}} |f(Z) - h(Z)| \geq \frac{1}{2} (|f(X) - h(X)| + |h(Y) - f(Y)|) \geq \frac{1}{2} |f(X) - f(Y)| = d/2$$

so this invariant function f cannot be approximated uniformly on $[0,1]^{nd}$ by functions in $\mathcal{H}_{\text{deepsets}}^{\text{max}}$.

For $\mathcal{H}_{\text{deepsets}}^{\text{sum}}$ the story is different, and we do have universality. This result will require some work which we will now begin:

New Lessons June 15-June 16

Our goal now is to prove that all permutation invariant functions on $\mathbb{R}^{d \times n}$ can be approximated uniformly on a compact set by functions in $\mathcal{H}_{\text{deepsets}}^{\text{sum}}$. We will do this by introducing a general framework to approximate invariant functions. To illustrate the generality, we will focus throughout simultaneously on two examples, the action of S_n on $\mathbb{R}^{d \times n}$, and the simple action of $\{-1, 1\}$ on $\mathbb{R}$ by multiplication.

Our methodology for universal approximation of invariant functions goes through the notion of quotient spaces:

3.1.3 Invariant functions and functions on quotient spaces

When a group G acts on a set V , an **orbit** of a point v is the set

$$[v] := \{g.v | g \in G\}.$$

The collection of all orbits is denoted by V/G . We call V/G the **quotient space**.

Example 3.11. The group $G = \{-1, 1\}$ acts on $\mathbb{R}$ by multiplication. We have that $[x] = [y]$ if and only if $|x| = |y|$, so we can identify $\mathbb{R}/G$ with $[0, \infty)$.

Invariant functions=functions on the quotient space There is a one to one correspondence between invariant functions and functions on the quotient space:

Let G be a group acting on a set V , and let $\tilde{f} : V/G \rightarrow Y$ be a function, where Y is some set. This induces a function $f : V \rightarrow Y$ defined by

$$f(v) = \tilde{f}([v]).$$

This function is invariant because for every $g \in G, v \in V$,

$$f(gv) = \tilde{f}([gv]) = \tilde{f}([v]) = f(v).$$

Conversely, if $f : V \rightarrow Y$ is invariant then we can define a function $\tilde{f}$ on the quotient space by

$$\tilde{f}([v]) = f(v)$$

this definition does not depend on the choice of the representative in the orbit $[v]$ because f is invariant.

Metrics on the quotient space To use tools like the Stone Weierstrass theorem on the quotient space, we will need to define a metric on the quotient space. This will require some additional assumptions, which will all be valid in the setting which we are interested in. In homework you will prove the following:

Problem 3.12 (Homework).

Let G be a group acting on a real finite dimensional normed vector space V . For every $x, y \in V$ define

$$d(x, y) = \inf_{g \in G} \|x - gy\| \tag{3.2}$$

Assume that

1. The norm is G invariant, in the sense that $\|gx - gy\| = \|x - y\|$ for all $g \in G$ and all $x, y \in V$.
2. The infimum in (3.2) is always obtained (the infimum is a minimum).

Let $[x]$ denote the orbit of $x \in V$ under the G action, and let V/G denote the space of all orbits. Denote

$$\tilde{d}([x], [y]) = d(x, y)$$

Prove that

1. $\tilde{d}$ is well defined (does not depend on choice of representative in the two orbits).
2. $\tilde{d}$ is a metric on V/G .

3. the map $q(x) = [x]$ from V to V/G is continuous.
4. An invariant function $f : V \rightarrow \mathbb{R}^m$ is continuous, if and only if the induced function $\tilde{f} : V/G \rightarrow \mathbb{R}^m$ is continuous.

note: The condition that the norm is G invariant is equivalent to requiring that $\|gx\| = \|x\|$ for all $g \in G$ and $x \in V$.

In particular, the assumptions of this problem are valid in the two examples we are currently interested in. Choosing $V = \mathbb{R}$ and $G = \{-1, 1\}$ with the standard absolute value norm $|x|$ on $\mathbb{R}$, we have that the norm is G invariant, and the infimum in (3.2) is always obtained, since the group is finite. Therefore, the metric $\tilde{d}$ is well defined on the quotient space, and is given explicitly by

$$\tilde{d}([x], [y]) = \min\{|x - y|, |x + y|\}.$$

Similarly, choosing $V = \mathbb{R}^{d \times n}$ with the standard Frobenius norm on matrices, we will have that the norm is invariant to the action of $G = S_n$. Moreover, the infimum is always obtained since the group is finite. Thus, we will get a valid metric $\tilde{d}$ on the quotient space. This is the metric

$$\tilde{d}([X], [Y]) = \min_{\tau \in S_n} \|X - \tau Y\|_F = \min_{\tau \in S_n} \sqrt{\sum_{j=1}^n \|x_j - y_{\tau(j)}\|_2^2}$$

3.1.4 Separating Invariants and Universality

Definition 3.13. Let G be a group acting on a set V . Let $\mathcal{S}$ be a collection of functions from V to $\mathbb{R}$. We say that $\mathcal{S}$ is a separating collection of continuous invariant mappings if

1. **Invariance** All mappings $s \in \mathcal{S}$ are invariant and continuous.
2. **Separation** If $x, y \in V$ and $[x] \neq [y]$, then there exists some $s \in \mathcal{S}$ such that $s(x) \neq s(y)$.

Example 3.14. For the example of the action of $G = \{-1, 1\}$ on $\mathbb{R}$, an example is the single mapping $s(x) = |x|$, so $\mathcal{S} = \{s\}$. Another example is the single mapping $\mathcal{S} = \{x \mapsto x^2\}$. In general $\mathcal{S}$ could be a finite or infinite family of functions.

Using this general notion of separation, we can formulate a general approximation result, based on the Stone-Weierstrass theorem. Our focus will be on the function spaces $\mathcal{N}(V, \mathcal{S})$, which are defined somewhat differently for finite and infinite sets $\mathcal{S}$. For finite $\mathcal{S} = \{s_1, \dots, s_m\}$, we will denote

$$\mathcal{N}(V, \mathcal{S}) = \{h(s_1(x), \dots, s_m(x)) \mid h : \mathbb{R}^m \rightarrow \mathbb{R} \text{ is a fully connected neural network}\}$$

For infinite $\mathcal{S}$ we will define

$$\mathcal{N}(V, \mathcal{S}) = \{h(s_1(x), \dots, s_m(x)) \mid \text{for some } m \in \mathbb{N} \text{ and } s_1, \dots, s_m \in \mathcal{S} \text{ and } h : \mathbb{R}^m \rightarrow \mathbb{R} \text{ is a fully connected neural network}\} \quad (3.3)$$

where the difference from the finite case is that m can be arbitrarily large.

In the following theorem we will show the universality of $\mathcal{N}(V, \mathcal{S})$:

Theorem 3.15. *Let G be a group acting on a normed real vector space V , and assume that the assumptions of Problem 3.12 are satisfied. Let $K \subseteq V$ be a compact subset. Let $\mathcal{S} \subseteq C(V)$ be a separating family of continuous invariant functions. Then for every invariant $f \in C(V)$ and any $\epsilon > 0$, there exists $h(s_1(x), \dots, s_m(x)) \in \mathcal{N}(V, \mathcal{S})$ such that*

$$|f(x) - h(s_1(x), \dots, s_m(x))| < \epsilon$$

Proof. Recall from homework that (3.2) induces a metric on the quotient space $\tilde{d}([x], [y]) = d(x, y)$, and that the quotient map $q(x) = [x]$ is continuous with respect to this metric on the quotient space. Recall also that every $f : V \rightarrow \mathbb{R}^m$ which is invariant can be identified with a function $\tilde{f} : V/G \rightarrow \mathbb{R}^m$. and by the homework f is continuous if and only if $\tilde{f}$ is continuous.

Assume we are given $\epsilon > 0$. and $f : V \rightarrow \mathbb{R}$ continuous and invariant. We would like to approximate the function $\tilde{f}$ on the compact set $q(K) = \tilde{K}$ (image of a compact set under a continuous function is compact). In first step, we will show that every such f can be approximated by functions in the set $C(\tilde{K}, \mathcal{S})$, which for infinite $\mathcal{S}$ is defined via

$$C(\tilde{K}, \mathcal{S}) = \{ \psi(\tilde{s}_1([x]), \dots, \tilde{s}_m([x])) \mid \text{for some } m \in \mathbb{N} \text{ and } s_1, \dots, s_m \in \mathcal{S} \text{ and } \psi : \mathbb{R}^m \rightarrow \mathbb{R} \text{ continuous} \} \quad (3.4)$$

where $\tilde{s}_j$ denotes the function on the quotient space $\tilde{K}$ satisfying $\tilde{s}_j([x]) = s_j(x)$ for all $x \in K$. For finite $\mathcal{S} = \{s_1, \dots, s_m\}$, we define

$$C(\tilde{K}, \mathcal{S}) = \{ \psi(\tilde{s}_1([x]), \dots, \tilde{s}_m([x])) \mid \text{for some } \psi : \mathbb{R}^m \rightarrow \mathbb{R} \text{ continuous} \}$$

We claim that $C(\tilde{K}, \mathcal{S}) \subseteq C(\tilde{K})$ satisfies all the conditions of the Stone Weierstrass theorem:

1. The functions in $C(\tilde{K}, \mathcal{S})$ are continuous, since all functions in $\mathcal{S}$ are continuous and therefore the corresponding functions $\tilde{s}$ are continuous as well, by Problem 3.12.
2. The function space $C(\tilde{K}, \mathcal{S})$ is a subalgebra of $C(\tilde{K})$ which contains the function $\psi([x]) = 1$.
3. The function space $C(\tilde{K}, \mathcal{S})$ is separating. To see this, for every $x, y \in K$ with $[x] \neq [y]$, we have that there exists some $s_1 \in \mathcal{S}$ such that $s_1(x) \neq s_1(y)$ and thus $\tilde{s}_1([x]) = s_1(x) \neq s_1(y) = \tilde{s}_1([y])$. Now for any extension of s_1 to $s_1, \dots, s_m$, we can choose $\psi : \mathbb{R}^m \rightarrow \mathbb{R}$ to be the function $\psi(t_1, \dots, t_m) = t_1$ and then we obtain

$$\psi(\tilde{s}_1([x]), \dots, \tilde{s}_m([x])) = \tilde{s}_1([x]) \neq \tilde{s}_1([y]) = \psi(\tilde{s}_1([y]), \dots, \tilde{s}_m([y]))$$

By Stone Weierstrass we have for a given $\epsilon > 0$ that for appropriate m and $s_1, \dots, s_m \in \mathcal{S}$ and $\psi : \mathbb{R}^m \rightarrow \mathbb{R}$ continuous, that

$$|\tilde{f}([x]) - \psi(\tilde{s}_1([x]), \dots, \tilde{s}_m([x]))| \leq \epsilon/2, \quad \forall x \in K.$$

It remains only to replace ψ by a neural network. Indeed, the image of the set $\tilde{K}$ under the continuous function

$$\tilde{K} \ni [x] \mapsto (\tilde{s}_1([x]), \dots, \tilde{s}_m([x]))$$

is a compact set, and by the universality theorem, ψ can be approximated to $\epsilon/2$ accuracy on this compact set by a neural network h . For this h we obtain

$$\begin{aligned} |f(x) - h(s_1(x), \dots, s_m(x))| &= |\tilde{f}([x]) - h(\tilde{s}_1([x]), \dots, \tilde{s}_m([x]))| \\ &\leq |\tilde{f}([x]) - \psi(\tilde{s}_1([x]), \dots, \tilde{s}_m([x]))| + |\psi(\tilde{s}_1([x]), \dots, \tilde{s}_m([x])) - h(\tilde{s}_1([x]), \dots, \tilde{s}_m([x]))| \\ &\leq \epsilon/2 + \epsilon/2. \end{aligned}$$

□

Using this theorem, we can first address our first simple example

Corollary 3.16. *Let $f : [-1, 1] \rightarrow \mathbb{R}$ be a continuous function which is even, that is $f(x) = f(-x)$. Then for every $\epsilon > 0$ there exists a neural network $h : \mathbb{R} \rightarrow \mathbb{R}$ such that*

$$|f(x) - h(x^2)| < \epsilon, \quad \forall x \in [-1, 1]$$

Proof. Apply Theorem 3.15 with $\mathcal{S} = \{x \mapsto x^2\}$.

□

To move to universality of $\mathcal{H}_{\text{deepsets}}^{\text{sum}}$ for permutation invariant functions, we will need to prove a non-trivial separation result:

Lemma 3.17. *Fix some non-polynomial continuous $\sigma : \mathbb{R} \rightarrow \mathbb{R}$. Let $X, Y \in \mathbb{R}^{d \times n}$ and assume that X and Y are not related by a permutation in S_n . Then there exists some (shallow) neural network $h : \mathbb{R}^d \rightarrow \mathbb{R}$ with activation σ , such that*

$$\sum_{j=1}^n h(x_j) \neq \sum_{j=1}^n h(y_j)$$

Before we prove this result, we will explain how it implies the universality of $\mathcal{H}_{\text{deepsets}}^{\text{sum}}$:

Theorem 3.18. *Let $f : [0, 1]^{d \times n} \rightarrow \mathbb{R}$ be a continuous, S_n invariant function. Then, for any $\epsilon > 0$, there exists $m \in \mathbb{N}$, neural networks $h' : \mathbb{R}^m \rightarrow \mathbb{R}, h : \mathbb{R}^d \rightarrow \mathbb{R}^m$ such that*

$$|f(X) - h' \left(\sum_{j=1}^n h(x_j) \right)| \leq \epsilon, \quad \forall X \in [0, 1]^{d \times n}$$

Proof. By Lemma 3.17, the set

$$\mathcal{S} = \left\{ s(X) = \sum_{j=1}^n h(x_j), \text{ where } h : \mathbb{R}^d \rightarrow \mathbb{R} \text{ is a fully connected neural network} \right\}$$

is a separating set of continuous invariant functions. By Theorem 3.15, for every continuous permutation invariant $f : \mathbb{R}^{d \times n} \rightarrow \mathbb{R}$, and every $\epsilon > 0$, there exists $m \in \mathbb{N}$ and neural networks $s_1(X) = \sum_{j=1}^n h_1(x_j), \dots, s_m(X) = \sum_{j=1}^n h_m(x_j)$ and a neural network $h' : \mathbb{R}^m \rightarrow \mathbb{R}$, such that

$$|f(X) - h' \left(\sum_{j=1}^n h_1(x_j), \dots, \sum_{j=1}^n h_m(x_j) \right)| < \epsilon.$$

Redefining $h(x) = (h_1(x), \dots, h_m(x))$ and noting that h is a neural network, we get the required result. $\square$

We now return to prove Lemma 3.17:

Proof of Lemma 3.17. Assume by contradiction the theorem is not true. Then we would have a pair

$$X = (x_1, \dots, x_n) \text{ and } Y = (y_1, \dots, y_n)$$

such that X, Y are not related by a permutation, and yet

$$\sum_{j=1}^n \sigma(a^T x_j + b) = \sum_{j=1}^n \sigma(a^T y_j + b), \forall a \in \mathbb{R}^d, b \in \mathbb{R}.$$

It follows that for all $K \in \mathbb{N}$, and all $c_1, \dots, c_K \in \mathbb{R}$ and $a_1, \dots, a_K \in \mathbb{R}^d$ and $b_1, \dots, b_K \in \mathbb{R}$

$$\sum_{k=1}^K c_k \sum_{j=1}^n \sigma(a_k^T x_j + b_k) = \sum_{k=1}^K c_k \sum_{j=1}^n \sigma(a_k^T y_j + b_k).$$

We can now apply the universality theorem: since σ is continuous and not a polynomial, any continuous function f defined on the compact set $K = \{x_1, \dots, x_n, y_1, \dots, y_n\}$ can be approximated to arbitrarily accuracy by functions of the form $\sum_{k=1}^K c_k \sigma(a_k^T x_j + b_k)$. It follows that for *every* continuous $f : K \rightarrow \mathbb{R}$, we have that

$$\sum_{j=1}^n f(x_j) = \sum_{j=1}^n f(y_j).$$

Note that for a finite set K , and function $f : K \rightarrow \mathbb{R}$ is a continuous function, and thus the inequality above holds for any such functions. To prove contradiction, we need only to show that this is not true. Indeed, there must be a point x_i which appears k_X times in X , and $k_Y \neq k_X$ times in Y . We can then choose a continuous function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ such that $f(x_i) = 1$, and such that for all $z \in \{x_1, \dots, x_n, y_1, \dots, y_n\} \setminus \{x_i\}$ we will have $f(z) = 0$. Thus

$$\sum_{j=1}^n f(x_j) = k_X \neq k_Y = \sum_{j=1}^n f(y_j),$$

which gives us a contradiction and concludes the proof. $\square$

As an example to illustrate the end of the proof consider the pair of non-equivalent $X, Y \in \mathbb{R}^5$ (here we think of $\mathbb{R}^5$ as $\mathbb{R}^{d \times n}$ with $n = 5, d = 1$)

$$X = (1, 1, 2, 2, 3), \quad Y = (1, 1, 2, 3, 3)$$

then $x_3 = 2$ appears $n_X = 2$ times in X and $n_Y = 1$ times in Y , and for $K = \{1, 2, 3\}$ the function f can be constructed as

$$f(x) = \begin{cases} 1 & \text{if } x = 2 \\ 0 & \text{if } x \in \{1, 3\} \end{cases}$$

New Lesson June 21

3.1.5 Universal Equivariant models

Assume we wish to learn a function in the equivariant function class

$$\mathcal{F}_{\text{equiv}} = \{f : \mathbb{R}^{d \times n} \rightarrow \mathbb{R}^n \mid f \text{ is continuous and } S_n \text{ equivariant}\}$$

let us first recall that this means that $f(\tau.X) = \tau.f(X)$ for every permutation τ and $X \in \mathbb{R}^{d \times n}$. In other words, permutations of the input correspond to permutation of the output. In class we discussed examples of learning problems which have this equivariant structure, such as point cloud segmentation and molecular dynamics.

Recall that we use the notation $X = (x_1, \dots, x_n)$ where each x_j is the j th column in X , a vector in $\mathbb{R}^d$. Also note that any function $f : \mathbb{R}^{d \times n} \rightarrow \mathbb{R}^n$ can be written as

$$f(X) = (f_1(X), \dots, f_n(X)),$$

where $f_j : \mathbb{R}^{d \times n} \rightarrow \mathbb{R}^n$. Equivariance is equivalent to requiring that

$$f_j(x_{\tau(1)}, \dots, x_{\tau(n)}) = f_{\tau(j)}(x_1, \dots, x_n), \quad \forall j \in \{1, \dots, n\}, \tau \in S_n, X \in \mathbb{R}^{d \times n} \quad (3.5)$$

We want to construct a parametric family of function $\mathcal{H}_{\text{equiv}}$ which is contained in $\mathcal{F}_{\text{equiv}}$ (that is, all hypothesis are equivariant and continuous) and is dense uniformly on compact sets. Our discussion of this topic is somewhat based on [Segol and Lipman, 2019, Villar et al., 2021].

In the context of the pointnet architecture, a natural choice would be

$$\mathcal{H}_{\text{pointnet-equiv}} = \{h(x_1, \dots, x_n) = (\hat{h}(x_1), \dots, \hat{h}(x_n)) \mid \hat{h} : \mathbb{R}^d \rightarrow \mathbb{R} \text{ is a fully connected neural network}\}$$

Note that functions in this function space are indeed equivariant, as

$$h_j((x_{\tau(1)}, \dots, x_{\tau(n)})) = \hat{h}(x_{\tau(j)}) = h_{\tau(j)}(x_1, \dots, x_n), \quad \forall j \in \{1, \dots, n\}, \tau \in S_n, X \in \mathbb{R}^{d \times n}$$

and so (3.5) holds. This leads to the following natural question:

Problem 3.19. Is $\mathcal{H}_{\text{pointnet-equiv}}$ dense in $\mathcal{F}_{\text{equiv}}$?

The answer is negative, as the following proposition shows (in the proposition we consider the case $d = 1$):

Proposition 3.20. Let $n \geq 2$ be a natural number, and let $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$

$$f(x) = \left(\sum_{j=1}^n x_j, \dots, \sum_{j=1}^n x_j \right) \in \mathbb{R}^n$$

then f is a continuous permutation equivariant function, but for every $h \in \mathcal{H}_{\text{pointnet-equiv}}$,

$$\max_{x \in [0,1]^n} \|f(x) - h(x)\|_{\infty} \geq 1/2 \quad (3.6)$$

Proof. First note that f is clearly continuous. To show it is permutation equivariant we show that (3.5) holds, because

$$f_j(x_{\tau(1)}, \dots, x_{\tau(n)}) = \sum_{i=1}^n x_{\tau(i)} = \sum_{i=1}^n x_i = f_{\tau(j)}(x_1, \dots, x_n), \quad \forall j \in \{1, \dots, n\}, \tau \in S_n, x \in \mathbb{R}^n.$$

Denote the all zero vector in $\mathbb{R}^n$ by 0_n . Note that for every $h \in \mathcal{H}_{\text{pointnet-equiv}}$,

$$f(e_1) = (1, 1, \dots, 1) \text{ and } h(e_1) = (\hat{h}(1), \hat{h}(0), \dots, \hat{h}(0))$$

while

$$f(0_n) = (0, 0, \dots, 0) \text{ and } h(0_n) = (\hat{h}(0), \hat{h}(0), \dots, \hat{h}(0)).$$

Since the second coordinate of h when applied to both e_1 and 0_n gives the same result, namely $h_2(0_n) = h_2(e_1)$, we obtain that h_2 cannot be a good approximation of $f_2(e_1)$ and $f_2(0_n)$ simultaneously, namely

$$|f_2(e_1) - h_2(e_1)| + |f_2(0_n) - h_2(0_n)| = |1 - \hat{h}(0)| + |\hat{h}(0) - 0| \geq 1$$

and thus

$$\max_{x \in [0,1]^n} \|f(x) - h(x)\|_\infty \geq \max_{x \in \{e_1, 0_n\}} |f_2(x) - h_2(x)| \geq 1/2 (|f_2(e_1) - h_2(e_1)| + |f_2(0_n) - h_2(0_n)|) \geq 1/2$$

□

How can we obtain a universal permutation equivariant model? The answer is provided by constructing slightly more powerful equivariant functions of the form

$$h(X) = \begin{pmatrix} N_2 \left(x_1, \sum_{j \neq 1} N_1(x_j) \right) \\ N_2 \left(x_2, \sum_{j \neq 2} N_1(x_j) \right) \\ \vdots \\ N_2 \left(x_n, \sum_{j \neq n} N_1(x_j) \right) \end{pmatrix} \quad (3.7)$$

where N_2, N_1 are neural networks. Any function of this form will be equivariant, because

$$h_j(x_{\tau(1)}, \dots, x_{\tau(n)}) = N_2 \left(x_{\tau(j)}, \sum_{i \neq j} N_1(x_{\tau(i)}) \right) = N_2 \left(x_{\tau(j)}, \sum_{i \neq \tau(j)} N_1(x_i) \right) = h_{\tau(j)}(x_1, \dots, x_n).$$

We will show that all equivariant functions can be approximated by functions of this form. The path to doing this goes through the observation that equivariant functions are generated by function invariant to the group

$$\text{stab}_n(1) = \{\tau \in S_n, \tau(1) = 1\}$$

in the following sense:

Lemma 3.21. *Let $f : \mathbb{R}^{d \times n} \rightarrow \mathbb{R}^n$ be a permutation equivariant function, then*

1. *The first coordinate of f , the function f_1 , is $\text{stab}_n(1)$ invariant.*
2. *For every $\ell \leq n$, let τ_ℓ denote the transposition $(1, \ell)$, namely*

$$\tau_\ell(1) = \ell, \tau_\ell(\ell) = 1, \tau_\ell(i) = i, \quad \forall i \in \{1, \dots, n\} \setminus \{1, \ell\},$$

then

$$f_\ell(X) = f_1(\tau_\ell(X)), \quad \forall X \in \mathbb{R}^{d \times n}$$

Proof. Recall for (3.5) that equivariance of f implies that

$$f_j(x_{\tau(1)}, \dots, x_{\tau(n)}) = f_{\tau(j)}(x_1, \dots, x_n), \quad \forall j \in \{1, \dots, n\}, \tau \in S_n, X \in \mathbb{R}^{d \times n}.$$

Taking $j = 1$ and any $\tau \in \text{stab}_n(1)$ we obtain that

$$f_1(x_{\tau(1)}, \dots, x_{\tau(n)}) = f_1(x_1, \dots, x_n), \quad \forall X \in \mathbb{R}^{d \times n}.$$

and so f_1 is invariant to the action of $\text{stab}_n(1)$.

Choosing $j = 1$ and $\tau = \tau_\ell$ leads to the second claim. □

Using this lemma, we can now employ our previous frameworks used to approximate permutation invariant functions via separating functions:

Theorem 3.22. *Let $f : [0, 1]^{d \times n} \rightarrow \mathbb{R}^n$ be a continuous, S_n equivariant function. Then, for all $\epsilon > 0$ there exists $m \in \mathbb{N}$ and neural networks $N_1 : \mathbb{R}^d \rightarrow \mathbb{R}^m$ and $N_2 : \mathbb{R}^{d+m} \rightarrow \mathbb{R}$ such that for the function h induced from N_1, N_2 as in (3.7), we have that*

$$\|f(X) - h(X)\|_\infty \leq \epsilon, \forall X \in [0, 1]^{d \times n}$$

Proof. Let f be as in the theorem, and choose some $\epsilon > 0$. By Lemma 3.21 we know that f_1 is $\text{stab}_n(1)$ invariant. We claim that the function space

$$\mathcal{S} = \{(x_1, \dots, x_n) \mapsto \ell(x_1, \sum_{j>1} h(x_j)) \mid h : \mathbb{R}^d \rightarrow \mathbb{R} \text{ is a neural network, } \ell : \mathbb{R}^{d+1} \rightarrow \mathbb{R} \text{ is linear}\}$$

is a separating family of continuous invariant functions. Continuity and invariance are straightforward. As for separation, for $X, Y \in \mathbb{R}^{d \times n}$ which do not have the same orbit, we must have either that $x_1 \neq y_1$, or that $(x_2, \dots, x_n)$ and $(y_2, \dots, y_n)$ are not related by permutation. In the first case, we can choose and neural network h , and take ℓ to be the projection onto the coordinate on which x_1 and y_1 differ. In the second case, according to Lemma 3.17 we can choose h such that $\sum_{j>1} h(x_j) \neq \sum_{j>1} h(y_j)$ and we can then choose ℓ to be the projection onto the last coordinate $\ell(z_1, \dots, z_{d+1}) = z_{d+1}$.

It follows from Theorem 3.15 that we can choose m , neural networks $h_1, \dots, h_m$, linear functions $\ell_1, \dots, \ell_m$, and a neural network h' such that

$$|f_1(X) - h' \left(\ell_1 \left(x_1, \sum_{j=2}^n h_1(x_j) \right), \dots, \ell_m \left(x_1, \sum_{j=2}^n h_m(x_j) \right) \right)| < \epsilon, \quad \forall X \in [0, 1]^{d \times n}$$

Introducing the linear mapping L

$$L(x_1, z_1, z_2, \dots, z_m) = (\ell_1(x_1, z_1), \dots, \ell_m(x_1, z_m))$$

and the neural networks

$$N_1(x) = (h_1(x), \dots, h_m(x)), \quad N_2 = h' \circ L,$$

we obtain that

$$|f_1(X) - N_2 \left(x_1, \sum_{j>1} N_1(x_j) \right)| < \epsilon, \quad \forall X \in [0, 1]^{d \times n}.$$

Now use these N_1, N_2 to define a function $h : \mathbb{R}^{d \times n} \rightarrow \mathbb{R}^n$ as in (3.7). Then we know that h_1 is an ϵ approximation of f_1 by the previous inequality. Using the second part of Lemma 3.21 we can extend this to all $\ell = 1, \dots, n$ because

$$|f_\ell(X) - h_\ell(X)| = |f_1(\tau_\ell X) - h_1(\tau_\ell X)| \leq \epsilon, \quad \forall X \in [0, 1]^{d \times n}.$$

□

3.1.6 Multisets, sets, and permutation invariant functions

Before we move on to our next topic of graph neural networks, it will be convenient to introduce the notion of multisets and explain its connection to permutation invariant neural networks.

A multiset is a collection of objects where order does not matter (like with sets) but where repetitions are allowed. It is denoted by double bracket. That is, for multisets

$$\{\{1, 1, 2\}\} = \{\{2, 1, 1\}\} \neq \{\{1, 2\}\}$$

while for sets

$$\{2, 1, 1\} = \{1, 2\}.$$

Let $\mathcal{P}_n(\mathbb{R}^d)$ denote the collection of all multisets with n elements in $\mathbb{R}^d$. There is a 'natural' bijection between the quotient space $\mathbb{R}^{d \times n}/S_n$ and this space. This is defined as follows: define a mapping $\pi : \mathbb{R}^{d \times n} \rightarrow \mathcal{P}_n(\mathbb{R}^d)$ by

$$\pi(x_1, \dots, x_n) = \{\!\{x_1, \dots, x_n\}\!\}.$$

This mapping is permutation invariant, and so induces a mapping $\tilde{\pi} : \mathbb{R}^{d \times n}/S_n \rightarrow \mathcal{P}_n(\mathbb{R}^d)$. It can be shown that this mapping is a bijection. Moreover, any invariant function f on $\mathbb{R}^{d \times n}$ can be identified with a function $\tilde{f}$ on $\mathcal{P}_n(\mathbb{R}^d)$ satisfying

$$\tilde{f}(\{\!\{x_1, \dots, x_n\}\!\}) = f(x_1, \dots, x_n), \quad \forall X \in \mathbb{R}^{d \times n}.$$

Multiset notation In general, for a subset $\mathcal{X} \subseteq \mathbb{R}^d$ we will use $\mathcal{P}_n(\mathcal{X})$ to denote the collection of all multisets with n elements in $\mathcal{X}$. We will also be interested in functions on collections of multisets for which the cardinality is not fixed, such as

$$\mathcal{P}_{\leq N}(\mathcal{X}) = \bigcup_{n \leq N} \mathcal{P}_n(\mathcal{X}), \quad \mathcal{P}(\mathcal{X}) = \bigcup_{n \in \mathbb{N}} \mathcal{P}_n(\mathcal{X}).$$

As discussed previously, the permutation invariant function spaces we described up to now are all applicable to multisets of varying size. For example, every neural network $h : \mathbb{R}^d \rightarrow \mathbb{R}^m$ defined a function $\tilde{h} : \mathcal{P}(\mathbb{R}^d) \rightarrow \mathbb{R}^m$ via

$$\tilde{h}(\{\!\{x_1, \dots, x_n\}\!\}) = \sum_{i=1}^n h(x_i).$$

Chapter 4

Graph Neural Networks and Graph Isomorphism tests

Our presentation here mostly follows [Xu et al., 2018]. We consider learning tasks over graphs. A graph is defined as usual through a finite set of n nodes V which we denote by $\{1, \dots, n\}$, and a set of unordered pairs of nodes $E \subseteq V \times V$ which we call edges. These can be encoded by the adjacency matrix A , where $A_{u,v} = 1$ if $(u, v) \in E$ and otherwise $A_{uv} = 0$. Additionally, we have a specified vector valued label per node $x_v \in \mathbb{R}^d$, which can be represented using a $d \times n$ matrix X . Thus our graph can be represented by two matrices $G = (A, X)$.

We would like to learn 'global' functions of the graph, of the form

$$G \mapsto F(G) \in \mathbb{R}^k \quad (4.1)$$

or node-level functions, of the form

$$G \mapsto (f_v(G))_{v \in V}, \quad f_v(G) \in \mathbb{R}^k. \quad (4.2)$$

For example, we can think of a graph representing a molecule, where the nodes V represent atoms, and the edges represent atoms which are linked by a chemical bond. The node features x_v in this case could represent the type of atom (hydrogen, oxygen, etc.). The function we would like to learn could be certain properties of the molecules e.g., is the molecule toxic? Is it effective for treatment of a given disease? Is it soluble in water? The answer to these three question could be encoded as a binary vector in $\mathbb{R}^3$, and a function answering these questions can be encoded by a function F as in (4.1) whose input is a graph and output is an $\mathbb{R}^3$ vector. For applications of graph neural networks for chemistry see e.g. [Reiser et al., 2022].

Another example could be a graph representing a social network such as LinkedIn, where nodes represent users, and edges between nodes represent users who are friends. The node features x_v could represent known data on users (e.g., age, number of degrees) and a node valued function $f_v(G)$ such as the one described in (4.2) could represent the likeliness that the user corresponding to node v will be a good job candidate.

4.0.1 Symmetries of graphs

We would like to design hypothesis classes for approximating F and f in (4.1) and (4.2) respectively, which respect the permutation symmetry of graphs. Note that relabeling the nodes of a graph $G = (A, X)$ using a permutation τ corresponds to the following action

$$(\tau.(A, X)) = (\tau.A, \tau.X)$$

where

$$(\tau.(A))_{ij} = A_{\tau^{-1}(i), \tau^{-1}(j)} \text{ and } (\tau.X)_{ij} = X_{i\tau^{-1}(j)}$$

A function F defined in (4.1) will typically be permutation invariant

$$F(\tau.(A, X)) = F(A, X)$$

and the function f defined in (4.2) will typically be permutation equivariant

$$f(\tau.(A, X)) = \tau.(f(A, X)).$$

Additional definitions The space of adjacency matrices with n nodes is denoted by

$$\mathcal{A}_n = \{A \in \{0, 1\}^{n \times n} \mid A_{ii} = 0, \forall i \in [n], A = A^T\}.$$

The feature space, the space where X resides, is $\mathbb{R}^{d \times n}$, but we will also sometimes assume the features come from some subset $\mathcal{X} \subseteq \mathbb{R}^d$. We denote

$$\mathcal{G}_n(\mathcal{X}) = \{(A, X) \mid A \in \mathcal{A}_n, x_j \in \mathcal{X}, \forall j = 1, \dots, n\}$$

Graph neural networks are typically defined for pairs $(A, X) \in \mathcal{A}_n \oplus \mathbb{R}^{d \times n}$, where n is allowed to vary, but d is fixed. In other words, the domain of a graph neural network is typically

$$\mathcal{G}(\mathbb{R}^d) = \bigcup_{n \in \mathbb{N}} \mathcal{G}_n(\mathbb{R}^d)$$

The neighborhood of a node $v \in [n]$ is the set $\mathcal{N}_v = \{u \in [n] \mid A_{u,v} = 1\}$.

Message Passing Neural Networks Message passing neural networks are a family of (arguably, the most) popular equivariant graph neural networks. They iteratively use the graph structure to redefine node labels. A message passing neural network is initialized via

$$x_v^{(0)} = x_v$$

where the features x_v all come from some input set $\mathcal{X} \subseteq \mathbb{R}^d$. We denote $\mathcal{X}^{(0)} = \mathcal{X}$. We then recursively define $x_v^{(t)}$ from $x_v^{(t-1)}$ via

$$\begin{aligned} y_v^{(t)} &= \text{AGGREGATE}^{(t)} \left(\{x_u^{(t-1)}, u \in \mathcal{N}_v\} \right) \\ x_v^{(t)} &= \text{COMBINE}^{(t)} \left(x_v^{(t-1)}, y_v^{(t)} \right). \end{aligned} \quad (4.3)$$

Here $\text{AGGREGATE}^{(t)}$ and $\text{COMBINE}^{(t)}$ are functions which are applied in the same fashion for all v . The input to the function $\text{AGGREGATE}^{(t)}$ is a multiset, which means it needs to be permutation invariant as discussed above. As we will, see the requirement of permutation invariance will be what makes this whole construction permutation equivariant, or invariant.

We apply this procedure some finite number of times T . The output of this procedure gives us a node valued function

$$f(A, X) = X^{(T)}. \quad (4.4)$$

If our final goal is to achieve a permutation invariant function we add an additional permutation invariant readout function

$$F(A, X) = \text{READOUT} \left(\{x_v^{(T)} \mid v \in V\} \right) \quad (4.5)$$

and the obtained function F is invariant. We note that the aggregate, combine and readout functions must be defined so that the composition makes sense, namely for the input domain $\mathcal{X} = \mathcal{X}^{(0)}$, there must be sets $\mathcal{X}^{(t)}, \mathcal{Y}^{(t)}$ such that

$$\begin{aligned} \text{AGGREGATE}^{(t)} : \mathcal{P}(\mathcal{X}^{(t-1)}) &\rightarrow \mathcal{Y}^{(t)}, \quad t = 1, \dots, T \\ \text{COMBINE}^{(t)} : \mathcal{X}^{(t-1)} \times \mathcal{Y}^{(t)} &\rightarrow \mathcal{X}^{(t)}, \quad t = 1, \dots, T \\ \text{READOUT} : \mathcal{P}(\mathcal{X}^{(T)}) &\rightarrow \text{OUT} \end{aligned}$$

Before we prove invariance and equivariance, let us see some example of popular message passing graph neural networks:

Example: GraphSAGE For example, the pooling variant of GraphSAGE [Hamilton et al., 2017] uses the aggregation function

$$y_v^{(t)} = \text{rowmax} \{ \text{ReLU} (W^{(t)} x_u^{(t-1)}) , u \in \mathcal{N}_v \}$$

where $W^{(t)}$ is some given matrix (which is learned in practice). This is a well defined multiset function, as the same function

$$y_u = \text{ReLU}(W^{(t)} x_u^{(t-1)})$$

is applied to all neighbors of v , and the maximum, which is applied row-wise to the matrix whose columns are $y_u, u \in \mathcal{N}(v)$ is a permutation invariant operation.

The COMBINE function in GraphSAGE is taken to be

$$x_v^{(t)} = \text{ReLU} \left(B^{(t)} \cdot \text{CONCAT} \left(x_v^{(t-1)}, y_v^{(t)} \right) \right),$$

after repeating this T times we get the final node level function

$$f_v(A, X) = x_v^{(T)}.$$

Note that $B^{(t)}, W^{(t)}$ are 'learnable matrices'. That is, we have here a family of graph neural networks, parameterized by these matrices, and we can optimize over the matrices to adapt the network to the graph function we want to learn (using gradient descent).

Example: GIN [Xu et al., 2018] Here we present the GIN architecture. This is defined by

$$\begin{aligned} y_v^{(t)} &= \text{AGGREGATE}_{GIN}^{(t)} \left(\{ x_u^{(t-1)}, u \in \mathcal{N}_v \} \right) = \sum_{u \in \mathcal{N}_v} x_u^{(t-1)} \\ x_v^{(t)} &= \text{COMBINE}_{GIN}^{(t)} \left(x_v^{(t-1)}, y_v^{(t)} \right) = h^{(t)} \left(\eta^{(t)} x_v^{(t-1)} + y_v^{(t)} \right) \end{aligned} \quad (4.6)$$

where $h^{(t)}$ is a fully connected neural network and $\eta^{(t)}$ is a number. The GIN architectures is actually a family of functions, depending on the parameters $\eta^{(t)}$ and the parameters of the fully connected neural network $h^{(t)}$. The READOUT function in GIN is also simply taken to be summation.

New Lesson

Theorem 4.1. *Let f, F be functions defined via message passing neural networks as in (4.5) and (4.4), respectively, then f is permutation equivariant and F is permutation invariant.*

Proof. For given $A \in \mathcal{A}_n$ and $X \in \mathbb{R}^{d \times n}$, let us denote the features obtained by the message passing procedure with the same notation as above:

$$x_v^{(t)}, y_v^{(t)}$$

Now, for some τ in S_n , let us denote

$$(\tilde{A}, \tilde{X}) = \tau.(A, X).$$

Let us denote the features obtained by applying the MPNN to this new graph by

$$\tilde{x}_v^{(t)}, \tilde{y}_v^{(t)}$$

By definition, we have that

$$A_{u,v} = \tilde{A}_{\tau(u), \tau(v)}, x_v = \tilde{X}_{\tau(v)}.$$

We will show by induction that

$$x_v^{(t)} = \tilde{x}_{\tau(v)}^{(t)}, \quad y_v^{(t)} = \tilde{y}_{\tau(v)}^{(t)}.$$

First, for the initialization we have $x_v^{(t)} = \tilde{x}_{\tau(v)}^{(t)}$ by assumption. Now, for $t \geq 1$, assuming correctness for t , we have

$$\begin{aligned} y_v^{(t)} &= \text{AGGREGATE}^{(t)} \left(\{ \{ x_u^{(t-1)}, u \in \mathcal{N}_v \} \} \right) \\ &= \text{AGGREGATE}^{(t)} \left(\{ \{ x_u^{(t-1)}, A_{u,v} = 1 \} \} \right) \\ &= \text{AGGREGATE}^{(t)} \left(\{ \{ \tilde{x}_{\tau(u)}^{(t-1)}, \tilde{A}_{\tau(u), \tau(v)} = 1 \} \} \right) \\ &= \text{AGGREGATE}^{(t)} \left(\{ \{ \tilde{x}_u^{(t-1)}, \tilde{A}_{u, \tau(v)} = 1 \} \} \right) \\ &= \tilde{y}_{\tau(v)}^{(t)} \end{aligned}$$

And thus

$$x_v^{(t)} = \text{COMBINE}^{(t)} \left(x_v^{(t-1)}, y_v^{(t)} \right) = \text{COMBINE}^{(t)} \left(\tilde{x}_{\tau(v)}^{(t-1)}, \tilde{y}_{\tau(v)}^{(t)} \right) = \tilde{x}_{\tau(v)}^{(t)}.$$

By induction we have proven that f is permutation equivariant. The invariance of F is also clear

$$F(A, X) = \text{READOUT} \left(\{ \{ x_v^{(T)} \mid v \in V \} \} \right) = \text{READOUT} \left(\{ \{ \tilde{x}_{\tau(v)}^{(T)} \mid v \in V \} \} \right) = F(\tilde{A}, \tilde{X}).$$

□

Approximation and Separation Power It would be natural to inquire what the expressive power of message passing neural networks (MPNN) is, and to hope that like the deepset architecture for point clouds, it will be universal in the sense that it can approximate any continuous permutation invariant function on graphs. As we saw, this question is very strongly linked to the question of the ability to separate all orbits. In other words, can MPNN separate any pair of graphs, up to permutation equivalence? As we will see, the answer to this question is negative, and as a result not all continuous invariant functions can be approximated. The fact that there are two graphs $G_1 = (A_1, X_1), G_2 = (A_2, X_2) \in \mathcal{A}_n \oplus \mathbb{R}^{d \times n}$ which are not related by a permutation, but cannot be separated by message passing neural networks, automatically implies that MPNNs cannot approximate uniformly continuous functions which can separate these graphs.

New Lesson June 30th

Maximally expressive MPNNs The separation capabilities of message passing neural networks obviously depends on the number of iterations T which are used, as well as the specific choice of the functions $AGGREGATE^{(t)}, COMBINE^{(t)}, t = 1, \dots, T$ and $READOUT$, that are used.

We will now describe how to construct maximally expressive MPNNs, and the closely related concept of WL-tests. We will assume that all initial features $x_v^{(0)}$ come from a (finite or infinite) countable set $\mathcal{X}^{(0)} \subseteq \mathbb{R}^d$. This assumption can be motivated by the chemical examples we discussed, where there typically is a finite number of node features, representing atom types. For a given natural number T , denoting the number of message passing iterations, we assume we are given sets $\mathcal{X}^{(t)}, \mathcal{Y}^{(t)}, t = 1, \dots, T$ and a set OUT . Let $\mathcal{P}(\mathcal{X})$ denote the collection of all finite multisets with elements in $\mathcal{X}$. We now assume that the functions

$$\begin{aligned} AGGREGATE^{(t)} : \mathcal{P}(\mathcal{X}^{(t-1)}) &\rightarrow \mathcal{Y}^{(t)}, \quad t = 1, \dots, T \\ COMBINE^{(t)} : \mathcal{X}^{(t-1)} \times \mathcal{Y}^{(t)} &\rightarrow \mathcal{X}^{(t)}, \quad t = 1, \dots, T \\ READOUT : \mathcal{P}(\mathcal{X}^{(T)}) &\rightarrow OUT \end{aligned}$$

are all *injective*. Let us call such MPNNs, maximally expressive T -iteration MPNNs. This name is justified by the following theorem:

Theorem 4.2. *Let $T \in \mathbb{N}$, let $\mathcal{X} \subseteq \mathbb{R}^d$ and let $\mathcal{G}(\mathcal{X})$ denote the set of all graphs with node features in $\mathcal{X}$. Let F and F' be two permutation invariant functions on $\mathcal{G}(\mathcal{X})$ obtained by applying T iterations of message passing, and assume that F is maximally expressive. Then*

$$\forall G_1, G_2 \in \mathcal{G}(\mathcal{X}) \quad F(G_1) = F(G_2) \implies F'(G_1) = F'(G_2)$$

You will prove this theorem in homework in the special case $T = 1$.

WL tests One way to achieve maximally expressive MPNN is by choosing all the intermediate sets $\mathcal{X}^{(t)}, \mathcal{Y}^{(t)}$ and OUT to be $\mathbb{N}$. Since the sets $\mathcal{P}(\mathcal{X}^{(t)})$ and $\mathcal{X}^{(t-1)} \times \mathcal{Y}^{(t-1)}$ are countable in this case, we know that there exists a choice of functions $AGGREGATE^{(t)}, COMBINE^{(t)}$ and $READOUT$ which is injective. We will call such choice of functions 'hash functions', and we will call the resulting MPNN a 1-Weisfeiler–Leman (1-WL) test.

This presentation of 1-WL as a special case of MPNNs makes sense mathematically, but historically the order is reversed. WL tests were originally introduced as an isomorphism test, to check whether a given pair of labeled graphs, G and G' are isomorphic (related by a permutation). As the name suggests, this idea was presented by Weisfeiler and Leman as early as 1968, long before the idea of learning on graphs.

Also note that WL tests cannot really serve as reasonable architectures for graphs. In MPNNs used for graph learning, like the GIN and graphSage examples described above, the aggregation, combine and readout functions are parameteric functions f_θ . Each parameter vector θ defines an MPNN, and the choice of the parameter vector is determined using gradient descent on the parameters θ . Such a differentiable construction is not possible when all features in the MPNN procedure are assumed to be in $\mathbb{N}$. We will later discuss how to build maximally expressive MPNN which are differentiable. For now, it is sufficient to know that if there exists any MPNN with T iterations which can separate a pair of graphs, then the WL test with T iterations can achieve the same separation. Our next step will be to understand the capabilities, and limitations, of WL tests in terms of separating graphs, through examples.

WL separation: examples For a given pair of graphs G, G' , if the graphs are isomorphic, that is $G = \tau(G')$ for some permutation τ , then the permutation invariance of MPNN implies that an MPNN F will assign the same global feature to both graphs,

$$F(G) = F(\tau(G')) = F(G').$$

In the other direction, there will be some pairs of graphs (G, G') for which $F(G) \neq F(G')$. In this case, we will say that the MPNN (or WL test if F is a WL test) separates the graphs. But there will be other cases where there will be no separation and $F(G) = F(G')$.

Problem 4.3. Figure 4.1 shows four examples of pairs of graphs which are not isomorphic. The colors of the nodes depicts the node features (nodes have the same feature if and only if they are assigned the same color in the figure). For each of the four examples: Can the graphs be separated by the WL test? If so, how many iterations T are needed?

1. Let us first consider the two graphs in Figure 4.1(a). They are not-isomorphic, and we will see that the WL iterations can separate them. Let us denote the line graph in Figure 4.1(a) by G and the second graph by G' . At initialization, the node features of all graphs are initialized to the same value, say $x_v^{(0)} = 0$. In the first WL iteration, we define

$$\begin{aligned} y_v^{(1)} &= \text{AGGREGATE}^{(1)}(\{x_u^{(0)}, u \in \mathcal{N}_v\}) \\ x_v^{(1)} &= \text{COMBINE}^{(1)}(x_v^{(0)}, y_v^{(1)}). \end{aligned}$$

For all nodes v in the graph G' , and three of the nodes in the graph G , we will get

$$y_v^{(1)} = \text{AGGREGATE}^{(1)}(\{0, 0\})$$

but for two nodes in G , the nodes u at the end of the line, we will get

$$y_u^{(1)} = \text{AGGREGATE}^{(1)}(\{0\})$$

By assumption, since the aggregation function in the WL test is injective, $y_v^{(1)}$ and $y_u^{(1)}$ will be assigned to different values in $\mathbb{N}$, e.g. $y_v^{(1)} = 1$ and $y_u^{(1)} = 2$.

By the injectivity of the $\text{COMBINE}^{(t)}$ function we will get

$$x_v^{(1)} = \text{COMBINE}^{(1)}(x_v^{(0)}, y_v^{(1)}) = \text{COMBINE}^{(1)}(0, 2) \neq \text{COMBINE}^{(1)}(0, 1) = x_u^{(1)}$$

Without loss of generality we can take $x_v^{(1)} = 2, x_u^{(1)} = 1$. If we end the WL iterations with $T = 1$ we will get

$$F(G) = \text{READOUT}\{1, 2, 2, 2, 1\} \neq \text{READOUT}\{2, 2, 2, 2, 2\} = F(G').$$

We see that these two graphs can be separated by $T = 1$ iterations.

Note that in essence, these graphs can be separated because their nodes do not have the same degrees. One can deduce from this example a more general principle: given two graphs which are initially unlabelled, the first WL iterations gives each node a new label: its degree. If the multiset of degree is not identical for both graphs, then they are not isomorphic, and WL can separate them after a single iteration.

2. Now let us consider the pair of graphs in Figure 4.1(b). They too are non-isomorphic as one is connected while the other is not. This cannot be proved using degrees, as the degree multiset of both graphs is $\{1, 1, 2, 2, 2\}$. Thus, after the first iteration we cannot separate the two graphs, and we can assume without loss of generality that $x_v^{(1)}$ is the degree of the node v . However, in the second step we will get separation. For graph on the left, which we denote by G , we get (ordering the nodes from top to bottom)

$$\begin{aligned} y_1^{(2)} &= y_5^{(2)} = \text{AGGREGATE}^{(2)}(\{2\}) \\ y_2^{(2)} &= y_4^{(2)} = \text{AGGREGATE}^{(2)}(\{1, 2\}) \\ y_3^{(2)} &= \text{AGGREGATE}^{(2)}(\{2, 2\}) \end{aligned}$$

while for the graph on the right, which we denote by $\hat{G}$, we get (ordering the nodes from top to bottom)

$$\begin{aligned} \hat{y}_1^{(2)} &= \hat{y}_2^{(2)} = \text{AGGREGATE}^{(2)}(\{1\}) \\ \hat{y}_3^{(2)} &= \hat{y}_4^{(2)} = \hat{y}_5^{(2)} = \text{AGGREGATE}^{(2)}(\{2, 2\}) \end{aligned}$$

Without loss of generality let us map each multiset appearing above to a different integer using a hash function

$$\begin{aligned}\{\{2\}\} &\mapsto 3 \\ \{\{1, 2\}\} &\mapsto 4 \\ \{\{2, 2\}\} &\mapsto 5 \\ \{\{1\}\} &\mapsto 6.\end{aligned}$$

We then obtain

$$\begin{aligned}x_1^{(2)} &= x_5^{(2)} = \text{COMBINE}^{(2)}(1, 3) = a \\ x_2^{(2)} &= x_4^{(2)} = \text{COMBINE}^{(2)}(2, 4) = b \\ x_3^{(2)} &= \text{COMBINE}^{(2)}(2, 5) = c \\ \hat{x}_1^{(2)} &= \hat{x}_2^{(2)} = \text{COMBINE}^{(2)}(1, 6) = d \\ \hat{x}_3^{(2)} &= \hat{x}_4^{(2)} = \hat{x}_5^{(2)} = \text{COMBINE}^{(2)}(2, 5) = c\end{aligned}$$

where by the injectivity of $\text{COMBINE}^{(2)}$ all of the four numbers a, b, c, d are distinct. If we end the WL iterations after two iterations we obtain, due to the injectivity of READOUT ,

$$F(G) = \text{READOUT}\{\{a, a, b, b, c\}\} \neq \text{READOUT}\{\{c, c, c, d, d\}\} \neq F(\hat{G})$$

3. The pair of graphs in Figure 4.1(c) is a famous example of graphs which cannot be separated by any number of WL iterations. Let us denote these graphs by G and G' . At initialization there exists some natural number n_0 such that for any node v in any one of the two graphs

$$x_v^{(0)} = n_0.$$

We can prove inductively that for every pair of nodes, and any natural t , there is some $n_t \in \mathbb{N}$ such that

$$x_v^{(t)} = n_t.$$

for every node v in either graph.

This is because by the induction assumption, we have this equality for $t - 1$. We then have for every pair of nodes v, u that

$$y_v^{(t)} = \text{AGGREGATE}^{(t)}(n_{t-1}, n_{t-1})$$

denote $m_t = \text{AGGREGATE}^{(t)}(n_{t-1}, n_{t-1})$. We then obtain for every node v in either graph

$$x_v^{(t)} = \text{COMBINE}^{(t)}(n_{t-1}, m_t) = n_t$$

where we denoted $n_t = \text{COMBINE}^{(t)}(n_{t-1}, m_t)$. We can recursively apply this procedure T times and finally obtain

$$F(G) = \text{READOUT}\{\{n_T, \dots, n_T\}\} = F(G').$$

We can deduce from this example a more general fact: Two graphs G, G' which are not isomorphic, but have the same number of nodes and are both r -regular (all nodes have r neighbors) cannot be separated by WL.

4. In Figure 4.1(d) we have the same graphs as in (c), but now the nodes do not all have the same label. You can verify that in this case a single WL iteration is sufficient for separation.

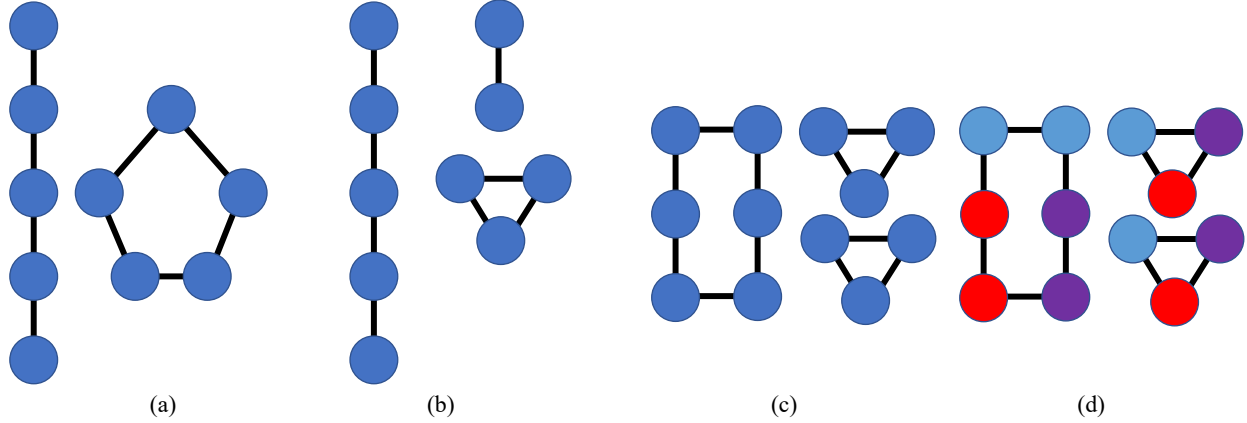


Figure 4.1: four examples of pairs of non-isomorphic graphs (a) graphs can be separated by vertex degree (first WL iteration) (b) graphs can be separated by the second WL iteration (c) graphs cannot be separated by WL (d) WL applied to labelled (colored) graph- in this example WL is able to separate the previously inseparable pair.

Lack of separation implies lack of universality We now show that the fact that MPNN cannot separate between some pairs of graphs (which are not related by permutation) implies that MPNN cannot approximate all continuous invariant functions. The basis for this is the observation that continuous invariant functions can separate graphs:

Problem 4.4. For given $n \in \mathbb{N}$ and $\mathcal{X} \subseteq \mathbb{R}^d$, show that for any $G_1, G_2 \in \mathcal{G}_n(\mathcal{X})$ there exists a continuous invariant function $F : \mathcal{G}_n(\mathcal{X}) \rightarrow \mathbb{R}$ such that $F(G_1) \neq F(G_2)$. Continuity here is in the standard Euclidean sense, where we are thinking of $\mathcal{G}_n(\mathcal{X})$ as a subset of $\mathbb{R}^{n \times n} \oplus \mathbb{R}^d$.

Solution: One possible solution is to define the polynomial

$$p(A, X) = \prod_{\tau \in S_n} (\|\tau(A) - A_1\|_F^2 + \|\tau(X) - X_1\|_F^2)$$

where we denote $G_1 = (A_1, X_1)$.

We have that p is continuous since it is a polynomial, and is permutation invariant. Moreover, $p(A, X) \geq 0$ and is equal to zero if and only if the graphs (A, X) and $G_1 = (A_1, X_1)$ are isomorphic, in particular

$$p(A_1, X_1) = 0 < p(A_2, X_2)$$

Lack of approximation For given graphs G_1, G_2 which cannot be separated by an MPNN, let p be the continuous function from before satisfying that $p(G_1) = 0$ and $p(G_2) > 0$. Then for any MPNN F we have that F cannot approximate p to any accuracy we want on a compact set of graphs, even if we only consider the compact set $\{G_1, G_2\}$, as we have

$$\max_{G \in \{G_1, G_2\}} |F(G) - p(G)| \geq \frac{1}{2} (|F(G_1) - p(G_1)| + |F(G_2) - p(G_2)|) \geq \frac{1}{2} |p(G_1) - F(G_1) + F(G_2) - p(G_2)| = \frac{p(G_2)}{2}$$

Conclusion and Graph Isomorphism We saw that MPNNs can fail to separate certain graphs. If so, why are they used? Why not use a model with perfect separation abilities as we saw in the point set examples?

The answer to this question can be understood via the graph isomorphism problem: the graph isomorphism problem is the problem of determining whether a pair of two graphs G, G' are isomorphic- that is- whether they are related by permutation. To date, there is no known polynomial time algorithm for this problem, although notably it is also not known to be NP hard. A recent quasi-polynomial algorithm to solve

graph isomorphism [Babai, 2016] is considered a significant breakthrough in the field. To get an intuition for the difficulty of graph isomorphism, consider the graphs in Figure 4.2. These are two isomorphic graphs, but it is not easy to see this in first sight.

There are many instances which make graph isomorphism tractable: for example, if the graph has bounded degree, if the eigenvalues of the adjacency matrix have bounded multiplicity. Similarly, the WL test, and MPNN architectures, are able to separate pairs of graphs which are not 'too hard'. There are alternative constructions (e.g. k -WL with $k > 1$ which we will perhaps discuss later on) which can separate more problematic pairs of graphs. However, we do not expect to come up with a perfectly separating architecture which can run in polynomial time, unlike the case with pointsets where this was achievable.

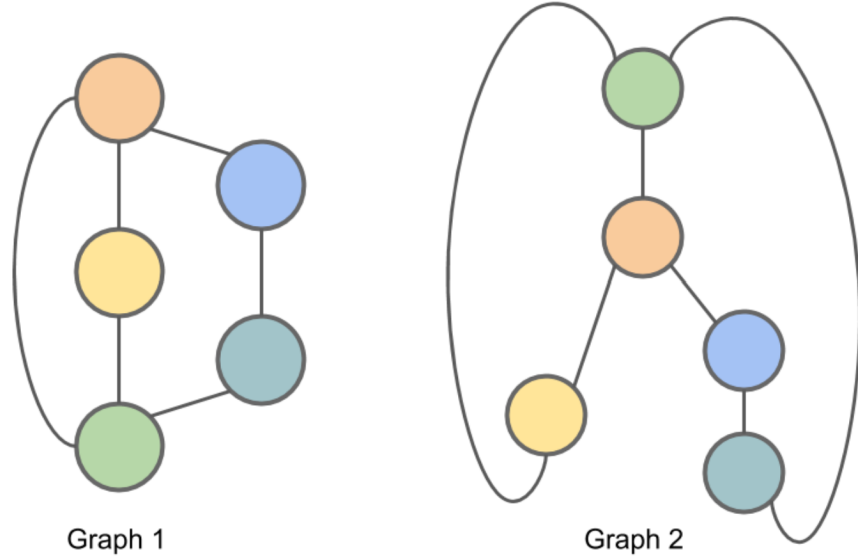


Figure 4.2: Example of two isomorphic graphs. Taken from [Bieber,]

New lesson: maximally expressive MPNN

We recall from the previous lesson, that we discussed message passing neural networks. The input to these networks are a graph, encoded by an adjacency matrix A , and n vectors in $\mathbb{R}^d$, one for each node. These are denoted by $x_v, v \in V$. A message passing neural network iteratively updates the node features by initializing

$$x_v^{(0)} = x_v$$

and then recursively defining $x_v^{(t)}$ from $x_v^{(t-1)}$ via
for $t = 1, \dots, T$:

$$\begin{aligned} y_v^{(t)} &= \text{AGGREGATE}^{(t)} \left(\{x_u^{(t-1)}, u \in \mathcal{N}_v\} \right) \\ x_v^{(t)} &= \text{COMBINE}^{(t)} \left(x_v^{(t-1)}, y_v^{(t)} \right). \end{aligned}$$

Finally, a graph level function is defined via

$$F(A, X) = \text{READOUT} \left(\{x_v^{(T)} | v \in V\} \right)$$

We saw in the previous lesson

1. Such a function F is always permutation invariant. Mamey, if (A, X) and (A', X') are related by a permutation, then

$$F(A, X) = F(A', X')$$

2. Not all graphs can be separated by MPNN. For example, if A, A' are two regular graphs, and the features x_v of both graphs are all the same in all nodes.
3. The best one can do, in terms of separation, is if the MPNN are maximally expressive. This means that the functions $\text{AGGREGATE}^{(t)}$, $\text{COMBINE}^{(t)}$ and READOUT are all injective.

In this lesson, we will discuss how to create maximally expressive MPNN. We will first review the construction from [Xu et al., 2018], and then suggest a more formal analysis suggested in (our paper) [Amir et al., 2023].

Formal Setting For a full formal discussion, we will consider the graph domain

$$\mathcal{G}_{\leq N}(\mathcal{X})$$

of graphs with at most N nodes, whose node features $x_1, \dots, x_n$ are all from a (finite or infinite) countable 'alphabet' $\mathcal{X} \subseteq \mathbb{R}^d$.

Consider an MPNN, with fixed aggregate, combined and readout functions, applied to the graph space $\mathcal{G}_N(\mathcal{X})$. We denote by $\mathcal{X}^{(t)}$ all features of the form $x_v^{(t)}$ obtained for any graph and any node v , and similarly denote $\mathcal{Y}^{(t)}$. These sets will be countable, and each will be a subset of $\mathbb{R}^k$ for some appropriate k . A maximally expressive MPNN, is one for which the functions:

$$\begin{aligned} \text{AGGREGATE}^{(t)} : \mathcal{P}_{\leq N}(\mathcal{X}^{(t-1)}) &\rightarrow \mathcal{Y}^{(t)}, \quad t = 1, \dots, T \\ \text{COMBINE}^{(t)} : \mathcal{X}^{(t-1)} \times \mathcal{Y}^{(t)} &\rightarrow \mathcal{X}^{(t)}, \quad t = 1, \dots, T \\ \text{READOUT} : \mathcal{P}_{\leq N}(\mathcal{X}^{(T)}) &\rightarrow \text{OUT} \end{aligned}$$

are all injective. Recall that $\mathcal{P}_{\leq N}(\mathcal{X})$ stands for the collection of all multisets with at most N elements in the set $\mathcal{X}$.

4.0.2 GIN*

The paper [Xu et al., 2018] suggest to build maximally expressive MPNN via the following process

For $t = 1, \dots, T$

$$y_v^{(t)} = \text{AGGREGATE}^{(t)} \left(\{x_u^{(t-1)}, u \in \mathcal{N}_v\} \right) = \sum_{u \in \mathcal{N}_v} f_{(t)} \left(x_u^{(t-1)} \right)$$

$$x_v^{(t)} = \text{COMBINE}^{(t)} \left(x_v^{(t-1)}, y_v^{(t)} \right) = \eta^{(t)} \cdot x_v^{(t-1)} + y_v^{(t)}.$$

and finally

$$F(A, X) = \text{READOUT} \left(\{x_v^{(T)} \mid v \in V\} \right) = \sum_{v \in V} f_{(T+1)}(x_v^{(T)})$$

The claim is that, for suitable choices of the scalar $\eta^{(t)}$, and the functions $f_{(1)}, \dots, f_{(T+1)}$, all the functions in the MPNN process will be injective. This is shown via the following two propositions:

Proposition 4.5. *Let $\mathcal{X} \subseteq \mathbb{R}^d$ and $\mathcal{Y} \subseteq \mathbb{R}^d$ be countable sets. Then there exists a countable set $N \subseteq \mathbb{R}$, such that for all $\eta \in \mathbb{R} \setminus N$, the mapping $f(\cdot, \cdot; \eta) : \mathcal{X} \times \mathcal{Y} \rightarrow \mathbb{R}^d$ defined by*

$$f(x, y; \eta) = \eta \cdot x + y$$

is injective.

Proof. Try on your own. □

Proposition 4.6. *For a fixed $N \in \mathbb{N}$ and a countable set $\mathcal{X}$, there exists a function $f : \mathcal{X} \rightarrow \mathbb{R}$ such that the function*

$$\{x_1, \dots, x_n\} \mapsto \sum_{j=1}^n f(x_j)$$

is injective on $\mathcal{P}_{\leq N}(\mathcal{X})$.

Proof idea. Let us consider for simplicity the case where $\mathcal{X} = \mathbb{N} \cup \{0\}$, and $N = 9$. We then choose the function

$$f(x) = (N+1)^x = 10^x$$

For a multiset in the domain, such as

$$S = \{1, 3, 3, 4\}$$

we get

$$\sum_{x \in S} f(x) = 10^1 + 2 \cdot 10^3 + 10^4 = 12,010.$$

The k -th digit of this number encodes the number of times the number k occurs in the multiset. Thus if

$$\sum_{x \in S} (N+1)^x = \sum_{y \in T} (N+1)^y$$

for some $S, T \in \mathcal{P}_N(\mathcal{X})$, then necessarily all natural numbers appear the same amount of time in S and in T , which means that they are equal.

You will give a full proof of this claim in homework. □

Conclusion We can build an MPNN with the functions $f_{(t)}$ and the parameters $\eta_{(t)}$ promised by the last two propositions. This will give us a maximally expressive MPNN. We will call this MPNN GIN*. What this means is that, if (A, X) and (A', X') are two graphs in $\mathcal{G}_N(\mathcal{X})$, and if the F defined by GIN* cannot separate the two graphs after T iterations, namely

$$F(A, X) = F(A', X')$$

then no MPNN with T iterations can separate the two graphs.

A disadvantage of the construction we just saw is that the functions $f_{(t)}$ chosen are not functions one would choose in practice. The MPNN suggested in the original paper [Xu et al., 2018] suggested to replace these functions with fully connected neural networks. This was justified by the observation that continuous functions can be approximated to arbitrary accuracy by fully connected neural networks, and thus $f_{(t)}$ can be approximated by some neural network $\tilde{f}_{(t)}$ to obtain maximal expressivity. This claim seems problematic for countable $\mathcal{X}$, as (i) It is not clear that the function in Proposition 4.6 is continuous for arbitrary countable sets and (ii) countable sets are not necessarily compact (e.g., $\mathbb{N}$ is not compact), and so applying universal approximation is not straightforward. However, if we assume that $\mathcal{X}$ is finite, then all functions are always continuous, and any finite set is compact, so these problems can be solved. Still, a disadvantage of this argument is the need to focus on finite alphabets and graphs of bounded cardinality, and moreover, that it is not clear how large the neural networks $\tilde{f}_{(t)}$ need to be to guarantee maximal expressivity. In the next lesson we will see an approach developed in our group that addresses all these issues.

New Lesson July 13

4.0.3 Maximally expressive MPNN using analytic activations

In the following, we will show an argument we developed in [Amir et al., 2023], which works for countable $\mathcal{X}$ and graphs of unbounded cardinality, and moreover shows that the $f_{(t)}$ can be chosen to be the smallest possible imaginable neural networks, namely a single layer with width 1, $x \mapsto \sigma(a^T x + b)$. This argument requires the assumption that σ is an analytic non-polynomial activation. So this argument would not work for ReLU activation, but would work for other common activations like sigmoid or tanh. What we will prove is

Theorem 4.7. *Let $\mathcal{X} \subseteq \mathbb{R}^d$ be a countable set, and let $T \in \mathbb{N}$. Let $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be an analytic activation which is not a polynomial. Then there exists vectors $a^{(1)}, \eta^{(1)} \in \mathbb{R}^d$ and real numbers $a^{(2)}, \dots, a^{(T+1)}, \eta^{(2)}, \dots, \eta^{(T)}$ and $b^{(1)}, \dots, b^{(T+1)}$ such that the MPNN defined for $t = 1, \dots, T$ by*

$$y_v^{(t)} = \text{AGGREGATE}^{(t)} \left(\{x_u^{(t-1)}, u \in \mathcal{N}_v\} \right) = \sum_{u \in \mathcal{N}_v} \sigma \left(a^{(t)} \cdot x_u^{(t-1)} + b^{(t)} \right)$$

$$x_v^{(t)} = \text{COMBINE}^{(t)} \left(x_v^{(t-1)}, y_v^{(t)} \right) = \eta^{(t)} \cdot x_v^{(t-1)} + y_v^{(t)}.$$

and finally

$$F(A, X) = \text{READOUT} \left(\{x_v^{(T)} \mid v \in V\} \right) = \sum_{v \in V} \sigma \left(a^{(T+1)} x_v^{(T)} + b^{(T+1)} \right)$$

is maximally expressive on $\mathcal{G}(\mathcal{X})$

To prove this theorem, we will first need some reminder and background on analytic functions.

Analytic functions We will quickly review some facts about analytic functions which are mostly taught in standard calculus courses. For more details see Chapter 1 in [Krantz and Parks, 2002].

We first recall the definition of an analytic function. Let $U \subseteq \mathbb{R}$ be an open set. We say that a function $\sigma : U \rightarrow \mathbb{R}$ is analytic at $x_0 \in U$, if there exists $r > 0$ and a sequence $a_0, a_1, \dots$ of real numbers, such that

$$f(x) = \sum_{n=0}^{\infty} a_n (x - x_0)^n, \quad \forall x \in (x_0 - r, x_0 + r).$$

We call r the radius of convergence. We say that f is analytic if it is analytic at every point $x_0 \in U$. It is known that if f has a radius of convergence r at x_0 , the f is analytic at every $x_1 \in (x_0 - r, x_0 + r)$.

Example 4.8. A polynomial $p : \mathbb{R} \rightarrow \mathbb{R}$ is analytic. At 0 we can write

$$p(x) = \sum_{n=0}^N a_n x^n, \quad \forall x \in \mathbb{R}$$

so p is analytic at 0. Similarly, at every point x_0 we can rewrite the polynomials as

$$p(x) = \sum_{n=0}^N b_n (x - x_0)^n, \quad \forall x \in \mathbb{R}$$

for appropriate b_n .

Example 4.9. The function e^x is analytic on all $\mathbb{R}$. We have that

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!}, \quad \forall x \in \mathbb{R}$$

So e^x is analytic at 0 with an infinite radius of convergence. This implies that e^x is analytic at every point x .

Example 4.10. The function $1/x$ is analytic on $\mathbb{R} \setminus \{0\}$. The idea to show this is that we know that for all $q \in (-1, 1)$,

$$\frac{1}{1-q} = \sum_{n=0}^{\infty} q^n$$

Thus, at any point $x_0 \neq 0$ we have that

$$\frac{1}{x} = \frac{1}{x_0 - (x_0 - x)} = \frac{1}{x_0} \frac{1}{1 - (x/x_0)} \stackrel{(*)}{=} \frac{1}{x_0} \sum_{n=0}^{\infty} (x/x_0)^n = \sum_{n=0}^{\infty} \frac{(-1)^n}{x_0^{n+1}} (x - x_0)^n,$$

where $(*)$ holds for all $x \in (x_0 - r, x_0 + r)$ for $r = |x_0|$.

Properties of analytic functions

1. Multiplication, addition, and composition of analytic functions are analytic functions. We can deduce that the SiLU, Tanh and sigmoid activations are analytic. For example, the sigmoid function is given by

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

By the composition rule, e^{-x} is analytic, as a composition of $x \mapsto -x$ and e^x . From the addition rule, $1 + e^{-x}$ is analytic. the composition of this function with $1/x$ is thus also analytic, and so $\sigma(x)$ is analytic.

2. An analytic function $f : U \rightarrow \mathbb{R}$ is ∞ times differentiable in U , and all its derivatives are analytic on U . Most smooth functions we know like sin, cos, log etc. are analytic at all points at which they are defined. However in general, smooth functions are not always analytic. For example, smooth non-zero functions with compact support, like the function defined in (1.4), are non analytic.
3. Assume U is open and connected, $f : U \rightarrow \mathbb{R}$ is analytic, and there exists a sequence t_n of points in U which converges to $t \in U$, such that $f(t_n) = 0$ for all n . Then $f(t) = 0$ for all $t \in U$.
4. From the previous property, it follows that if U is open and connected, and $f : U \rightarrow \mathbb{R}$ is analytic and non-zero, and $[a, b] \subseteq U$, then there will be only a finite number of points $t_1, \dots, t_K$ for which $f(t_k) = 0$. If there was an infinite number of such points, then they would have an accumulation point in $[a, b] \subseteq U$, and as a result f would be identically zero.
5. It follows that, if $f : \mathbb{R} \rightarrow \mathbb{R}$ is analytic and not identically zero, then it will have a countable (finite or infinite) number of roots, since it has a finite number of roots in each interval $[k, k+1]$, and so a countable number in $\mathbb{R} = \cup_{k \in \mathbb{Z}} [k, k+1]$.

Multivariate analytic functions We now define analytic function in several variables $f : \mathbb{R}^d \rightarrow \mathbb{R}$.

Definition 4.11. Let $U \subseteq \mathbb{R}^d$ be an open set. We say that $f : U \rightarrow \mathbb{R}$ is analytic, if for every $x_0 \in U$, there exists an neighborhood of x_0 , such that, for all x in that neighborhood, $\sum_{\alpha \in \mathbb{N}_0^d} |c_\alpha| |x - x_0|^\alpha < \infty$ and

$$f(x) = \sum_{\alpha \in \mathbb{N}_0^d} c_\alpha (x - x_0)^\alpha.$$

Naturally, polynomial functions from $\mathbb{R}^d$ to $\mathbb{R}$ are analytic. As one could expect, linear combinations, multiplications, and composition of analytic functions are analytic. In particular, if $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ is analytic, then the function

$$f_\sigma(x, a, b) = \sigma(a^T x + b)$$

is analytic, as the function $(a, x, b) \mapsto a^T x + b$ is a multivariate polynomial of degree 2, and the function $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ is analytic. Similarly, for every fixed x , the function $(a, b) \mapsto f_\sigma(x; a, b)$ will be analytic. This will be the main property we will use later on.

Sets of measure zero We saw that one-dimensional non-zero analytic functions can only have a countable number of roots. A multivariate non-zero analytic function can have an uncountable number of roots. For example, the roots of the linear function $f(x, y) = y$ are all the points on the x -axis. What we can say is that the set of roots will be of measure zero, a concept which we will now define.

A set has measure zero, if its Lebesgue measure is zero. Since Lebesgue integration is not necessarily a course prerequisite, we will give an independent definition of what it means to be a set of measure zero:

We define the volume of a box $B = \prod_{i=1}^d [a_i, b_i]$ (where $a_i < b_i$ for all i) to be $|B| = \prod_{i=1}^d (b_i - a_i)$. We say that a set $Z \subseteq \mathbb{R}^d$ has measure zero, if for every $\epsilon > 0$, there exists a countable collection of boxes $B_1, B_2, \dots$ in $\mathbb{R}^d$, such that

$$Z \subseteq \bigcup_{n \in \mathbb{N}} B_n \text{ and } \sum_{n \in \mathbb{N}} |B_n| < \epsilon.$$

Problem 4.12. 1. Show that a finite set in $\mathbb{R}$ has measure zero.

2. Show that a countable set in $\mathbb{R}$ has measure zero.

3. Show that a countable (finite or infinite) union of measure zero sets is a measure zero set.

4. Show that if $N \subseteq \mathbb{R}^m$ has measure zero. Then $N \times \mathbb{R}^n \subseteq \mathbb{R}^{n+m}$ has measure zero.

5. Show that the line $\{(x, y) | y = 0\}$ has measure zero in $\mathbb{R}^2$.

Thus, the analytic function $f(x, y) = y$ which we discussed before, has an uncountable number of roots, but the set of roots $\{(x, y) | y = 0\}$ has measure zero as shown above. This is the case for all non-zero analytic functions

Proposition 4.13 (For proof see [Mityagin, 2015]). *Let $U \subseteq \mathbb{R}^n$ be a open connected set, and let $f : U \rightarrow \mathbb{R}$ be an analytic function which is not identically zero. Then the set of the function's roots*

$$\{x \in U \mid f(x) = 0\}$$

has measure zero.

We now have all the tools we need to start proving Theorem 4.7. The main stage is proving the following proposition

Proposition 4.14. *Let $\mathcal{X} \subseteq \mathbb{R}^d$ be a countable set, and let $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be an analytic non-polynomial function, then there exist $a \in \mathbb{R}^d, b \in \mathbb{R}$, such that the function*

$$\{x_1, \dots, x_n\} \mapsto \sum_{j=1}^n \sigma(a^T x_j + b) \tag{4.7}$$

is injective on $\mathcal{P}(\mathcal{X})$.

We recall that $\mathcal{P}(\mathcal{X})$ is the set of all finite multisets with elements in $\mathcal{X}$. So

$$\mathcal{P}(\mathcal{X}) = \bigcup_{N \in \mathbb{N}} \mathcal{P}_N(\mathcal{X}).$$

Proof. For fixed $X, Y \in \mathcal{P}(\mathcal{X})$ where $X \neq Y$, we saw in homework that there exist $(a, b) \in \mathbb{R}^d \oplus \mathbb{R}$, such that

$$\sum_{x \in X} \sigma(a^T x + b) \neq \sum_{y \in Y} \sigma(a^T y + b). \tag{4.8}$$

Equivalently, this means that the analytic function

$$\psi(a, b) = \sum_{x \in X} \sigma(a^T x + b) - \sum_{y \in Y} \sigma(a^T y + b)$$

is not identically zero. This in turn implies that the set of roots of ψ has measure zero. So there is a measure zero set $N_{X,Y} \subseteq \mathbb{R}^{d+1}$ such that if $(a,b) \notin N_{X,Y}$, we have that (4.8) holds.

This is true for all $X \neq Y$ in $\mathcal{P}(\mathcal{X})$, and the collection of all such multisets is countable. Since a countable union of measure zero sets is measure zero, it follows that the set

$$N = \bigcup_{(X,Y) \in \mathcal{P}(\mathcal{X}) \times \mathcal{P}(\mathcal{X}), X \neq Y} N_{X,Y},$$

also has measure zero, and that for all $(a,b) \notin N$, we will have for all $X \neq Y$ that (4.8) holds, and thus that the function (4.7) is injective. $\square$

The second ingredient we will need is a slight variation of Proposition 4.5, to prove injectivity of the combine function. The difference from that proposition is that now η is a vector:

Proposition 4.15. *Let $\mathcal{X} \subseteq \mathbb{R}^d$ and $\mathcal{Y} \subseteq \mathbb{R}$ be countable sets. Then there exists a measure zero set $N \subseteq \mathbb{R}^d$, such that for all $\eta \in \mathbb{R}^d \setminus N$, the mapping $f(\cdot, \cdot; \eta) : \mathcal{X} \times \mathcal{Y} \rightarrow \mathbb{R}^d$ defined by*

$$f(x, y; \eta) = \eta \cdot x + y$$

is injective.

Proof. For a given pair $(x_1, y_1) \neq (x_2, y_2)$ we have that the set $N(x_1, y_1, x_2, y_2) = \{\eta \in \mathbb{R}^d \mid f(x_1, y_1; \eta) - f(x_2, y_2; \eta) = 0\}$ is not all of $\mathbb{R}^d$, and thus since f is linear in η and in particular analytic, this set has measure zero. Taking the countable union over all $(x_1, y_1) \neq (x_2, y_2)$ we get a measure zero set N , such that if $\eta \notin N$, then $f(x_1, y_1; \eta) - f(x_2, y_2; \eta) \neq 0$ for all $(x_1, y_1) \neq (x_2, y_2)$, and as a result $f(x, y; \eta)$ is injective on $\mathcal{X} \times \mathcal{Y}$. $\square$

Finally, we can prove Theorem 4.7:

Proof of Theorem 4.7. We need to prove that we can choose the parameters $\eta^{(t)}, a^{(t)}$ and $b^{(t)}$ so that all functions in the MPNN procedure are maximally expressive. This can be done recursively using the claims we already proved. Firstly, since $\mathcal{X}$ is countable, By proposition 4.14 we can choose $a^{(1)}, b^{(1)}$ so that the first aggregation function is injective. Once these $a^{(1)}, b^{(1)}$ is fixed, the aggregation function has a countable amount of possible outputs, which we denote by the set $\mathcal{Y}^{(1)}$. Applying Proposition 4.15 on the domain $\mathcal{X} \times \mathcal{Y}^{(1)}$, we can choose $\eta^{(1)}$ so that the combine function is injective. The set of all possible outputs at this step is denoted by $\mathcal{X}^{(1)}$ is countable, and so we can again apply Proposition 4.14 and choose $a^{(2)}, b^{(2)}$ so that the second aggregation function is injective. We continue in this fashion until all parameters are chosen. $\square$

In [Amir et al., 2023] we also consider the case where $\mathcal{X}$ is taken to be all of $\mathbb{R}^d$, rather than a countable subset. Here, one has to develop some non-trivial tools to prove the following result:

Theorem 4.16 (Without proof, see [Amir et al., 2023]). *For $n, d \in \mathbb{N}$, denote $m = 2nd + 1$. Let $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be an analytic non-polynomial activation function. Then there exists a set of measure zero $Z \subseteq \mathbb{R}^{m \times d} \oplus \mathbb{R}^m$, such that for all $(A, b) \in \mathbb{R}^{m \times d} \oplus \mathbb{R}^m \setminus Z$, the function $F_\sigma^m(\cdot; A, b) : \mathcal{P}_n(\mathbb{R}^d) \rightarrow \mathbb{R}^m$ defined by*

$$F_\sigma^m(x_1, \dots, x_n; A, b) = \sum_{i=1}^n \sigma(Ax_i + b)$$

is injective.

This concludes our discussion of MPNN expressivity.

4.0.4 k -WL Tests and Graph Neural Networks

The somewhat embarrassing failure of MPNN to separate pairs of non-isomorphic regular graphs (like those in Figure 4.1(c)) inspired the extension of the WL test to a hierarchy of tests named the k -WL tests. Here $k = 1, 2, 3, \dots$ and larger k correspond to graph isomorphism test which are increasingly more powerful, and also increasingly more computationally expensive. The basic idea is that instead of updating node features as in the standard WL test, we will update features corresponding to k tuples of nodes.

Our presentation of k -WL loosely follows [Morris et al., 2023] which is also a good reference with nicer pictures than those showed here.

2-WL test Let us first explain how the 2-WL test works: for a graph $G = (V, E, X)$ with n nodes $V = \{1, \dots, n\}$, the 2-WL test is initialized by choosing for every pair of nodes $i, j \in \{1, \dots, n\}$ an initial feature which contains all pairwise information. We define this feature as:

$$\begin{aligned} C^{(0)}(i, j) &= (x_i, x_j, e_{i,j}), \text{ if } i \neq j \\ C^{(0)}(i, j) &= (x_i, x_i, *) \quad \text{if } i = j \end{aligned}$$

where $e_{i,j} = 1$ if (i, j) is an edge and zero otherwise, and $*$ is a special symbol. Next, in analogy to the standard WL test, we will want to aggregate features over the 'neighbors' of (i, j) . We define 'neighbors' in this context as pairs of indices (ℓ, m) where either $\ell = i$ or $k = m$ (note that, perhaps surprisingly, this definition has nothing to do with the graph). We then define the 'message passing' step via

$$\begin{aligned} D^{(t)}(i, j) &= \text{AGGREGATE}^{(t)} \left(\{C^{(t-1)}(i, \ell), C^{(t-1)}(\ell, j) \mid \ell = 1, \dots, n\} \right) \\ C^{(t)}(i, j) &= \text{COMBINE}^{(t)} \left(C^{(t-1)}(i, j), D^{(t)}(i, j) \right) \end{aligned}$$

this procedure is repeated T times, and then a final function on graphs is defined by aggregating all these features

$$F_2(G = (V, E, X)) = \text{READOUT} \left(\{C^{(t)}(i, j), \quad (i, j) \in [n] \times [n]\} \right)$$

As in the standard WL algorithm, the requirement from the $\text{AGGREGATE}^{(t)}$, $\text{COMBINE}^{(t)}$ and READOUT functions are that they are injective. Assuming the graph domain includes only features from a countable set, this can be done by choosing features in $\mathbb{N}$ as discussed for the WL test. This procedure is permutation invariant, in the sense that if G_1, G_2 are isomorphic then $F_2(G_1) = F_2(G_2)$. Conversely, if G, G' are not isomorphic, then F_2 may in some cases fail to separate, but these cases are rarer than the standard WL.

Example 4.17. For example, let's look at the standard counter example for the WL test discussed in 4.1(c), and show that the 2-WL can separate it in a single iterations. Let G denote the 6 cycle in that example, and $\hat{G}$ denote the graph obtained by a disjoint union of two 3-cycles, as shown in Figure 4.3. We order the nodes of G and $\hat{G}$ cyclically as shown in the figure, and the node features for all nodes are taken to be the same node feature, which we choose to be $x_v = -1$ and $\hat{x}_v = -1$ for all nodes in G and $\hat{G}$, respectively.)

The initialization of the pairwise features for G are as follows: for nodes $i < j$ we have

$$\begin{aligned} C^{(0)}(i, i+1) &= C^{(0)}(6, 1) = (-1, -1, 1), \quad \forall i = 1, \dots, 5 \\ C^{(0)}(i, j) &= (-1, -1, 0), \text{ if } 2 \leq j - i \leq 5 \end{aligned}$$

the initialization for (j, i) with $j > i$ is identical to the initialization for (i, j) . In other words, we have two different types of features which correspond to whether or not i, j are connected by an edge.

The pairs of identical indices (i, i) will have $C^{(0)}(i) = -1$ for all $i = 1, \dots, 6$.

If we consider the initialization of $\hat{G}$, we get the same type of initial features with the same multiplicities. For $i \neq j$,

$$\hat{C}^{(0)}(i, j) = \begin{cases} (-1, -1, 1) & \text{if } (i, j) \in \hat{E} \\ (-1, -1, 0) & \text{if } (i, j) \notin \hat{E} \end{cases}$$

and for pairs (i, i) we have $C^{(0)}(i) = -1$. Accordingly, $T = 0$ iterations are not sufficient to separate the inputs (the multisets of initialized features are distinct).

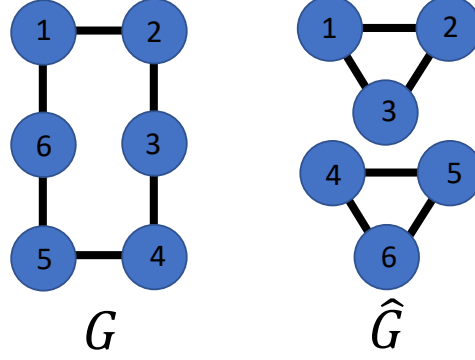


Figure 4.3: The classical example which cannot be separated by WL, can be separated by 2-WL.

For convenience, since the first two coordinates of all features in the zero-th iteration are $(-1, -1)$, let us remove all of these entries so that the pairwise features are now just a single number in $\{0, 1\}$ denoting whether or not (i, j) is an edge.

Now let us consider what happens in the $t = 1$ iteration, and focus on the indices $(1, 2)$, we have

$$\begin{aligned} D^{(1)}(1, 2) &= \text{AGGREGATE}^{(1)} \left(\{ (C^{(0)}(1, \ell), C^{(0)}(\ell, 2)) \mid \ell = 1, \dots, 6 \} \right) \\ &= \text{AGGREGATE}^{(1)} \left(\{ (*, 1)(1, *) (0, 1)(0, 0), (0, 0), (1, 0) \} \right) \end{aligned}$$

while

$$\begin{aligned} \hat{D}^{(1)}(1, 2) &= \text{AGGREGATE}^{(1)} \left(\{ (\hat{C}^{(0)}(1, \ell), \hat{C}^{(0)}(\ell, 2)) \mid \ell = 1, \dots, 6 \} \right) \\ &= \text{AGGREGATE}^{(1)} \left(\{ (*, 1)(1, *) (1, 1)(0, 0), (0, 0), (0, 0) \} \right) \end{aligned}$$

We see that the multisets defining $D^{(1)}(1, 2)$ and $\hat{D}^{(1)}(1, 2)$ are different- for example the 2WL procedure 'sees' that in G the nodes 1, 2 are not both connected to the same node, while in $\hat{G}$ they do (and therefore $(1, 1)$ appears in the multiset). Accordingly, if $D^{(1)}(1, 2) = a$ then $\hat{D}^{(1)}(1, 2) = b$. From symmetry considerations, it is clear that

$$D^{(1)}(i, j) = a \text{ and } \hat{D}^{(1)}(k, \ell) = b, \quad \forall (i, j) \in E \text{ and } (k, \ell) \in \hat{E}$$

We deduce that for all $(i, j) \in E$ we have that

$$C^{(1)}(i, j) = \text{COMBINE}^{(1)} \left(C^{(0)}(i, j), D^{(1)}(i, j) \right) = \text{COMBINE}^{(1)}(1, a) \neq \text{COMBINE}^{(1)}(1, b) = \hat{C}^{(1)}(1, 2)$$

Similarly, if $(i, j) \notin E$, then $C^{(0)}(i, j)$ is either 0 or $*$, and therefore

$$C^{(1)}(i, j) = \text{COMBINE}^{(1)} \left(C^{(0)}(i, j), D^{(1)}(i, j) \right) \neq \text{COMBINE}^{(1)}(1, b) = \hat{C}^{(1)}(1, 2).$$

We deduce that the multiset of all $C^{(1)}(i, j)$ features does not contain a single element equal to $\hat{C}^{(1)}(1, 2)$, and therefore $F_2(G) \neq F_2(\hat{G})$.

k -WL We now explain how to define the k -WL test for general $k \in \mathbb{N}$. It will be useful to denote a shortened notation for k tuples of indices:

$$\mathbf{i} = (i_1, \dots, i_k) \in [n]^k.$$

For each k tuple, we denote an initial feature

$$C^{(0)}(\mathbf{i}) = (x_{i_1}, \dots, x_{i_k}, A_*[\mathbf{i}]),$$

where A_* denotes the adjacency matrix with $*$ symbols in the diagonal to differentiate the diagonal entries from the non-diagonal entries, and $A_*[\mathbf{i}]$ is the $k \times k$ submatrix of the adjacency matrix corresponding to the indices in $\mathbf{i}$.

Neighbors of $\mathbf{i}$ will be k -tuples obtained from $\mathbf{i}$ by changing a single index. To define this exactly, for a given $j \in [n]$ define

$$\phi_1(\mathbf{i}, j) = (j, i_2, i_3, \dots, i_k), \phi_2(\mathbf{i}, j) = (i_1, j, i_3, \dots, i_k), \dots, \phi_k(\mathbf{i}, j) = (i_1, \dots, i_{k-1}, j)$$

and then

$$D^{(t)}(\mathbf{i}) = \text{AGGREGATE}^{(t)} \left(\left\{ \left[C^{(t-1)}(\phi_1(\mathbf{i}, j)), C^{(t-1)}(\phi_2(\mathbf{i}, j)), \dots, C^{(t-1)}(\phi_k(\mathbf{i}, j)) \right] \mid j \in [n] \right\} \right)$$

$$C^{(t)}(\mathbf{i}) = \text{COMBINE}^{(t)} \left(C^{(t-1)}(\mathbf{i}), D^{(t)}(\mathbf{i}) \right)$$

and after this is run for T iterations we define the final graph level function

$$F_k(G) = \text{READOUT} \left(\left\{ C^{(T)}(\mathbf{i}) \mid \mathbf{i} \in [n]^k \right\} \right)$$

Compatibility with standard WL We note that the case $k = 2$ corresponds exactly to our definition of 2-WL above (where the pairs of indices are denoted previously by (i, j) rather than $\mathbf{i}$). Note that our definition of k -WL is only for $k \geq 2$. While one could use the same definitions for $k = 1$, this would involve aggregating over all nodes and so is not equivalent to the standard WL algorithm aggregates only over neighbors. The term 1-WL is reserved for the standard WL algorithm.

k -WL variants There are actually two versions of k -WL tests for $k \geq 2$. Here we introduced the stronger version which is also sometimes called Folklore k -WL or k -FWL. The second version is called Oblivious k -WL or k -OWL. It is known that k -FWL is equivalent in separation power to $k + 1$ -OWL for all $k \geq 2$.

k -WL and graph neural networks Graph neural networks which are equivalent in expressive power to k -WL, can be obtained by replacing the aggregate, combine and readout function with differentiable parametric function. For example, based on k -GNNs from [Morris et al., 2019], we can define

$$D^{(t)}(\mathbf{i}) = \sum_{j \in [n]} \sigma \left(W_2^{(t)} \left[C^{(t-1)}(\phi_1(\mathbf{i}, j)), C^{(t-1)}(\phi_2(\mathbf{i}, j)), \dots, C^{(t-1)}(\phi_k(\mathbf{i}, j)) \right] + b_2^{(t)} \right)$$

$$C^{(t)}(\mathbf{i}) = W_1^{(t)} C^{(t-1)}(\mathbf{i}) + D^{(t)}(\mathbf{i})$$

and after T iterations define

$$F_k(G) = \sum_{\mathbf{i} \in V^k} \sigma \left(W_3 C^{(T)}(\mathbf{i}) + b_3 \right)$$

This is a graph neural network whose parameters are the entries of the matrices $W_1^{(t)}, W_2^{(t)}, W_3$ and bias vectors $b_1^{(t)}, b_2^{(t)}, b_3$. Any pair of graphs which cannot be separated by k -WL will not be separated by such a neural network. In the other direction, using similar considerations than those outlined for the derivation of maximally expressive MPNN, one can show that with analytic activations one can obtain a maximally expressive k -GNN, whose separation power is *equivalent* to the separation power of k -WL.

In general, the WL tests have become a standard benchmark to evaluating the expressive power of graph neural networks. For example, subgraph-GNN enhance the expressive power of MPNN by generating families of modified graphs from the original graph, and then applying MPNN to each of the modified graphs and aggregating the results. It is shown in [Zhang et al., 2023] that various subgraph-GNNs are more discriminative than 1-WL, but less discriminative than 2-WL. Similarly, expressive power of MPNN is often boosted by adding spectral information, using the eigendecomposition of the graph Laplacian. It is shown in [Zhang et al., 2024] that many spectral GNN are also bounded in expressivity between 1-WL and 2-WL (in this paper they talk about 3-WL, but they mean 3-OWL which is equivalent to our definition of 2-WL.)

4.0.5 Geometric graphs and point clouds

Geometric graphs are an important generalization of graphs. One setting where this comes up is for learning on molecules. Recall that in these applications, each graph node represents an atom of the molecule, and

node features x_v represent atom type (e.g, oxygen). In many tasks, it is useful to utilize the physical position of the atoms. For every atom v , this will be an additional feature $p_v \in \mathbb{R}^3$. In general, for natural n, d, q , a geometric graph will be of the form $G = (V, E, X, P)$, where $V = \{1, \dots, n\}$, E represents edges, $X = (x_1, \dots, x_n)$ with each $x_j \in \mathbb{R}^d$ represents 'standard' node features, and $P = (p_1, \dots, p_n)$ with each $p_v \in \mathbb{R}^q$, represented 'position' features. We denote the set of all geometric graphs with n nodes, d features and q -dimensional features by $\mathcal{G}(n, d, q)$. Typically in applications $q = 3$.

Geometric graphs introduce new symmetries we need to take into account: besides the permutation symmetry already discussed, there is also symmetry to rigid motion, since applying rigid motions to the molecules positions should not change any quantities we are interested in.

To defined this properly, we first denote the group of rigid motions by

$$E(q) = \{(R, t) | R \in O(q), t \in \mathbb{R}^q\}$$

where we recall that $O(q)$ is the group of orthogonal transformations, that is, matrices $R \in \mathbb{R}^{q \times q}$ satisfying that $RR^T = I_q$. Rigid motions are transformations which preserve Euclidean distances. Namely, if we denote

$$(R, t).p = Rp + t$$

then we have that for all $p, p' \in \mathbb{R}^q$,

$$\|(Rp + t) - (Rp' + t)\| = \|R(p - p')\| = \|p - p'\|.$$

The product in this group is denoted by

$$(Q, s) \times (R, t) = (QR, s + Qt).$$

The reason for this choice is that it makes $(R, t).q$ a proper group action. Indeed, note that

$$(Q, s).(R, t).p = Q.s(Rp + t) = QRp + Qt + s = (QR, s + Qt).p = ((Q, s) \cdot (R, t)).p.$$

the action of $E(q) \times S_n$ on the space of geometric graphs with n nodes is defined as

$$(R, t, \tau)(A, X, P) = (\tau(A), \tau(X), \tau((R, t).P))$$

where the action of τ on A is defined by permuting both indices simultaneously, while the action on matrices $X \in \mathbb{R}^{d \times n}$ and $P \in \mathbb{R}^{q \times n}$ is defined by permuting only the second index.

A **Geometric Graph Neural Network** will be a function on spaces of geometric graphs $\bigcup_{n \in \mathbb{N}} \mathcal{G}(n, d, q)$: that is, we want to think of graphs with possibly varying amount of nodes, but we can assume d, q are known and fixed. This function will be required to be invariant (or equivariant) to the action of $O(q) \times S_n$ on geometric graphs. One of the simplest ways to do this is Geometric Message Passing Neural Networks, which use standard message passing, and take positional information into account using pairwise distances. A Geometric MPNN is defined as follows:

Geometric MPNN We initialize by choosing the standard node features

$$x_v^{(0)} = x_v$$

as in MPNN. We then recursively define $x_v^{(t)}$ from $x_v^{(t-1)}$ via

$$\begin{aligned} y_v^{(t)} &= \text{AGGREGATE}^{(t)} \left(\{x_u^{(t-1)}, \|p_u - p_v\|, u \in \mathcal{N}_v\} \right) \\ x_v^{(t)} &= \text{COMBINE}^{(t)} \left(x_v^{(t-1)}, y_v^{(t)} \right). \end{aligned}$$

This procedure is applied T times, and a final invariant function is obtained via

$$F(G) = \text{READOUT} \left(\{x_v^{(T)}, v \in [n]\} \right)$$

The only difference from standard MPNN is that in the aggregation step we also include pairwise distances. Using the fact that the pairwise distance are preserved under rigid motions, it is not difficult to show that F is invariant to the action of $O(q) \times S_n$.

As always, Geometric MPNN is a framework rather than a specific architecture. Such an architecture can be obtained by specific choices of aggregate, combine and readout functions. Examples of Geometric MPNN include well known models such as SchNet [Schütt et al., 2017], DimeNet [Gasteiger et al., 2021] and SphereNet [Liu et al., 2022].

Point clouds Point clouds are a degenerate but important case of geometric MPNN. Namley, a point cloud is just a matrix $P \in \mathbb{R}^{q \times n}$ denoting positions. Such data comes up when positional data is the only data available. In the context of point clouds, we also want to define functions which are invariant to $E(q) \times S_n$. This can be done by identifying the point cloud P with a degenerate geometric graph $G(P) = (V, E, X, P)$ where

$$V = \{1, \dots, n\}, E = V \times V, X = (0, 0, \dots, 0).$$

Expressivity analysis

As we have done before, it is natural to inquire what geometric graphs can be separated by geometric MPNN? If all non-equivalent graphs can be separated, then we can obtain universality. As we have done before, we will call a geometric MPNN *maximally expressive* if all aggregation, combine and readout functions are injective. The existence of abstract maximally expressive geometric MPNN can be guaranteed by group theory considerations, as long as one chooses the features to reside in a set of cardinality $\aleph$ throughout the procedure (e.g., features in $\mathbb{R}$ or in $\mathbb{R}^N$). Differentiable 'neural' maximally expressive constructions are also possible, see e.g., [Hordan et al., 2024].

Reduction to Combinatorial Graphs One observation which can be easily made is that any combinatorial graph $G = (V, E, X)$ can be identified with a trivial geometric graph $G_0 = (V, E, X, 0_{q \times n})$, in the sense that a pair of graphs G, G' are isomorphic as combinatorial graphs, if and only if G_0 and G'_0 are isomorphic as geometric graphs. Due to this observation, it is clear that geometric MPNN cannot separate all geometric graphs, as they cannot separate all combinatorial graphs, like the pair in Figure 4.3.

Point Clouds separation When focusing on point clouds, one could ask the following question: for fixed q, n , can geometric MPNNs separate between all pairs of non-isomorphic point clouds? A relevant statement is the following proposition which you proved in hw5:

Proposition 4.18. *If $P, P' \in \mathbb{R}^{q \times n}$ and*

$$\|p_i - p_j\| = \|p'_i - p'_j\|, \quad \forall 1 \leq i < j \leq n$$

then there exists a rotation $R \in O(q)$ and translation $t \in \mathbb{R}^q$ such that

$$p_i = Rp'_i + t, \quad \forall i = 1, \dots, n$$

In other words, P, P' are related by a rigid motion if and only if all pairwise distances are equal. This gives us a simple set of complete invariants with which to check equivalence to rigid motions. However, the question of completeness with respect to the joined action of $E(q) \times S_n$ is more complex. One can immediately obtain from Proposition 4.18 the following conclusion

Problem 4.19. Point clouds P, P' are related by a $E(q) \times S_n$ transformation, if and only if there exists a permutation $\tau \in S_n$ such that

$$\|p_{\tau(i)} - p_{\tau(j)}\| = \|p'_i - p'_j\|, \quad \forall 1 \leq i < j \leq n$$

In other words, checking equivalence of point clouds with respect to $E(q) \times S_n$, can be reduced to checking equivalence of distance matrices up to the action of S_n . However, the action of S_n on distance matrices is the action of permutations on graphs, for which separation results are more challenging than for the action of permutations on multisets, which is the relevant action for point clouds. On the other hand, distance matrices are special types of weighted graphs for which, perhaps, separation results may be easier to obtain. We will mention some results in this regard.

We first show that geometric MPNN provide separation for 'most' point clouds. Denote

$$\mathbb{R}_{\text{distinct}}^{q \times n} = \{P \in \mathbb{R}^{q \times n} \mid \text{if for } i \neq j \text{ and } r \neq s, \quad \|p_i - p_j\| = \|p_r - p_s\| \text{ then } \{i, j\} = \{r, s\}\}$$

We first claim that 'most' point clouds are in $\mathbb{R}_{\text{distinct}}^{q \times n}$.

Proposition 4.20. *For every natural $n \geq 2, q \geq 1$, the complement of $\mathbb{R}_{\text{distinct}}^{q \times n}$ in $\mathbb{R}^{q \times n}$ has measure zero.*

Proof. For all $i \neq j$ and $r \neq s$ such that $\{i, j\} \neq \{r, s\}$ we define the set

$$F[i, j, r, s] = \{P \in \mathbb{R}^{q \times n} \mid \|p_i - p_j\|^2 - \|p_r - p_s\|^2 = 0\},$$

and note that

$$\mathbb{R}^{q \times n} \setminus \mathbb{R}_{\text{distinct}}^{q \times n} = \bigcup_{\{i, j\} \neq \{r, s\}} F[i, j, r, s]$$

It is thus sufficient to show that each $F[i, j, r, s]$ has measure zero. Since this set is defined by a polynomial equation, by Proposition 4.13 it is of measure zero, or equals all of $\mathbb{R}^{q \times n}$. It is thus sufficient to show

Problem 4.21. Let $n \geq 2$ and q be natural numbers. For given $i \neq j$ and $r \neq s$ such that $\{i, j\} \neq \{r, s\}$, show that there exists a $P \in \mathbb{R}^{q \times n}$ which is not in $F[i, j, r, s]$. □

Next, we make the following claim:

Proposition 4.22. *For natural numbers n, q , Let F be a maximally expressive geometric MPNN with $T = 1$ iterations. Then for all $\hat{P} \in \mathbb{R}_{\text{distinct}}^{q \times n}$ and $P \in \mathbb{R}^{q \times n}$, we have that $F(P) = F(\hat{P})$ if and only if P and $\hat{P}$ are related by a permutation and rigid motion.*

Proof. Since $F(G(P)) = F(G(\hat{P}))$ we can without loss of generality permute the nodes of $\hat{P}$ so that

$$x_i^{(1)} = \hat{x}_i^{(1)}, \forall i \in [n]$$

which implies that

$$y_i^{(1)} = \hat{y}_i^{(1)}, \forall i \in [n].$$

Fix some $r \neq i$. Then the latter equality implies that there exists some $j \neq i$ such that

$$\|p_i - p_r\| = \|\hat{p}_i - \hat{p}_j\|.$$

Similarly, the equality $y_r^{(1)} = \hat{y}_r^{(1)}$ implies that there exists some $s \neq r$ such that

$$\|p_i - p_r\| = \|\hat{p}_r - \hat{p}_s\|.$$

These two equalities imply the equality of two pairs in $\hat{P}$, and since $\hat{P}$ is in $\mathbb{R}_{\text{distinct}}^{q \times n}$ these two pairs must be identical, namely $\{i, j\} = \{r, s\}$. Since $i \neq r$ this implies that $i = s, r = j$ and so

$$\|p_i - p_r\| = \|\hat{p}_i - \hat{p}_r\|.$$

This is true for all $i \neq r$, and so by question 4 in homework 5, P and $\hat{P}$ are related by a $E(q)$ transformation. □

Proposition 4.22 is slightly stronger than the statement that F is injective, up to symmetries, on $\mathbb{R}_{\text{distinct}}^{q \times n}$. We will call this type of property *generic injectivity*.

It can be interesting to compare geometric MPNN with a simpler invariant function for point clouds, which just looks at the distribution of distances. Namely, we can define a function F_{dist} on $\mathbb{R}^{q \times n}$ via

$$F_{\text{dist}}(P) = \mathbf{sort}(\{\|p_i - p_j\|, 1 \leq i < j \leq n\})$$

The function F_{dist} is $E(q) \times S_n$ invariant. It can also be shown to satisfy generic injectivity, though we will not give the proof here:

Proposition 4.23 (Theorem 1.6 in [Boutin and Kemper, 2004]). *For natural n, q with $n \geq q + 2$, there exists a subset M of $\mathbb{R}^{q \times n}$ whose complement is of measure zero, such that for all $P' \in M$ and $P \in \mathbb{R}^{q \times n}$, we have that $F_{\text{dist}}(P') = F_{\text{dist}}(P)$ if and only if P and P' are related by a permutation and rigid motion.*

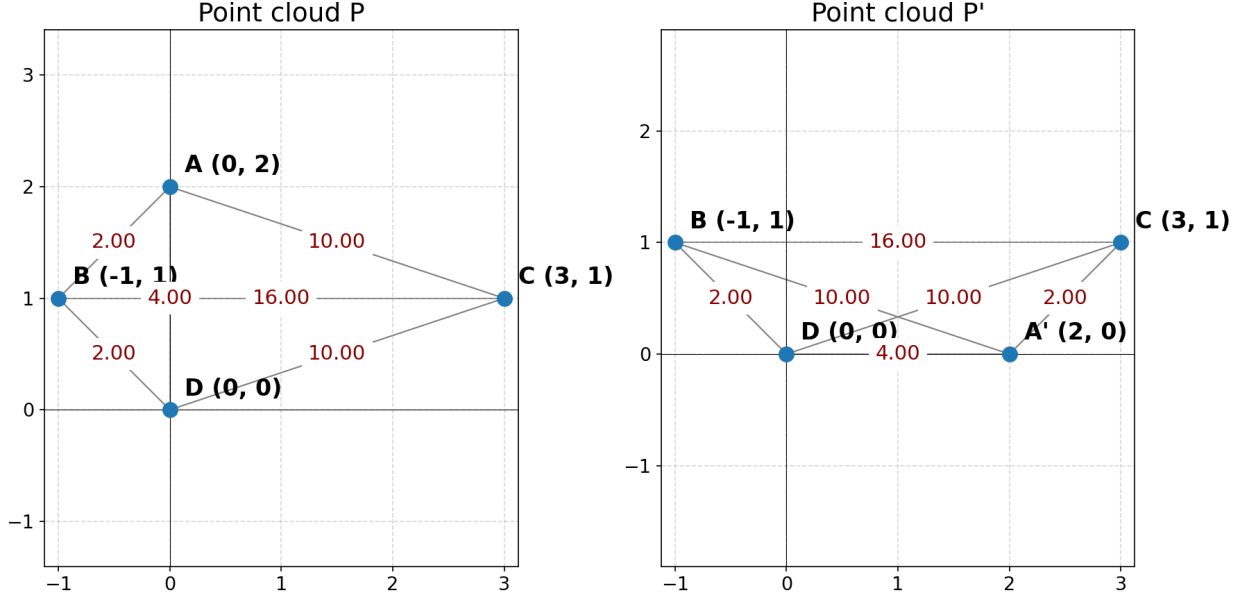


Figure 4.4: Point clouds P, P' which have the same distance distribution, but can be separated by a single geometric MPNN iteration. The figure shows P, P' and the squared distances between the points.

In general, F_{dist} can be shown to be less expressive than maximally expressive geometric MPNN, namely, if $F_{dist}(P) \neq F_{dist}(P')$ then geometric MPNN can separate P and P' as well. Conversely, Figure 4.4 shows a pair of point clouds P, P' in $\mathbb{R}^{2 \times 4}$ with

$$F_{dist}(P) = (\sqrt{2}, \sqrt{2}, \sqrt{4}, \sqrt{10}, \sqrt{10}, \sqrt{16}) = F_{dist}(P')$$

However, a single iteration of a geometric MPNN does separate P from P' . To see this, note that the feature $y_C^{(1)}$ obtained in the first message passing iteration for the point cloud P at the point C , is given by

$$y_C^{(1)} = \text{AGGREGATE}^{(1)}(\{\sqrt{10}, \sqrt{10}, \sqrt{16}\})$$

Looking at the point cloud P' , we see that no point is of distance $\sqrt{10}$ from two other points. It follows that, if we denote the outputs of the message passing iterations applied to P' using $\hat{y}, \hat{x}$ etc., we have that

$$\hat{y}_v^{(1)} \neq y_C^{(1)}, \forall v \in \{A', B, C, D\}, \text{ and therefore } \hat{x}_v^{(1)} \neq x_C^{(1)}, \forall v \in \{A', B, C, D\}.$$

It follows that

$$\text{READOUT}(\{\{x_v^{(1)} \mid v \in \{A, B, C, D\}\}\}) \neq \text{READOUT}(\{\{\hat{x}_v^{(1)} \mid v \in \{A', B, C, D\}\}\})$$

and so the geometric MPNN can separate P from P' .

We conclude by mentioning two interesting results, without proof.

1. For $q \geq 3$, Geometric MPNN cannot separate all non-isomorphic point clouds. Examples of non-equivalent P, P' in three dimensions which cannot be separated by geometric-MPNN are given in [Pozdnyakov and Ceriotti, 2022, Li et al., 2023].
2. For $q = 2$, Geometric MPNN can separate all non-isomorphic point clouds. For $q = 3$ a geometric 2-WL analogue of geometric MPNN can separate all non-isomorphic point clouds, and in general geometric $(q - 1)$ WL is sufficient to separate between q dimensional point clouds. This is shown in [delle Rose et al., 2023].

4.0.6 Generic recovery for sparse graphs

Generic recovery for geometric MPNN happens when all pairwise distances are taken into account. For geometric graphs where the graph may be sparse, the situation is very different. For example, we have the following simple result which is self explanatory:

Proposition 4.24. *Let $V = \{1, 2, 3\}$ and $E = \{(1, 2), (2, 3)\}$, $X = (0, 0, 0)$. Let $P, P' \in \mathbb{R}^{2 \times 3}$ satisfying*

$$\|p_1 - p_2\| = \|p'_1 - p'_2\|, \quad \|p_3 - p_2\| = \|p'_3 - p'_2\|.$$

Denote $G = (V, E, X, P)$ and $G' = (V, E, X, P')$. Then for any geometric MPNN F , we have that $F(G) = F(G')$.

An illustration of these two geometric graphs is given in Figure 4.5. The situation in Proposition 4.24 is very different than generic recovery. In fact, for the fixed (V, E, X) in the proposition, for 'most' P , we can find P' such that the geometric graphs G, G' are not isomorphic, but cannot be separated by geometric MPNN. This is not true for all P , because if $p_1 = p_2$ or $p_2 = p_3$, then the geometric graph G' satisfying Proposition 4.24 will be isomorphic to G .

4.0.7 Equivariant Geometric MPNN

Equivariant Geometric MPNN are a variant of geometric MPNN which is very popular. The difference is that besides the features $x_j^{(t)}$ which are updated in the message passing, there are also 'equivariant' features $p_j^{(t)}$, where equivariance means that applying a rigid motion to the input leads to the same rigid motion being applied to the features $p_j^{(t)}$. In [Joshi et al., 2023, Sverdllov and Dym,] it was observed that for graphs which are not fully connected, it has much better separation abilities, and in particular it recovers the generic recovery properties which geometric MPNN lack.

First, let us give an example of a geometric MPNN: **EGNN** [Satorras et al., 2021] is a well known equivariant geometric MPNN. It can be defined via the following recursive procedure

$$m_{ij}^{(t)} = \phi_e^{(t)}(x_i^{(t-1)}, x_j^{(t-1)}, \|p_i^{(t-1)} - p_j^{(t-1)}\|^2) \quad (4.9)$$

$$p_i^{(t)} = p_i^{(t-1)} + \sum_{j \in \mathcal{N}(i)} (p_i^{(t-1)} - p_j^{(t-1)}) \phi_p^{(t)}(m_{ij}^{(t)}) \quad (4.10)$$

$$m_i^{(t)} = \sum_{j \in \mathcal{N}(i)} m_{ij}^{(t)} \quad (4.11)$$

$$x_i^{(t)} = \phi_x^{(t)}(x_i^{(t-1)}, m_i^{(t)}). \quad (4.12)$$

$$(4.13)$$

A final invariant function can be obtained by aggregating the invariant node features

$$F_{\text{EGNN}}(G) = \sum_{j=1}^n x_j^{(T)}$$

The functions $\phi_e^{(t)}, \phi_p^{(t)}, \phi_x^{(t)}, \phi_{out}$ are taken to be 'standard' neural networks, and in what follows we will allow ourselves to choose any continuous function for simplicity.

Why is this helpful? We show this at least for example in Figure 4.5:

Proposition 4.25. *Let $V = \{1, 2, 3\}$ and $E = \{(1, 2), (2, 3)\}$, $X = (0, 0, 0)$. Let $P, P' \in \mathbb{R}^{2 \times 3}$ satisfying*

$$\|p_1 - p_2\| = \|p'_1 - p'_2\| = a, \quad \|p_3 - p_2\| = \|p'_3 - p'_2\| = b,$$

and assume that $a \neq b$ and $\|p_1 - p_3\| \neq \|p'_1 - p'_3\|$. Denote $G = (V, E, X, P)$ and $G' = (V, E, X, P')$. Then there exists an instantiation of EGNN with $T = 2$ such that $F_{\text{EGNN}}(G) \neq F_{\text{EGNN}}(G')$

Proof. Choose $\phi_e^{(t)}(a, b, c) = \sqrt{c}$ so that

$$m_{ij}^{(t)} = \|p_i^{(t-1)} - p_j^{(t-1)}\|.$$

Choose $\phi_p^{(1)}$ so that $\phi_p^{(1)}(\|p_1 - p_2\|) = 0$ and $\phi_p^{(1)}(\|p_2 - p_3\|) = -1$. Then

$$p_1^{(1)} = p_1, p_2^{(1)} = p_3, p_3^{(1)} = p_2$$

and it follows that

$$m_{12}^{(2)} = \|p_1^{(1)} - p_2^{(1)}\| = \|p_1 - p_3\|$$

while

$$m_{23}^{(2)} = \|p_3 - p_2\|.$$

We deduce that by taking $\phi_x^{(t)}(x, m) = m$ and $\phi_{out}(x) = x$, we obtain

$$F_{\text{EGNN}}(G) = m_1^{(2)} + m_2^{(2)} + m_3^{(2)} = 2\|p_1 - p_3\| + 2\|p_2 - p_3\|$$

If we would apply the same EGNN function to G' we would obtain a different result

$$F_{\text{EGNN}}(G') = 2\|p'_1 - p'_3\| + 2\|p'_2 - p'_3\| \neq 2\|p_1 - p_3\| + 2\|p_2 - p_3\|$$

□

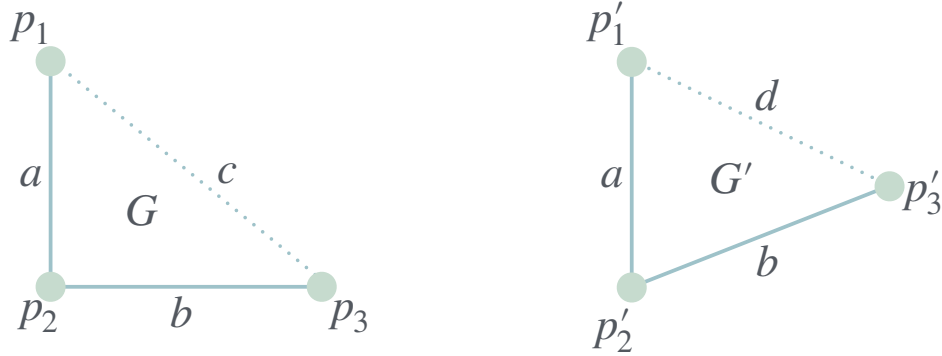


Figure 4.5: Geometric graphs G, G' which cannot be separated by geometric MPNN.

A much more general statment is possible: we state it here somewhat informally

Theorem 4.26 (Informal, see [Sverdlov and Dym,]). *Let $G = (V, E)$ be a **connected** combinatorial graph with n nodes, and take $X = (0, 0, \dots, 0)$. Then there exists an equivariant geometric MPNN F , and a set $M \subseteq \mathbb{R}^{q \times n}$ whose complement is of measure zero, such that, for all $P' \in M$ and $P \in \mathbb{R}^{q \times n}$, $F(G) = F(G')$ if and only if $G = (V, E, X, P)$ and $G' = (V, E, X, P')$ are isomorphic as geometric graphs.*

Bibliography

- [Amir et al., 2023] Amir, T., Gortler, S. J., Avni, I., Ravina, R., and Dym, N. (2023). Neural injective functions for multisets, measures and graphs via a finite witness theorem.
- [Babai, 2016] Babai, L. (2016). Graph isomorphism in quasipolynomial time.
- [Bieber,] Bieber, D. David bieber blog on wl isomorphism test. <https://davidbieber.com/post/2019-05-10-weisfeiler-lehman-isomorphism-test/>.
- [Boutin and Kemper, 2004] Boutin, M. and Kemper, G. (2004). On reconstructing n-point configurations from the distribution of distances or areas. *Advances in Applied Mathematics*, 32(4):709–735.
- [Daubechies et al., 2021] Daubechies, I., DeVore, R., Foucart, S., Hanin, B., and Petrova, G. (2021). Non-linear approximation and (deep) relu networks. *Constructive Approximation*, pages 1–46.
- [delle Rose et al., 2023] delle Rose, V., Kozachinskiy, A., Rojas, C., Petrache, M., and Barcelo, P. (2023). Three iterations of $(d - 1)$ -WL test distinguish non isometric clouds of d-dimensional points. In *Thirty-seventh Conference on Neural Information Processing Systems*.
- [DeVore et al., 2021] DeVore, R., Hanin, B., and Petrova, G. (2021). Neural network approximation. *Acta Numerica*, 30:327–444.
- [Dym et al., 2020] Dym, N., Sober, B., and Daubechies, I. (2020). Expression of fractals through neural network functions. *IEEE Journal on Selected Areas in Information Theory*, 1(1):57–66.
- [Eldan and Shamir, 2016] Eldan, R. and Shamir, O. (2016). The power of depth for feedforward neural networks. In *Conference on learning theory*, pages 907–940. PMLR.
- [Gasteiger et al., 2021] Gasteiger, J., Yeshwanth, C., and Günnemann, S. (2021). Directional message passing on molecular graphs via synthetic coordinates. In *Proceedings of the 35th International Conference on Neural Information Processing Systems, NIPS ’21*, Red Hook, NY, USA. Curran Associates Inc.
- [Hamilton et al., 2017] Hamilton, W., Ying, Z., and Leskovec, J. (2017). Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30.
- [Hanin and Rolnick, 2019] Hanin, B. and Rolnick, D. (2019). Deep relu networks have surprisingly few activation patterns. *Advances in neural information processing systems*, 32.
- [Hordan et al., 2024] Hordan, S., Amir, T., Gortler, S. J., and Dym, N. (2024). Complete neural networks for complete euclidean graphs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 12482–12490.
- [Joshi et al., 2023] Joshi, C. K., Bodnar, C., Mathis, S. V., Cohen, T., and Liò, P. (2023). On the expressive power of geometric graph neural networks. *arXiv preprint arXiv:2301.09308*.
- [Krantz and Parks, 2002] Krantz, S. G. and Parks, H. R. (2002). *A primer of real analytic functions*. Springer Science & Business Media.

- [Lee et al., 2019] Lee, J., Lee, Y., Kim, J., Kosiorek, A., Choi, S., and Teh, Y. W. (2019). Set transformer: A framework for attention-based permutation-invariant neural networks. In *International conference on machine learning*, pages 3744–3753. PMLR.
- [Li et al., 2023] Li, Z., Wang, X., Huang, Y., and Zhang, M. (2023). Is distance matrix enough for geometric deep learning? In *Thirty-seventh Conference on Neural Information Processing Systems*.
- [Liu et al., 2022] Liu, Y., Wang, L., Liu, M., Lin, Y., Zhang, X., Oztekin, B., and Ji, S. (2022). Spherical message passing for 3d molecular graphs. In *International Conference on Learning Representations*.
- [Lu et al., 2021] Lu, J., Shen, Z., Yang, H., and Zhang, S. (2021). Deep network approximation for smooth functions. *SIAM Journal on Mathematical Analysis*, 53(5):5465–5506.
- [Malach and Shalev-Shwartz, 2019] Malach, E. and Shalev-Shwartz, S. (2019). Is deeper better only when shallow is good? *Advances in Neural Information Processing Systems*, 32.
- [Mityagin, 2015] Mityagin, B. (2015). The zero set of a real analytic function. *arXiv preprint arXiv:1512.07276*.
- [Morris et al., 2023] Morris, C., Lipman, Y., Maron, H., Rieck, B., Kriege, N. M., Grohe, M., Fey, M., and Borgwardt, K. (2023). Weisfeiler and leman go machine learning: the story so far. *J. Mach. Learn. Res.*, 24(1).
- [Morris et al., 2019] Morris, C., Ritzert, M., Fey, M., Hamilton, W. L., Lenssen, J. E., Rattan, G., and Grohe, M. (2019). Weisfeiler and leman go neural: higher-order graph neural networks. In *Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence and Thirty-First Innovative Applications of Artificial Intelligence Conference and Ninth AAAI Symposium on Educational Advances in Artificial Intelligence*, AAAI’19/IAAI’19/EAAI’19. AAAI Press.
- [Pinkus, 1999] Pinkus, A. (1999). Approximation theory of the mlp model in neural networks. *Acta numerica*, 8:143–195.
- [Pozdnyakov and Ceriotti, 2022] Pozdnyakov, S. N. and Ceriotti, M. (2022). Incompleteness of graph neural networks for points clouds in three dimensions. *Machine Learning: Science and Technology*, 3.
- [Qi et al., 2017] Qi, C. R., Su, H., Mo, K., and Guibas, L. J. (2017). Pointnet: Deep learning on point sets for 3d classification and segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 652–660.
- [Reiser et al., 2022] Reiser, P., Neubert, M., Eberhard, A., Torresi, L., Zhou, C., Shao, C., Metni, H., van Hoesel, C., Schopmans, H., Sommer, T., et al. (2022). Graph neural networks for materials science and chemistry. *Communications Materials*, 3(1):93.
- [Satorras et al., 2021] Satorras, V. G., Hoogeboom, E., and Welling, M. (2021). E(n) equivariant graph neural networks. In *International Conference on Machine Learning*.
- [Schütt et al., 2017] Schütt, K. T., Kindermans, P.-J., Sauceda, H. E., Chmiela, S., Tkatchenko, A., and Müller, K.-R. (2017). Schnet: a continuous-filter convolutional neural network for modeling quantum interactions. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS’17, page 992–1002, Red Hook, NY, USA. Curran Associates Inc.
- [Segol and Lipman, 2019] Segol, N. and Lipman, Y. (2019). On universal equivariant set networks. In *International Conference on Learning Representations*.
- [Simmons, 1963] Simmons, G. F. (1963). *Introduction to topology and modern analysis*, volume 44. Tokyo.
- [Sverdlov and Dym,] Sverdlov, Y. and Dym, N. On the expressive power of sparse geometric mpnns. In *The Thirteenth International Conference on Learning Representations*.

- [Telgarsky, 2016] Telgarsky, M. (2016). Benefits of depth in neural networks. In *Conference on learning theory*, pages 1517–1539. PMLR.
- [Villar et al., 2021] Villar, S., Hogg, D. W., Storey-Fisher, K., Yao, W., and Blum-Smith, B. (2021). Scalars are universal: Equivariant machine learning, structured like classical physics. *Advances in Neural Information Processing Systems*, 34:28848–28863.
- [Xu et al., 2018] Xu, K., Hu, W., Leskovec, J., and Jegelka, S. (2018). How powerful are graph neural networks? In *International Conference on Learning Representations*.
- [Yarotsky, 2017] Yarotsky, D. (2017). Error bounds for approximations with deep relu networks. *Neural Networks*, 94:103–114.
- [Zaheer et al., 2017] Zaheer, M., Kottur, S., Ravanbakhsh, S., Poczos, B., Salakhutdinov, R. R., and Smola, A. J. (2017). Deep sets. *Advances in neural information processing systems*, 30.
- [Zhang et al., 2023] Zhang, B., Feng, G., Du, Y., He, D., and Wang, L. (2023). A complete expressiveness hierarchy for subgraph gnns via subgraph weisfeiler-lehman tests. In *International Conference on Machine Learning*.
- [Zhang et al., 2024] Zhang, B., Zhao, L., and Maron, H. (2024). On the expressive power of spectral invariant graph neural networks. In Salakhutdinov, R., Kolter, Z., Heller, K., Weller, A., Oliver, N., Scarlett, J., and Berkenkamp, F., editors, *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 60496–60526. PMLR.